开发者社区

DTK

DTK 26.10
HIP C++编程指南

DTK-261000-012-0915-927

目 录

1 编写第一个 DCU 程序 1
1.1 HIP 编程实战-数组相加 1
1.2 总结 6
2 HIP 核函数 9
2.1 线程执行模型 9
2.1.1 线程束和线程块 9
2.1.2 线程执行 12
2.1.3 线程束分化 12
2.1.4 关于线程调度和执行模型大小的优化 13
2.1.5 同步 14
2.2 可编程存储结构 14
2.2.1 存储结构简介 15
2.2.2 共享内存 16
2.3 HIP 修饰符 18
2.3.1 存储修饰符 18
2.3.2 函数修饰符 19
2.4 内置函数和数据类型 20
2.4.1 标准数学函数 20
2.4.2 快速数学函数 22
2.4.3 集合函数 27
2.4.4 线程束洗牌函数 29
2.4.5 原子函数 34
2.4.6 内置数据类型 37
2.5 编译与运行 38
2.6 总结 41
3 全局内存管理与数据传输 42
3.1 全局内存 42
3.2 页锁定内存 44

开发者社区

3.3 内存亲和性.....47
3.4 数据传输.....48
3.5 总结.....51
4 HIP 并发执行.....52
4.1 HIP 流.....52
4.1.1 什么是流.....52
4.1.2 流的执行模型.....53
4.1.3 使用流.....56
4.1.4 事件.....58
4.1.5 流的同步.....60
4.1.6 流回调.....62
4.2 流的异步并发执行实例.....63
4.3 总结.....66
4.4 hipGraph.....66
4.4.1 hipGraph 简介.....66
4.4.2 函数接口说明.....67
4.4.3 示例.....76
5 HIP 设备管理.....87
5.1 HIP 设备查询与选择.....87
5.2 总结.....90
6 单进程多 DCU 编程.....91
6.1 多 DCU 操作.....91
6.2 多 DCU 异步执行.....93
6.3 DCU 间数据传输与通信.....95
6.4 多 DCU 同步方法.....101
6.5 总结.....103
7 性能分析.....104
7.1 使用 hipprof 分析程序.....104
7.2 hiptx-trace 功能.....110
开发者社区

7.3 多节点与多进程性能优化.....112
7.4 总结.....115

开发者社区

1 编写第一个 DCU 程序

1.1 HIP 编程实战-数组相加

学习 CPU 平台上的编程语言，我们第一个程序一般会是“Hello world!”。鉴于 DCU 多任务数据处理架构的特点，我们写个与众不同的第一个 DCU 程序——两数组相加，也即数组 A 与数组 B 进行相加，结果赋值给数组 C。这是个足够简单的程序，但这个程序能让你掌握 DCU 编程的基本方法以及了解 DCU 编程和 CPU 编程之间的重要差别，还有编译运行方法。

- **CPU 代码和编译**

首先，我们看下 CPU 平台 C 语言版数组相加的代码，是一个 for 循环的简单程序。

```
#include <stdlib.h>
#define N 10000
int main() {
   float *A = (float *) malloc(N * sizeof(float));
   float *B = (float *) malloc(N * sizeof(float));
   float *C = (float *) malloc(N * sizeof(float));
   for (int i = 0; i < N; i++) {
      A[i] = 1;
      B[i] = 1;
      C[i] = 0;
   }
      for (int i = 0; i < N; i++) {
      C[i] = A[i] + B[i];
   }
   free(A);
   free(B);
   free(C);
   return 0;
```

```
}
```

对 CPU 的程序可以采用 gcc 编译器进行编译，假如你将程序命名为 test.cpp，在 Linux 平台下使用下面的命令进行程序编译，运行就可以看到 C 数组内的值都被置为了 2，数组相加完成。

```
gcc  vector_add_cpu.cpp -o vector_add_cpu
```

- DCU 代码

接下来，我们看下 DCU 版程序会是什么样的，我们来看下面的代码。大家先不用着急看明白每一行代码的意思，后面会详细的介绍。

```
#include <iostream>
#include "hip/hip_runtime.h"
#include <hip/hip_runtime.h>
#define N 10000
__global__ void add(float *d_A, float *d_B, float *d_C) {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    if (tid < N) {
        d_C[tid] = d_A[tid] + d_B[tid];
    }
}
int main() {
    float *A = (float *) malloc(N * sizeof(float));
    float *B = (float *) malloc(N * sizeof(float));
    float *C = (float *) malloc(N * sizeof(float));
    float *d_A = NULL;
    float *d_B = NULL;
    float *d_C = NULL;
    hipMalloc((void **) &d_A, N * sizeof(float));
    hipMalloc((void **) &d_B, N * sizeof(float));
    hipMalloc((void **) &d_C, N * sizeof(float));
    for (int i = 0; i < N; i++) {
```

```
    A[i] = 1;
    B[i] = 1;
    C[i] = 0;
  }
  hipMemcpy(d_A, A, sizeof(float) * N, hipMemcpyHostToDevice);
  hipMemcpy(d_B, B, sizeof(float) * N, hipMemcpyHostToDevice);
  hipMemcpy(d_C, C, sizeof(float) * N, hipMemcpyHostToDevice);
  dim3 blocksize(256, 1);
  dim3 gridsize(N / 256 + 1, 1);
  add<<<gridsize, blocksize >>> (d_A, d_B, d_C);
  hipMemcpy(C, d_C, sizeof(float) * N, hipMemcpyDeviceToHost);
  for (int i = 0; i < N; i++) {
    std::cout << C[i] << std::endl;
  }
  free(A);
  free(B);
  free(C);
  hipFree(d_A);
  hipFree(d_B);
  hipFree(d_C);
}
```

- DCU 程序编译

对 DCU 的程序需要采用专用的编译器 hipcc 进行编译，编译出的二进制代码可以被设备端加载运行，编译命令与 gcc 相似，同上编译后运行，如果你成功了，应该会在屏幕当中看到 2 这个数字不断被打印。

```
hipcc vector_add_dcu.cpp -o vector_add_dcu
```

恭喜你，到这里你已经运行了第一个 DCU 的程序，并且了解了 DCU 编译和 CPU 编译方法的不同，抛开代码来看，似乎二者并没有什么区别，仅仅是更换了编译器。

- 查看 DCU 程序运行

我们如何用简单方法确定数组相加这个过程是在 DCU 上执行了呢，答案是使用 rocm-smi(DCU2 使用 hy-smi)命令。与在 linux 系统上使用 Top 命令查看 CPU 负载类似，rocm-smi 命令可以查看 DCU 负载情况，当然它还有更多的用途。在执行上面程序的同时，使用 rocm-smi，可以实时监控 DCU 负载情况，应该会有如下输出，第一列是 DCU 的序号，如果你的电脑里有不止一块 DCU 就会有多行，%DCU 列代表了 DCU 的实时负载，可以看到在运行程序时候第一块 DCU 是满载状态。

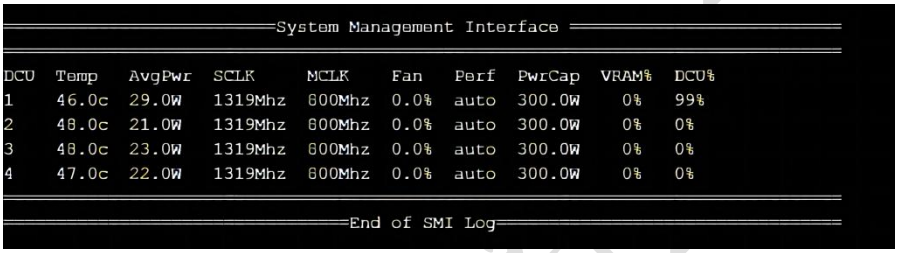


图 1.1 rocm-smi 命令输出

运行完程序后，我们回头来看 CPU 端和 DCU 端的代码差异，发现二者还有不小的区别，下面我们详细来讨论产生区别的具体原因，具体编程方法我们将在后面的章节详细介绍。

- DCU 代码初分析

上面程序主要包含以下四个部分。

1.空间管理

首先要明确一点，数组相加最开始的输入数据和输出数据都是在 CPU 端进行初始化，这些数据一开始都是在内存当中，DCU 要参与计算需要拿到这些数据，要实现这一目的，需要在 DCU 的显存上开辟一块空间，这块空间与 CPU 端内存大小相对应，然后经过 PCI-E 总线将这些数据传输到 DCU 当中。

与 CPU 内存管理类似，通过调用 hipMalloc 这样函数接口就可以完成 DCU 内存分配，hipMalloc 是最基础的内存分配接口，当然还有一些其他的内存分配方式，例如页锁定内存，这一部分将在后面章节进行介绍。

hip 程序其实是 C 程序的一种扩展，所以同样需要注意管理空间申请和释放防止内存泄露，所以当程序结束前要将申请过的地址空间释放，使用 hipFree 来达到释放显存的目的。

2. 数据拷贝

完成空间申请后，下面一步就需要将 CPU 端准备好的数据传输给 DCU 显存。

使用 hipMemcpy 完成这一过程，需要注意的是 hipMemcpy 具有方向性，最后一个参数 hipMemcpyHostToDevice，指示了方向，代表这一拷贝过程是从主机端到设备端，同样的 hipMemcpyDeviceToHost 指示的是从设备端到主机端的传输。

3. 函数定义

```
__global__ void add(float * d_A,float * d_B,float * d_C)
```

global 是一个函数标识符，代表了定义的函数是在主机端被调用，但是执行是在设备端，这个标识符会被 hipcc 编译器识别到，然后会按照 DCU 端指令去翻译下面的代码段。

被 global 标识的函数又被称为核函数或者是"kernel"，核函数代表了设备端的入口，一旦进入核函数，函数的执行和控制就交给了设备端。

除了 global 标识符，hip 还有其他的标识符提供给开发者使用，通过标识符，可以管理设备端的存储以及让开发者有更灵活的程序设计方式，这些内容在后面章节会详细介绍。

进入到核函数体中，首先使用 threadIdx.x，blockIdx.x ，blockDim.x 来定位当前的线程编号，根据线程编号来算出当前线程读取数据的偏移地址，接着再往下需要判断线程索引的数据是否超越数据边界，如果没有超过，运行核函数真正的计算逻辑，进行数据相加。

4. 函数执行

从 CPU 编程到 DCU 编程，你最需要转换的是你的思考方式，在设计核函数时，你需要站在一个线程的角度去思考你的问题如何被分解为一个个相似的小问题，而每一个小问题需要一个线程去做什么样的事情来解决它。如果你在 CPU 上进行过多线程编程，就会很容易的迁移到 DCU 上去理解这一个概念。

```
dim3 blocksize(256,1);
dim3 gridsize(N/256+1,1);
add<<<gridsize, blocksize >>>(d_A,d_B,d_C);
```

上面代码中核函数需要采用三个尖括号的方式调起，一旦调用起来，设备端就会自动创建 gridsize*blocksize 数量的线程。这些线程如何被组织起来，这里先不讨论，只需要知道一共有 gridsize*blocksize 个线程都会运行 add 函数的代码，括号里的参数就是执行该函数时需要的数据，在上面已经讨论过了。这些启动的线程将会被设备端的调度器分配到众多的计算核心中运行，从而实现并行化。这一点就是 DCU 能实现计算加速的关键点。

需要注意的是核函数被调用后，执行权限交给了设备端以后，此时主机端不会等待核函数执行完毕再执行下面的代码，而是立刻往下执行，也就是说主机端和设备端是异步的，当然 hip 也提供了命令可以保证主机端和设备端进行同步，其中 hipMemcpy 就是隐式同步的，一旦主机端遇到 hipMemcpy，就会等待主机端的所有操作包括数据传输执行完毕以后才会往下执行。这样会效率低下？当然。不过 hip 也具有异步拷贝功能，这就需要"流"这个功能配合使用，如果你感兴趣可以参看 HIP 并发执行这一个章节，通过这个技术可以同时启动多个核函数多个拷贝，至于剩下的就交给设备端调度执行了。

1.2 总结

我们将 HIP 编程一般流程总结如图 1.2。

开发者社区



图 1.2 HIP 编程一般流程

HIP 主要 API 释义，在后面的章节中会详细讲解下面的 API 使用场景以及区别：

表 1.1HIP 常用 API

API 名称	含义
hipGetDeviceCount	获取机器上的设备个数
hipGetDeviceProperties	获取选定设备的设备属性
hipMalloc	申请 DCU 内存
hipHostMalloc	在 CPU 端申请页锁定内存

hipStreamCreate	创建流
hipMemcpyAsync	CPU 和 DCU 内存异步拷贝，拷贝有两个方向，CPU 到 DCU，DCU 到 CPU
hipMemcpy	CPU 和 DCU 内存同步拷贝，会造成 CPU 端程序暂停等待拷贝的完成才会继续下面的指令，同上拷贝有两个方向
hipFree	释放 DCU 端的内存

开发者社区

2 HIP 核函数

2.1 线程执行模型

2.1.1 线程束和线程块

在 1 章节中介绍了核函数被调用起来后，会自动启动数千个线程来执行核函数的代码，这些线程会被划分为线程块，线程块的大小是通过三尖括号的第二个参数 blocksize 来配置的，三尖括号的第一个参数指的是一共启动多少个线程块。例如 gridsize 为 1，blocksize 为 256，带入下面的代码段，翻译过来就是一共启动 1*256 个线程来执行 add 的代码。

```
add<<<gridsize, blocksize >>>(d_A,d_B,d_C);
```

线程块在逻辑上可以被配置为一维、二维或三维，但是所谓的维度其实是人为的一种划分，其实在执行时候线程都被组织成一维形式，这个概念就好像是我们使用 C 语言申请了一个二维的数组 a[N][M]，虽然我们在使用数组时候有维度这个概念，但是在物理空间中其实这些数据就是连续存储的，我们当然可以使用一维数组去替代二维的，只要我们计算好偏移就可以了。

在一个块中每个线程都有唯一 ID，如果将 x 作为第一个维度即最内层维度，y 作为第二个维度，z 作为第三个维度即最外层维度，不管是二维或三维线程块都可以转化为一维线程布局。

如果需要使用多维线程组织模型，对应于代码中在设置 gridsize 和 blocksize 时就需要配置为 dim3 类型。在下面的例子中可以看到，对于 myGpuMatrix 这个核函数，我们分配了网格尺寸是 dimGrid(90,90),其描述的网格上 x 轴上线程块 90 个，y 轴线程块 90 个。每个线程块尺寸是 dimBlock(64,1)。

```
dim3 dimGrid(90,90);
dim3 dimBlock(64,1);
hipLaunchKernelGGL(myGpuMatrix,dimGrid,dimBlock,0,0,dev_A,dev_B,dev_C_t);
```

可以看到对于这两项尺寸的描述，使用的变量类型是 dim3。此类型是基于 uint3 指定维度的整数向量类型。也即是说我们定义名为 dim3 的结构体，分别用 x,y,z 来表示网格三个维度上线程块的个数，或者线程块三个维度上线程数的个数。另外，x、y、z 中未指定的任何部分默认初始化为 1，结构体定义如下。

```
typedef struct dim3 {
   uint32_t x;
   uint32_t y;
   uint32_t z;
   dim3(uint32_t _x=1, uint32_t _y=1, uint32_t _z=1) : x(_x), y(_y), z(_z) {};
};
```

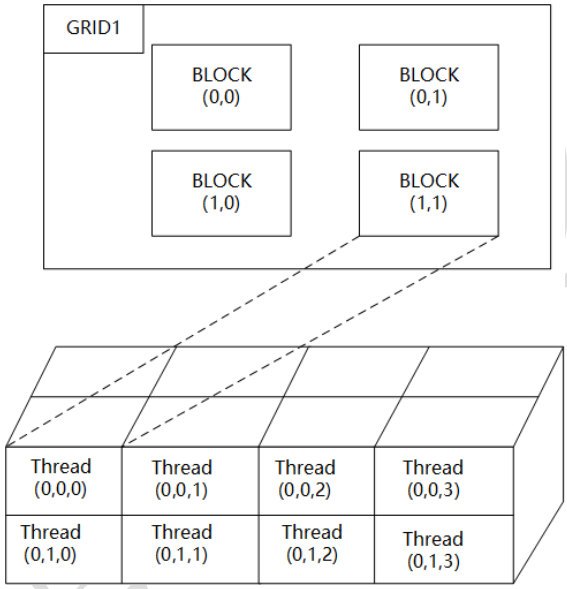


图 2.1 多维线程组织模型

由于 GRID 和 BLOCK 大小和维度可以自由组合，就会形成不同的现成索引方式:

1) 对于 1D grid, 1D block，线程 Id = blockIdx.x * blockDim.x + threadIdx.x；

2) 对于 3D grid, 1D block，线程 id = (gridDim.x * gridDim.y * blockIdx.z + gridDim.x * blockIdx.y + blockIdx.x) * blockDim.x + threadIdx.x；

3) 对于 1D grid, 2D block，线程 Id = blockIdx.x * (blockDim.x * blockDim.y) + blockDim.x * threadIdx.y + threadIdx.x；

4) 对于 3D grid, 3D block，线程 Id = (gridDim.x * gridDim.y * blockIdx.z + gridDim.x * blockIdx.y + blockIdx.x) * (blockDim.x * blockDim.y * blockDim.z) + blockDim.x * blockDim.y * threadIdx. z + blockDim.x * threadIdx.y + threadIdx.x;

5) 对于多维线程组织，其线程的增长方向是沿着 x，y，z 依次递增，这个规则就像在访问一个二维或者三维的数组一样。

一个线程块会被划分为一定数量的线程束，每个线程束有 64 个线程，如果线程块大小不是线程束大小的整数倍，那么最后那些除以 64 余下的线程依然会组成一个线程束。图 3-4 是一个在 x 轴为 80、y 轴为 2 的二维线程块，整个块中有 160 个线程，除以 64 的余数是 32，最后 32 个线程组成一个线程束，但是该线程束却要占用 64 个线程的调度资源和存储资源，所以这种现象的发生会降低性能。

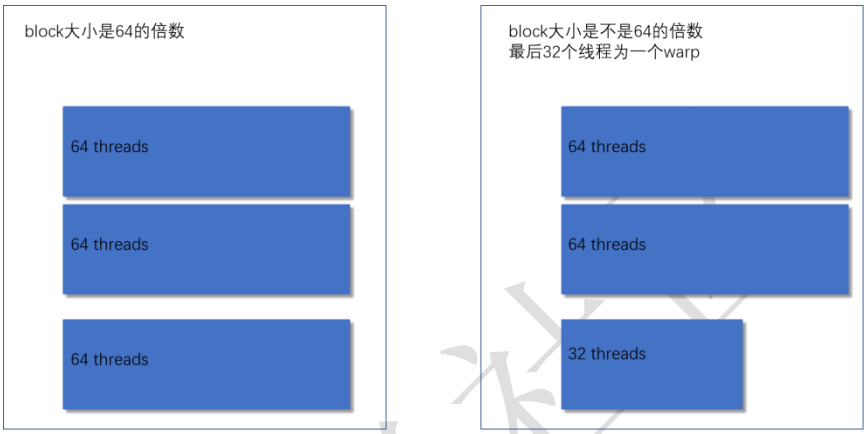


图 2.2 线程束分配示意图

线程束是线程块更加细化的组织，当一个核函数启动后，线程块分布在 CU 中；一旦线程块被调度到一个 CU 上，线程块中的线程会被进一步划分为线程束来执行；一个线程束中的所有线程按单指令多线程（SIMT）方式执行。图 2.3 展示了线程块的逻辑视图和硬件视图之间的关系：

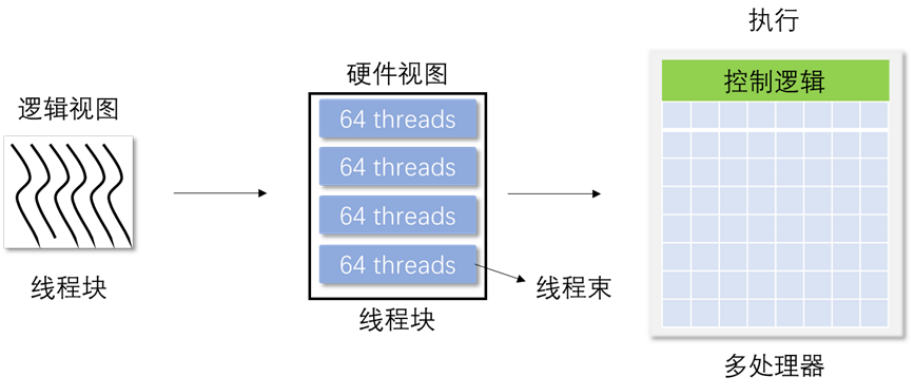


图 2.3 线程块逻辑视图与硬件视图关系图

综上，从逻辑角度来看，线程块是线程集合，可以被组织为一维、二维或三维布局；但从硬件角度看，线程块是一维线程束的集合，在线程块中线程被组织成一维布局，每 64 个线程组成一个线程束。

2.1.2 线程执行

一个线程块只能在一个 CU 上被调度，一旦线程块在一个 CU 上被调度，就会保存在该 CU 上直到执行完成。在 CU 中，共享内存和寄存器是非常重要的资源。共享内存被分配在 CU 上的常驻线程块中，寄存器在线程中被分配。线程块中的线程通过这些资源可以进行相互合作和通信。尽管线程块里的所有线程在逻辑上都可以并行运行，但在物理层面并非都能同时执行，因此线程可能会以不同的速度前进。

在并行线程中共享数据会引起竞争：多个线程以不定的顺序访问同一数据，会导致不可预测的程序行为。HIP 提供了一种方法来同步线程块里的线程，以保证所有线程在进一步动作之前都达到执行过程中的一个特定点，这个方法被称为线程块同步。

尽管线程块里的线程束可以任意顺序调度，但活跃线程束的数量还是会由 CU 的资源所限。当线程束因任何原因闲置时，CU 可以从同一 CU 上的常驻线程块中调度其他可用线程束。在并发的线程束间切换没有开销，因为硬件资源已经被分配到了 CU 上的所有线程和块中，所以最新被调度的线程束的状态已经被存储在 CU 上。

CU 是 DCU 架构的核心，寄存器和共享内存是 CU 中的稀缺资源。HIP 将这些资源分配到 CU 中的所有常驻线程里，因此这些有限的资源限制了在 CU 上活跃的线程束数量，活跃的线程束数量对应于 CU 上的并行量，关于 DCU 硬件的架构可以参看 DCU 软硬件架构内容。

2.1.3 线程束分化

在同一线程束中的线程执行不同的指令，被称为线程束分化，这种情况最常见于程序中的 if 和 else 使用。因为 DCU 没有类似 CPU 那样复杂的分支预测机制，在一个指令周期中只有满足条件的线程才会执行，不满足条件的线程什么都不执行等待其他线程执行完毕，然后同一个线程束内线程继续同步往下执行。比如下面的语句：

```
A;
if(threadIdx.x<4)
```

```
   { B;}
else
   { C;}
A;
```

以 32 个线程举例，具体执行情况如下图 2.4 所示：

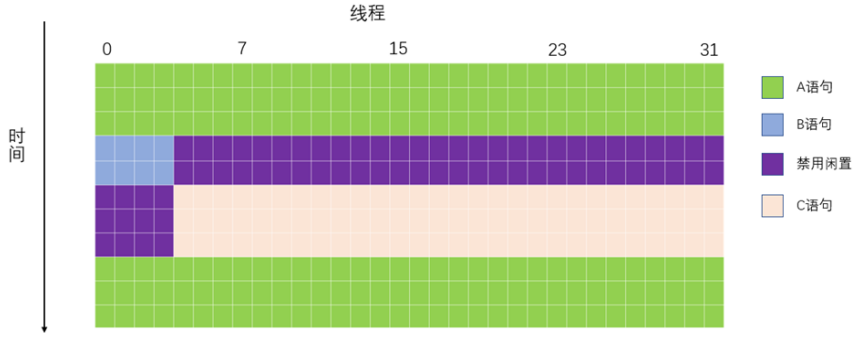


图 2.4 线程束分化

当线程束分化时，32 个线程会连续执行含有 B 语句和 C 语句的分支，对于不执行这一路径的线程，会被禁用闲置，这种情况将会导致程序性能下降。

2.1.4 关于线程调度和执行模型大小的优化

提高占用率也是重要的一方面。占用率是每个 CU 中活跃线程束占最大线程束数量的比值：占用率=活跃的线程束数量/最大线程束数量。在 HIP 核心中指令是顺序执行的，当一个线程束阻塞时，CU 切换其他符合条件的线程束，理想情况下需要有足够的线程束占用设备的核心。为了提高占用率，还需要调整线程块配置或重新调整资源的使用情况，以允许更多的线程束同时处于活跃状态和提高计算资源的利用率。极端地操纵线程块会限制资源的利用，如果每个块中线程太少，会在所有资源被充分利用之前导致硬件达到每个 CU 的线程束数量的限制。如果每个块中有太多的线程，会导致在每个 CU 中每个线程可用的硬件资源较少。所以在编程时应该注意以下原则：

- 保持每个块中线程数量是 64 的倍数
- 避免块太小，每个块一般设置为 128 或 256 个线程
- 避免单个线程消耗资源过大，导致无法同时开启足够的线程影响程序效率

- 块的数量要多于 CU 的数量，从而在设备中可以有足够的并行度

2.1.5 同步

栅栏同步是一个原语，在许多并行编程语言中都十分常见。在 HIP 中，同步可以在两个级别执行：

- 设备级：阻塞主机端的操作，等待设备完成所有工作

对于主机来说，基本上 HIP 调用或核函数执行都是异步的，hipDeviceSynchronize 函数可以用来阻塞主机应用程序，直到所有 HIP 操作完成。这个函数可能会从先前异步 HIP 操作返回错误。

- 线程级：在设备执行过程中对一个线程块的线程进行同步

因为在一个线程块中线程束以一个未定义的顺序执行，HIP 提供了一个局部栅栏来同步它们的执行，可以使用__syncthreads()函数在内核中标记同步点。当__syncthreads()被调用时，在同一个线程块中每个线程都必须等待直至该线程块中所有其它线程都达到这个同步点。该函数可以协调同一个块中线程之间的通信，但它强制线程束空闲，因此可能对性能产生负面影响。

2.2 可编程存储结构

对于开发人员来说，一般有两种类型的存储器：

- 可编程的：你需要显式地控制哪些数据存放在可编程内存中，例如全局内存和共享内存。
- 不可编程的：程序将自动生成存放位置，例如 L1 和 L2。

在 CPU 内存层次结构中，一级缓存和二级缓存都是不可编程的存储器。在第二章中已经介绍了 HIP 内存模型，提到了多种可编程内存的类型如图 2.5：

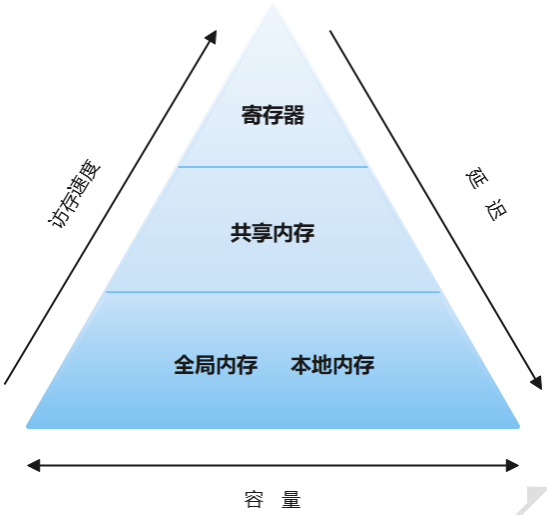


图 2.5 可编程存储结构

2.2.1 存储结构简介

- 寄存器

寄存器是 DCU 上访问速度最快的内存空间。如果核函数中声明的自变量没有其他修饰符，那么它通常存储在寄存器中。在核函数声明的数组中，如果用于引用该数组的索引是常量且能在编译时确定，那么该数组也存储在寄存器中。

寄存器变量对于每个线程来说都是私有的，通常使用寄存器来保存需要频繁访问的线程私有变量。寄存器变量与线程的生命周期相同。一旦线程执行完毕，就不能对寄存器变量进行访问了。

如果使用了超过硬件限制数量的寄存器，则会用本地内存替代寄存器，这种寄存器溢出会给性能带来不利影响。

- 全局内存

全局内存是 DCU 中容量最大、延迟最高并且最常使用的内存。全局内存常驻于 DCU 内存中，可通过内存事务进行访问，这些内存事务最好是 4 字节对齐，这样可以利用全局内存合并访存的特性来减少对全局内存的访问次数，关于合并访存具体介绍请参看 DCU 应用移植与优化章节。优化全局内存事务对于获得最优性能来说是至关重要的。

- 共享内存

共享内存是 HIP 编程模型中提供的可编程缓存。在 DCU 上，每个 CU 都有一块共享内存，大小为 64KB，其带宽和延迟远远优于全局内存。共享内存在核

函数的范围内声明，是线程之间相互通信的基本方式。其生命周期伴随着整个线程块，一个块内的线程通过使用共享内存中的数据可以相互合作。

- 本地内存

本地内存，位于全局内存上的一个空间。在逻辑上，它属于线程私有的。

当一个线程执行核函数时，核函数的变量、数组、结构体首先会考虑放在寄存器中。当定义的变量、数组、结构体过大时，编译器判断这部分数据过度占用寄存器资源，就会将他们直接存放在本地内存中。

也就是说，当核函数中的变量较少，寄存器区的大小足够放下这些变量，那么它们就放在 DCU 中的寄存器区。

当核函数中有大数组、大结构体以至于寄存器区放不下他们，编译器在编译阶段就会将他们放到位于片外的本地内存中，其物理位置和全局内存一样，我们称这种情况为寄存器溢出。溢出到本地内存中的变量存储在全局内存中，且仅线程可见。

本地内存的访存速度和访问全局内存大致相等，溢出到本地内存中的变量本质上与全局内存在同一块存储区域，因此本地内存访问的特点是高延迟和低带宽。

2.2.2 共享内存

在 HIP 编程模型中，为了获得高内存带宽，使用共享内存是一种常用的手段。共享内存被分为 32 个同样大小的内存模型，被称为存储体（Bank），可以被同时访问。共享内存是一个一维地址空间，在不同模式下会映射到不同的存储体中。从结构上，可以把存储体形象的比喻为一串可以穿无数个山楂的糖葫芦串，每一个穿山楂的签子就对应了一个内存通道称为 Bank，每个山楂就是存储空间以 4 字节为单位。物理上，串上的相邻两个存储单元在地址上相隔了 32。如果通过线程束发布共享内存加载或存储操作，且在每个存储体上只访问不多于一个的内存地址，那么该操作可由一个内存事务来完成。否则，该操作由多个内存事务来完成，这种情况被称为 Bank 冲突，会降低内存带宽的利用率。

在编写核函数中使用共享内存可以减少对全局内存的访问。可以将经常使用到的数据，首先存放在共享内存中，利用共享内存高带宽、低延迟的特性，当多次访问该数据时，访问共享内存所用的时间将会远远小于访问全局内存的

时间。

共享内存块中，无论声明一维或二维的数组均可通过计算得到一维的内存偏移，在实际的存储中数据都是线性排布的。两种共享内存静态声明如下：

```
__shared__ int tile[N];
__shared__ int tile[Row][Col];
```

图 2.6 展示了一个有 32 行 32 列，按行主序进行存储的共享内存块，显示四字节数据元素和存储体映射的二维共享内存逻辑视图。

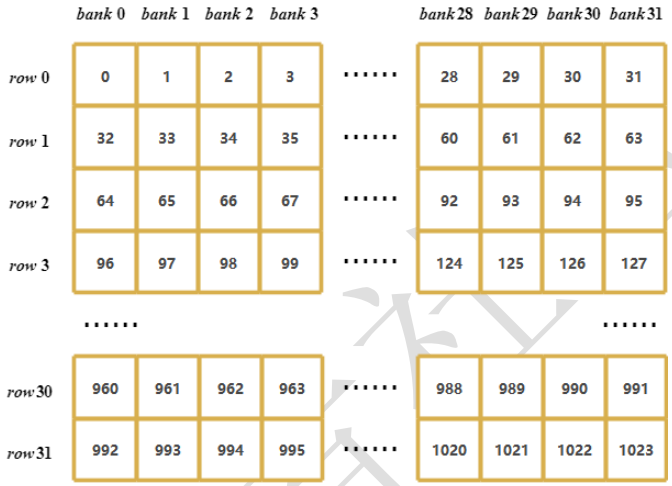


图 2.6 32 行 32 列数据排列

四字节的一维数据的实际排布如图 2.7 所示。

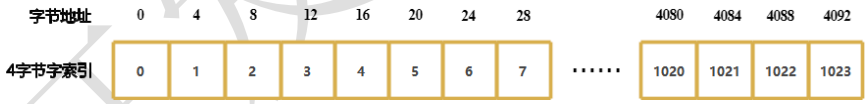


图 2.7 四字节数据排布示意图

图 2.6 中的共享内存块的行和列数量相同， 我们可以有两种访问模式：

```
tile[hipThreadIdx_y][hipThreadIdx_x]
tile[hipThreadIdx_x][hipThreadIdx_y]
```

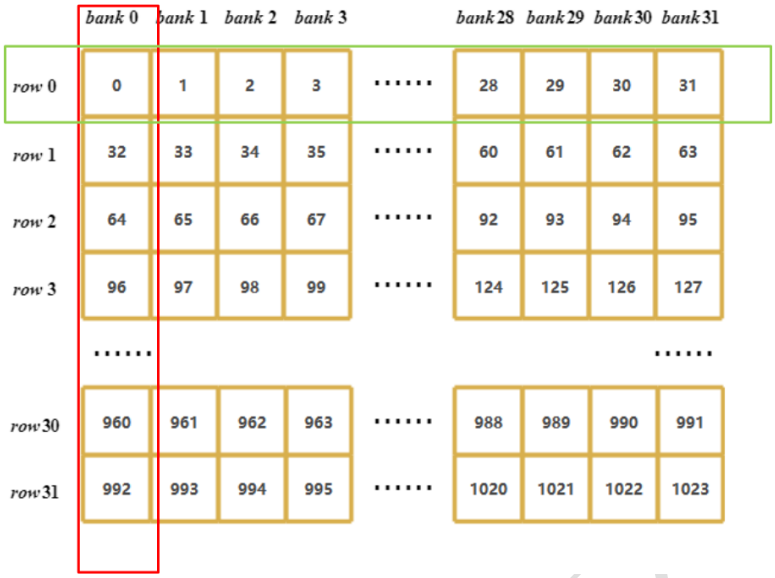


图 2.8 两种访问模式区别

由于在线程的组织中，线程编号的增长顺序为 x，y，z 依次增加，所以对于第一种访问模式，相邻线程访问的数据正好在相邻的 bank 中，bank 冲突尽可能的避免，这种情况对应于图 2.8 横向框；但是对于第二种访问模式，相邻线程访问的数据正好在同一个 Bank 中，这时候 bank 冲突概率就比较大，这种情况对应图 2.8 竖向框。

以上的两种方式在性能方面有一定的差异，相同线程束中的线程访问不同 bank，是最优的访存方式，邻近线程在内层维度访问相邻的单元会因为产生更少的 bank 冲突而得到更好的性能，关于 bank 冲突优化方法可以参考章节 GEMM 优化问题。

2.3 HIP 修饰符

2.3.1 存储修饰符

寄存器及本地内存修饰符：

在核函数中，没有修饰符修饰的变量将存储在寄存器或本地内存中。存储在寄存器和本地内存的变量仅作用于声明该变量的线程中，只有声明该变量的线程有权访问和操作该变量。存储在寄存器和本地内存的变量的生命周期伴随着声明该变量的线程一起创建一起消亡。

- shared 共享内存
  - 在核函数中使用如下修饰符修饰的变量存放在共享内存中：

__shared__

- 存储在共享内存的变量作用于声明该变量的线程所在的线程块中的所有线程，线程块中的所有线程都可对该变量进行访问和修改。存储在共享内存的变量伴随相应的线程块一起创建一起消亡。

- device 全局内存
  - 一个全局内存变量可以被静态声明或动态声明。你可以使用如下修饰符在设备代码中静态地声明一个变量：__device__
  - 存储在全局内存的变量作用当前程序中的所有线程，所有线程均拥有对该变量的访问和操作权限。存储在全局内存的变量的生命周期伴随整个应用程序一起创建一起消亡, 下表总结了 HIP 变量声明和它们相应的存储位置、作用域、生命周期和修饰符。

表 2.1 HIP 变量和类型修饰符（+即可以表示标量也可以表示数组。）

修饰符	变量名称	存储器	作用域	生命周期
	float var	寄存器	线程	线程
	float var[100]	本地	线程	线程
__shared__	float var+	共享	块	块
__device__	float var+	全局	全局	应用程序

2.3.2 函数修饰符

核函数是在设备端执行的代码。在核函数中，需要为一个线程规定要进行的计算以及要进行的数据访问。当核函数被调用时，许多不同的 HIP 线程并行执行同一个计算任务。以下是用__global__声明定义核函数：

```
__global__ void kernel_name(argument list);
```

核函数返回类型必须为 void，表 2.2 中总结了 HIP 程序中的函数类型限定符。函数类型限定符指定一个函数在主机上执行还是在设备上执行以及可被主机调用还是被设备调用。

表 2.2 HIP 函数修饰符

限定符	执行	调用	备注
__global__	在设备端执行	也可从主机端调	必须有一个

		用	void 返回类型
__device__	在设备端执行	仅能从设备端调用	
__host__	在主机端执行	仅能从主机端调用	

- __global__：在 device 上执行，从 host 中调用，返回类型必须是 void，不支持可变参数，不能成为类成员函数。注意用__global__定义的 kernel 是异步的，这意味着 host 不会等待 kernel 执行完就执行下一步。
- __device__：在 device 上执行，仅可以从 device 中调用，不可以和__global__同时用。
- __host__：在 host 上执行，仅可以从 host 上调用，一般省略不写，不可以和__global__同时用，但可和__device__同时用，此时函数会在 device 和 host 都编译。

注意：__device__和__host__限定符可以一齐使用，这样函数可以同时在主机和设备端进行编译。

2.4 内置函数和数据类型

在使用 HIP 编程过程中会频繁用到执行数学运算的函数，例如求绝对值，求解正弦函数，或者是向上向下取整等。我们按照计算速度和计算精度的不同，以及在进行编程时是否能在核函数中调用，把这些数学函数分为两种类型：

- 标准数学函数
- 快速数学函数

2.4.1 标准数学函数

标准数学函数可以在主机端和设备端代码中使用，在写程序调用时大致上类似于标准 C++ 编译器中标准数学库包含的“math.h”头文件中的数学运算，如 sqrt、exp 和 sin。单指令运算如乘法和加法，也包含在标准数学函数中。区别是：如果是对 int 操作，直接使用函数名，如果操作 long 类型，在函数名前加 l，如果操作双精度浮点数，在函数名前后加 f，如果操作单精度，在函数名后都加

上 f。

```
int sin(int, int);//对 int 类型操作
long lsin(long,long);//对 long 类型操作
double fsinf(double,double);//对 double 类型操作
float sinf(float,float);//对 float 操作
```

下面的核函数 sin()用来计算参数 x 的正弦值，然后返回-1 至 1 之间的计算结果存储到 c 中。可以看到在核函数中调用方式与 C++是类似的。

```
__global__ void sinKernel(float *a, float *b, float *c) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    c[i] = sinf(a[i] - b[i]);//在设备端中调用
}
int main(int argc, char *argv[]) {
    ...
    float d_a, d_b, d_c;
    float c_host, x_host;
    ...
    c_host = sinf(x_host);//在主机端中调用
    ...
    dim3 dimGrid(1);
    dim3 dimBlock(64);
    hipLaunchKernelGGL(sinkernel, dimGrid, dimBlock, 0, 0, d_a, d_b, d_c);
    ...
}
```

下面是一些常用标准数学函数的写法以及功能介绍。

表 2.3 标准数学函数

函数	功能
sinf(x)	计算参数 x 的正弦值
sincosf(x,sptr,cptr)	计算第一个输入参数 x 的正弦和余弦
logf(x)	计算输入参数的自然对数 ln(x)

log2f(x)	计算输入参数的以 2 为底的对数
expf(x)	指数函数 e^x
exp10f(x)	计算输入参数的以 10 为底的指数
powf(x,y)	计算 x^y，x 的 y 次幂
fminf(x,y)	确定参数 x,y 中的较小值
fmaxf(x,y)	确定参数 x,y 中的较大值
fmodf(x,y)	计算 x / y 的浮点余数
fabsf(x)	计算参数 x 的绝对值
ceil(x)	向上取整，即不小于 x 的最大整数

2.4.2 快速数学函数

相对于标准数学函数，在 HIP 编程中支持一些精度有所降低但速度更快的函数版本。它们的名称相同，但带有__前缀（例如 sinf 与__sinf(x)），这些函数被称为快速数学函数。快速数学函数只能对设备代码进行访问，在编程中，如果一个函数是快速数学函数，那么在编译时对它的行为会有更少更优化的底层指令生成。许多快速数学函数与标准函数是有关联的，存在与快速数学函数功能相同的标准函数。

下面的核函数在之前的例子基础上用功能上等价的__sinf()来替换 sin()来计算参数 x 的正弦值，然后返回-1 至 1 之间的计算结果存储到 c 中。可以看到在核函数中调用方式与之前的核函数是类似的。

```
__global__ void fast_sinKernel(double *a,double *b, double *c)
{
   int i = blockIdx.x*blockDim.x+threadIdx.x;
   c[i] = __sinf(a[i]-b[i]);//只能在设备端中调用
}
```

除了正弦函数之外，也有很多其他用途的数学函数也是以类似的方法进行调用：

```
__global__ void fast_math(float a,float b,float c,float c_1,float c_2){
   c = __powf(a,b);//a 的 b 次幂
```

developer.sourcefind.cn

```
  c_1 = __fmul_rn(a,b);//舍入到最接近的偶数。
  c_2 = __logf(x)(c);
}
```

以下是一些使用快速数学函数对标准数学函数进行替换的对照表：

表 2.4 标准数学和快速数学函数比对

标准数学函数	快速数学函数
x/y	__fdividef(x,y)
sinf(x)	__sinf(x)
sincosf(x,sptr,cptr)	__sincosf(x,sptr,cptr)
logf(x)	__logf(x)
log2f(x)	__log2f(x)
expf(x)	__expf(x)
powf(x,y)	__powf(x,y)

- 标准数学函数与快速数学函数的差别

标准函数和快速数学函数在数值精确度和性能上的表现是不同的。标准函数支持大部分的数学运算，而许多等效的快速数学函数能够使用较少的指令、改进的性能和更低的数值精确度来实现相同的功能。在精度上对标准数学函数的要求是小于 1ulp，快速数学函数则是小于 3ulp。

- ULP 是什么？

调用数学函数时，需要对函数功能的准确性信息进行定量描述。ULP（Unit in the Last Place）是计算机科学中针对浮点数提出的一个误差单位，它的定义是：对于给定的浮点数格式，特定实数值的 ulp 是与此实数值左右最近的两个浮点数的距离。

以 IEEE754 标准的 32 位浮点数为例。如果想表示实数值 0.1 是无法准确表示的，只能用最接近 0.1 的可以精确表示的浮点数来表示。这样的数有两个，设为 A、B，它们的十六进制和十进制表示分别为：

- A:

十六进制：0x3dcccccc

十进制：0.099999994039536

- B:

十六进制：0x3dcccccd

十进制：0.10000000149012

- 那么 0.1 的 ulp 就等于|A-B| = 0.00000000745076；
- 如果计算机 1 使用向下近似的方法用 A 来表示 0.1。那么误差 = 0.1 - 0.099999994039536 = 0.000000005960464 = （0.000000005960464 / 0.00000000745076) * ulp ≈ 0.8 ulps；
- 如果计算机 2 使用向上近似的方法用 B 来表示 0.1。那么误差 = 0.10000000149012 - 0.1 = 0.00000000149012 = (0.00000000149012 / 0.00000000745076) * ulp ≈ 0.2 ulps；

由上述计算可知，在 IEEE754 标准的 32 位浮点数的前提下，计算机 1 关于 0.1 的计算误差为 0.8 ulps，计算机 2 关于 0.1 的计算误差为 0.2 ulps。

➢ 除法的样例

快速数学函数分解成了比与它们等价的标准函数更少的指令。这会导致快速数学函数比等价的标准函数更快，但数值精确度却更低。可以在同一应用中交替使用标准函数和快速数学函数，但是它们在性能和数值精确度上会有所不同。

标准函数中执行单精度浮点除法运算的快速数学函数：__fdividef 与"/"具有对应关系。内部数学函数__fdividef与运算符“/”相比，在执行浮点数除法时速度更快但数值精确度相对较低。假设有下面的核函数 divide：

```
__global__ void divide(float a,float b,float c){
   float c = a / b;
}
```

可以用功能上等价的__fdividef来替换“/”。

```
__global__ void fast_divide(float a,float b,float c){
   float c = __fdividef(a , b);
}
```

常规的浮点除法和__fdividef(x,y)拥有同样的精度，但对于 2^126 < y < 2^128，__fdividef(x,y) 的结果为零，而常规的除法可以得到正确的结果。同样的，对于 2^126< y < 2^128 ，如果 x 是无穷大，__fdividef(x,y) 的结果是 NaN（结果是无

穷大乘于零），而常规的除法返回无穷大。

- 正弦函数的样例

为了更清楚的观察在 HIP 编程中标准数学函数与快速数学函数之间计算精度与计算速度的差别，我们通过调用单精度浮点数的正弦函数的例子来对调用结果进行分析，在这里分三种方式来计算正弦函数：

- 在主机端调用标准数学函数 sinf()。
- 在设备端调用标准数学函数 sinf()。
- 在设备端调用快速数学函数__sinf()。

在这里我们把主机端得到的正弦函数的计算时间和计算结果作为基准值，对比后两种方式计算得出的时间和结果，从而体现出标准函数与快速数学函数之间的差别。

三种调用的代码为：

```
#include <stdio.h>
#include <stdlib.h>
#include <hip/hip_runtime.h>
#include <hip/hip_runtime_api.h>
#include <sys/time.h>
#include <time.h>
#define Maxsize 5760
//在主机端调用标准数学函数 sinf()
void cpu_sinf(float *a, float *b, float *c) {
   for (int i = 0; i < Maxsize; i++)
      c[i] = sinf(a[i] + b[i]);
}
//在设备端调用标准数学函数 sinf()。
__global__ void dcu_sin(float *a_device, float *b_device, float *c_device) {
   int i = hipThreadIdx_x;
   c_device[i] = sinf(a_device[i] + b_device[i]);
}
```

```
//在设备端调用快速数学函数__sinf()
__global__ void dcu_sin_fast(float *a_device, float *b_device, float *c_device) {
    int i = hipThreadIdx_x;
    c_device[i] = __sinf(a_device[i] + b_device[i]);
}
int main() {
    ...
for (int i = 0; i < Maxsize; i++)
//初始化计算数组 a,b 为随机数
        a[i] = rand() % 16 / 16.0;
    for (int i = 0; i < Maxsize; i++)
//初始化 c 为 0
        c[i] = c_device[i] = c_device_fast[i] = 0;
    ...
    dim3 dimGrid(1, 1);
    dim3 dimBlock(256, 1);
    hipLaunchKernelGGL(dcu_sin, dimGrid, dimBlock, 0, 0, a_device, b_device,
c_device);
    ...
    hipLaunchKernelGGL(dcu_sin_fast, dimGrid, dimBlock, 0, 0, a_device, b_device,
c_device_fast);
    ...
    cpu_sinf(a, b, c);
    ...
}
```

以下是程序的部分输出，计算结果格式为保留小数点后 15 位。具体运行结果和时间分别为：

```
//在设备端调用标准数学函数 sinf()的结果和时间
result from sinf on dcu:     0.966826558113098              0.585097253322601
```

```
0.186403289428455
cost time from sinf on dcu: 0.04700 ms
//在设备端调用快速数学函数__sinf()的结果和时间
result from __sinf on dcu:0.966826498508453 0.585097253322601
0.186403319239616
cost time from __sinf on dcu: 0.03100 ms
//在主机端调用标准数学函数 sinf()的结果和时间
result from sinf on cpu: 0.966826558113098 0.585097253322601
0.186403289437294
cost time from sinf on cpu: 0.10000 ms
```

以主机端所得结果为基准值可以看到，在设备端调用标准数学函数在精度上误差较小，并且速度有所提升；在设备端调用快速数学函数的计算速度是三种方式中最快的，但与在主机端调用标准数学函数相比，其精度影响较大。可印证上述内容，即快速数学函数计算速度与标准数学函数相比，采用了以精度换速度的优化策略。

2.4.3 集合函数

在 HIP 编程时，核函数中支持调用集合函数（选举函数）的操作，这类函数允许线程之间共享 bit 位。在硬件层面保证线程束内的所有 lane 都将以锁步方式执行，因此不需要额外的同步指令，并且集合函数不使用共享内存。

首先介绍一下束内线程（lane）的概念。简单来说，lane 就是一个线程束内各个线程的索引。因此 lane 的取值范围是[0-64]。由于线程块中线程总数是在启动核函数时定义的，所以一个线程块中可能有多个线程束，例如一个线程块启动 128 个线程的话，就分为 2 个线程束，这两个线程束之间的线程可以有相同的束内线程索引 lane（就像同一网格中的多个线程可以有相同 threadIdx.x 值一样）。在一维线程块中，对于给定线程的束内线程索引和线程束索引可以按以下公式进行计算：

```
laneID = threadIdx.x % 64
workgroupID = threadIdx.x / 64
```

例如，线程块中的线程 1 和线程 65 都有束内线程 ID=1，但它们有不同的线

程束 ID。对于二维线程块，可以将二维线程坐标转换为一维线程索引，并应用前面的公式来确定束内线程和线程束的索引。

对于集合函数，每个线程束的 lane 对应着 1bit 位的值(predicate)，可以有效的广播到线程束中的所有 lane。从每个 lane 得到结果后再综合返回一个最终的结果。

在 HIP 编程中支持的集合函数的具体定义如下:

```
int __all(int predicate)
```

all(int predicate)：函数将 predicate 与 0 进行比较，如果当前线程所在的线程束所有线程对应 predicate 不为 0，则返回 1。

```
int __any(int predicate)
```

wavefront 块内的所有线程计算 predicate，当且仅当任意线程的 predicate 非零时返回非零值。实际写代码时可以使用 hasWarpVote 设备属性或 HIP_ARCH_HAS_WARP_VOTE 编译器定义来测试当前平台是否支持 any/all 指令。

```
uint64_t __ballot(int predicate)
```

__ballot(int predicate)：当前线程所在的线程束中第 N 个线程对应的 predicate 值不为 0，则将返回值第 N 位进行置 1。也就是返回一个 64 位无符号整数，包含来自每个 lane 的 1 位 predicate 值，结果的第 n 位包含 warp 中第 n 个 lane 所对应的 1 位。

集合函数具体的实现例子：

```
__global__ void test_any(int* a, int* b, int* c) {
 const auto tid = threadIdx.x + blockIdx.x * blockDim.x;
 a[tid] = __any(threadIdx.x == 16);
}
//若 warp 中至少有一个线程是 16，则所有线程中 warp 的返回值将为 1。否则返回 0。
__global__ void test_all(int* a, int* b, int* c) {
 const auto tid = threadIdx.x + blockIdx.x * blockDim.x;
 a[tid] = __all(threadIdx.x == 16);
```

```
}
//若当前线程所在的线程束所有线程对应都是 threadIdx.x == 16，则返回 1。
__global__ void test_ballot(int* a, int* b, int* c) {
  const auto tid = threadIdx.x + blockIdx.x * blockDim.x;
  a[tid] = __ballot(threadIdx.x == 16);
}
```

2.4.4 线程束洗牌函数

在 HIP 编程中支持 shuffle instruction（线程束洗牌函数）指令，如果两个线程在同一个线程束中，允许线程直接读其他线程的寄存器值。我们知道，核函数内部的变量都在寄存器中，一个线程束可以看做是 64 个线程并行执行，也就是说这 64 个线程中所使用的寄存器，在硬件上具有空间局部性，这样就为相互访问提供了物理基础。

这种机制比通过共享内存进行线程间的通讯效果更好，延迟也更低，同时也不消耗额外的内存资源来执行数据交换，使得通信效率提高很多。洗牌函数支持的数据类型包括整型和浮点型（int、unsigned int、long、long long、float、double），线程数量大小的取值为 2n。

在 HIP 编程中目前支持的洗牌指令是以下四种，其中 width 参数在不输入参数的情况下默认是 64：

```
__shfl(int var, int lane, int width=warpSize)
__shfl_up(int var, unsigned int delta, int width=warpSize)
__shfl_down(int var, unsigned int delta, int width=warpSize)
__shfl_xor(int var, int laneMask, int width=warpSize)
```

下面就对这四种函数进行进一步分析。

- 线程束范围内的广播

线程束内对整型变量进行操作，函数的基本格式和参数如下所示：

```
int __shfl(int var, int lane, int width = warpSize);
```

首先对函数的参数及其含义进行分析。参数中 lane 的含义变化取决于输入参数宽度 width，默认的 width 设置为线程束大小（即 64），如果为了保持和 CUDA 的兼容性也可以选择把 width 设置为 32。洗牌函数在整个线程束范围内

执行，并且 lane 指定源线程的束内线程索引。对于非 64 的 width 值，线程的束内线程 ID 和其在洗牌操作中的 ID 不一定相同。在这种情况下，一维线程块中的线程洗牌 ID 可以按以下公式进行计算：

```
shuffleID = threadIdx.x % width;
```

函数__shfl 的返回值是 var，它的值是由同一线程束中指定的线程广播的，通过 width 值来确定 shuffleID。对于 shuffleID 的值等于输入参数 lane 的线程，作用是向束内其他线程广播其输入参数 var 对应的值；对于 shuffleID 的值不等于输入参数 lane 的线程，接受来自 lane 对应线程广播来的参数 var。

例如，如果 shfl 被线程束中的每个线程通过以下参数调用：

```
int y = shfl(x,2,16);
```

那么线程 0～15 将从线程 2 接收 x 的值，线程 16～31 将从线程 18 接收 x 的值（在线程束的前 16 个线程中其偏移量为 2）。当传递给 shfl 的束内线程索引与线程束中所有线程的值相同时，指令从特定的束内线程到线程束中所有线程都执行线程束广播操作。

为了方便说明，我们继续以线程块中有 16 个线程为例来进行说明：

```
#define width 16
```

调用核函数的方法如下。通过最后一个参数给 shfl_broadcast 中的 lane 赋值，将进行广播的束内线程设置为每个线程束内的第三个线程（线程 2），全局内存的输入数据和输出数据被传递到内核。

```
dim3 dimGrid(1);
dim3 dimBlock(width);
hipLaunchKernelGGL(shfl_brodcast,dimGrid,dimBlock,0,0,d_out,d_in,2);
```

下面的核函数实现了线程束层面的广播操作，每个线程都有一个寄存器变量 value。源束内线程由变量 lane 指定，它等同于束内所有线程范围内。每个线程都直接从源线程复制数据。

```
__global__ void shfl_brodcast(int *d_out ,int *d_in,int const lane){
   int value = d_in[threadIdx.x];
   value = __shfl(value,lane,width);
   d_out[threadIdx.x] = value;
```

```
}
```

线程束中每个线程的初始值是根据自己的线程索引来设置的。调用__shfl后，第三个线程（在束内线程 2 中且值为 2）的数值将被广播到其他所有线程中。上述指令执行的流程及结果如图 2.9：

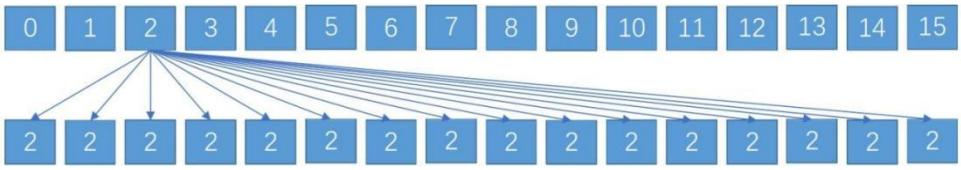


图 2.9 广播

- 线程束内上移

洗牌函数的另一种形式是__shfl_up，作用是从相对于当前线程具有低索引值的线程中复制数据到当前调用线程。

__shfl_up 函数的作用是，调用线程得到当前束内线程编号，减去 delta 的编号计算源束内线程索引，width 和__shfl 中一样，默认是 64，返回由源线程所持有的 var 值。因此，这一指令通过束内线程 delta 将 var 右移到线程束中。由于是线程束上移两个线程，所以在最左边的两个线程值保持不变，也就是线程束中最低的线程 delta 将保持不变。线程束中每个线程的源束内线程都是不同的，并由它自身的线程索引减去 delta 来确定。 通过指定 delta 为 2 来调用核函数：

```
dim3 dimGrid(1);
dim3 dimBlock(width);
hipLaunchKernelGGL(shfl_up, dimGrid, dimBlock, 0, 0, d_out, d_in ,2);
下面的核函数实现了洗牌函数线程束上移的操作。
__global__ void shfl_up(int *d_out, int *d_in, unsigned int const delta) {
   int value = d_in[threadIdx.x];
   value = __shfl_up(value, delta, width);
   d_out[threadIdx.x] = value;
}
```

最左边的两个束内线程值保持不变，每个线程的值向右移动两个束内线程，结果如图 2.10 所示。

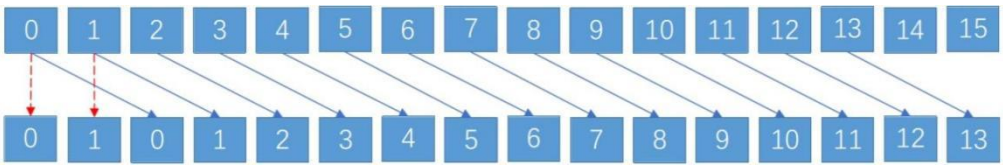


图 2.10 束内线程上移两位

- 线程束内下移

洗牌指令的第三种形式是__shlf_down，其作用与__shlf_up 正好相反，是从相对于当前线程具有高索引值的线程中复制到当前调用线程。下面的核函数通过指定 delta 为 2 来实现线程束下移操作。线程束中每个线程的源束内线程都是唯一的，由它自身的线程索引加上 delta 来确定。

```
__global__ void shfl_down(int *d_out, int *d_in, unsigned int const delta) {
   int value = d_in[threadIdx.x];
   value = __shfl_down(value, delta, width);
   d_out[threadIdx.x] = value;
}
```

最高位的两个束内线程值保持不变，每个线程的值向左移动两个束内线程，执行流程及结果如图 2.11 所示。

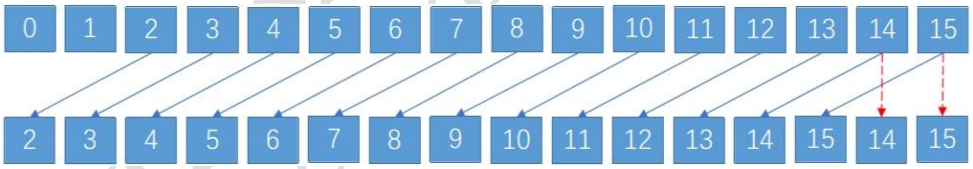


图 2.11 束内线程下移两位

- 线程束内的蝴蝶交换

洗牌函数的最后一种形式是__shfl_xor。此函数调用束内线程索引自身的按位异或来传输束内线程中的数据。函数中的 xor 是异或操作，这个操作和在 c 语言中一致，逻辑中我们假设只有 0，1 两种信号，用“xor”表示异或运算符即有如下规则：

0 xor 0=0;

1 xor 0=1;

0 xor 1=1;

1 xor 1=0;

异或操作是二元操作，只要两个参数不同就会得到真，否则为假。也就是

说我们的__shfl_xor 操作就是包含异或操作在内的洗牌函数，接下来对函数的具体参数进行分析。

```
__shfl_xor(int var, int laneMask, int width=warpSize)
```

参数 laneMask，就是进行异或操作的掩码，如果输入的 laneMask 的值是 1，可以得到它对应的二进制是，束内线程的索引范围是，所以我们用 laneMask 与当前线程索引进行异或操作就可以得到目标线程的编号，当前 laneMask 是 1，因此把 1 与 0~63 分别进行异或可以得到如下结果：

000001 xor 000000=000001;

000001 xor 000001=000000;

000001 xor 000010=000011;

000001 xor 000011=000010;

...

000001 xor 111110=111111;

000001 xor 111111=111110;

异或操作后，对于线程 0，得到线程 1；对于线程 1，得到线程 0；以此类推，可以得出当 laneMask 为 1 时，束内线程所对应的操作是从低到高，以两个线程为跨度，每两个线程之间与相邻的线程进行数值交换，也就是蝴蝶寻址模式（a butterfly addressing pattern），通过指定 laneMask 为 1 来调用核函数：

```
dim3 dimGrid(1);
dim3 dimBlock(width);
hipLaunchKernelGGL(shfl_xor,dimGrid,dimBlock,0,0,d_out,d_in,1)
__global__ void shfl_xor(int *d_out, int *d_in, int const mask){
   int value = d_in[threadIdx.x];
   value = __shfl_xor(value, mask, width);
   d_out[threadIdx.x] = value;
}
```

上述核函数通过调用线程和线程掩码实现了两个线程之间的蝴蝶寻址模式。执行流程及结果如图 2.12 所示。

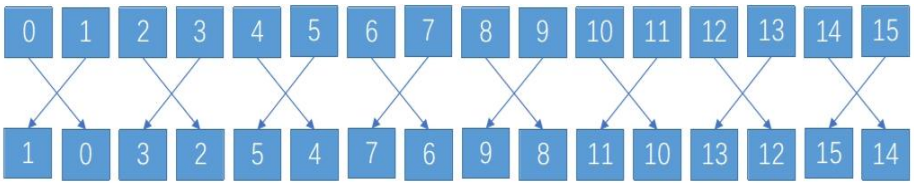


图 2.12 束内线程间的蝴蝶寻址

2.4.5 原子函数

原子操作实现了对在多个线程间共享的变量的互斥保护，确保任何一次对变量的操作的结果的正确性。原子操作确保了在多个并行线程间共享的内存的读写保护，每次只能有一个线程对该变量进行读写操作，一个线程对该变量操作的时候，其他线程如果也要操作该变量，只能等待前一线程执行完成。原子操作确保了安全，代价是牺牲了性能。

以 HIP 中的 atomicCAS 函数来实现一个原子级 32 位整型加法运算。atomicCAS 的函数签名为：

```
int atomicCAS(int *address, int compare, int val);
```

通过原子级 CAS，可以在 HIP 中定义自己的原子函数。CAS 将 3 个内容作为输入：内存地址、存储在此地址中的期望值，以及实际想要存储在此位置的新值。CAS 中将会读取目标地址并将该处地址的存储值与预期值进行比较。如果存储值与预期值相等，那么新值将存入目标位置。 如果存储值与预期值不等，那么目标位置不会发生变化。不论发生什么情况，一个 CAS 操作总是返回目标地址中的值。使用返回值可以用来检查一个数值是否被替换成功。如果返回值等于传入的预期值，那么 CAS 操作一定成功了。上文中“address”代表目标内存地址，“compare”是预期值，“val”代表实际要写入的新值。

那么，通过 atomicCAS 实现加法操作就需要分析起始状态和结束状态。原子加法中，起始状态是加法运算的基值，结束状态是起始状态和增量的和。转化为 atomicCAS，“compare”是起始状态，“val”是结束状态。确定目的地址（address）和输入的值（incr），读取 address 可以得到预期值（expected）。与输入值相加可得到实际写入的新值。确定地址、预期值和实际写入的值后，就可以调用 atomicCAS 来实现加法运算：

```
__device__ int myAtomicAdd(int *address, int incr) {
```

```
    // Create an initial guess for the value stored at *address
    int expected = *address;
    int oldValue = atomicCAS(address, expected, expected + incr);
    ...
}
```

目标位置是由多线程共享的，当一个值处于被“expected”读入和 atomicCAS 修改之间，另一个线程修改“address”的值是有可能的。利用这一点可以用来检查失败，在一个循环中重试直到 atomicCAS 成功。

```
__device__ int myAtomicAdd(int *address, int incr) {
    // Create an initial guess for the value stored at *address
    int expected = *address;
    int oldValue = atomicCAS(address, expected, expected + incr);

    // Loop while expected is incorrect
    while (oldValue != expected) {
        expected = oldValue;
        oldValue = atomicCAS(address, expected, expected + incr);
    }
    return oldValue;
}
```

上述代码中，若第一个 atomicCAS 失败，oldValue 值不会改变并与 expected 相等。函数 myAtomicAdd 会循环执行，预期值重置为最近读取的值。直到 atomicCAS 成功，返回值与预期值不同。以上以 atomicCAS 的使用为例，其他原子函数如表 2.5。

表 2.5 原子操作常用函数

操 作	函 数	支持的数据类型
加法	atomicAdd	int, unsigned int, unsigned long long int, float
减法	atomicSub	int, unsigned int

无条件替换	atomicExch	int, unsigned int, unsigned long long int, float
最小值	atomicMin	int, unsigned int, unsigned long long int
最大值	atomicMax	int, unsigned int, unsigned long long int
增量	atomicInc	unsigned int
减量	atomicDec	unsigned int
CAS	atomicCAS	int, unsigned int, unsigned long long int
与	atomicAnd	int, unsigned int, unsigned long long int
或	atomicOr	int, unsigned int, unsigned long long int
异或	atomicXor	int, unsigned int, unsigned long long int

使用原子函数会在一些应用中保证正确性，但需要付出性能上的代价。以原子加法 atomicAdd 为例，与不使用原子操作的普通加法进行对比：

```
__global__ void normal(int *var, int *read, int N, int iters)
{
   int i;
   int tid = hipBlockIdx_x * hipBlockDim_x + hipThreadIdx_x;
   if (tid >= N) return;
   int value = *var;
   *var = value + 1;
   read[tid] = value;
   for (i = 0; i < iters; i++)
   {
      int value = *var;
      *var = value + 1;
   }
}
```

使用原子操作的函数可将上述代码中的 value 相关内容替换为：

```
atomicAdd(var, 1);
```

对比两函数一个示例的输出，可得到如表 2.6。

表 2.6 原子操作常用函数

加法函数	耗时	结果数量	部分线程读取值
原子加法函数	1.166637s	6400064	0 1 2 3 14 15 18 19 24 26
普通加法函数	0.001739s	100001	0 0 0 0 0 0 0 0 0 0

在耗时上两函数有明显的差别，原子加法函数因每次只能有一个线程对某变量进行读写操作，耗时远远大于另一加法函数。读取的值更能体现出两函数结果的冲突，原子函数中线程有独有的增量，而非每个线程从 0 开始增大并全部写入相同的值。

以上示例表明原子操作会导致耗时增加，但对函数性能上的优化需要在保证正确性的前提下进行。

2.4.6 内置数据类型

- 短向量类型

在进行 HIP 编程时，涉及到一些想要使用较少的代码控制更多数据的实际情况，比如向量乘法中一列中多个元素和一行中多个元素的乘加操作。这种情况下常常使用短向量类型来描述所需要的变量。由于结构以及定义的相似性，在学习时可以类比 c 语言中的复合类型（Compound type）来理解短向量类型。

短向量类型是从基本整数和浮点类型派生的向量类型，也就是基于 char，short，int，long，longlong，float，double 这些类型所构建的结构体。它们是定义在 hip_vector_types.h 中，都带形如 make_的构造函数；例如：

```
double3 make_double3(double x,double y,double z);
```

它创建了一个类型为 double3 的向量，其值为(x，y，z)，实际上可以把一个 double3 数据看作是连续的三个 double 数据的结构体，而 make_double3.x，make_double3.y，make_double3.z 分别代表结构体中的第一，第二，第三个数据。如图 2.13 所示，将连续的 double 类型数据分割成了 double3 类型的数据。

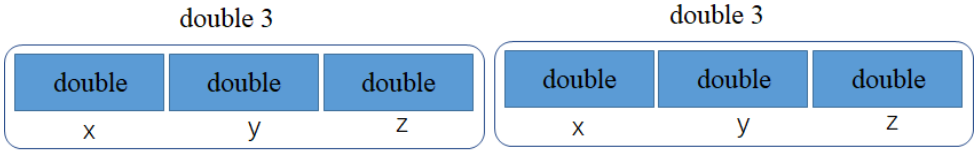


图 2.13 六个 doubule 数据分为两个 double3 数据

2.5 编译与运行

当使用 HIP 完成 DCU 程序的编写后，需要使用 hipcc 编译器将源码编译为可执行程序。与 CPU 程序一样，使用 hipcc 对 HIP 源码编译时也需要分别进行编译和链接步骤，每个步骤会输出对应的文件，并用于下一个阶段的处理，直至生成可执行文件或动态链接库。使用特定的编译参数，可以从源码生成不同阶段的文件，hipcc 编译器所使用的文件类型后缀和对应的生成参数可以通过表 2.7 查看。

表 2.7 编译阶段涉及到的文件及意义

文件后缀	作用
.cpp/.hip	hip 源文件，包括主机端和设备端代码
.ii	经过预处理的 hip 源文件，编译选项为--precompile 或 -E
.bc	生成的中间二进制文件
.s	生成的中间汇编文件，编译选项为-S
.o	目标文件，编译选项为-c
exe	链接目标文件，生成的可执行文件

在第 1 章节我们已经使用 hipcc 完成对 HIP 程序的编译和链接，如果希望在编译和链接的阶段做更多的处理，则需要增加额外的参数完成对应的功能。下述指令展示了如使用-c 参数将 HIP 程序 src1.cpp 编译为目标文件 object.o，完成独立的编译过程。在编译的过程中，使用-I 参数添加额外的 C/C++头文件路径${C_INCLUDE_PATH}，帮助编译器正确的寻找 HIP 程序所需的头文件。当完成编译后，就可以进一步对 HIP 程序的目标文件进行链接，可使用-L 选项指定链接时所用到的库文件路径，-l 选项指定链接时所用到的库名，编译器将最终

完成对程序的链接，并生成可执行程序。

```
hipcc -c -o object.o src1.cpp -I${C_INCLUDE_PATH}
hipcc -o exec object.o -L${LIBRARY_PATH} -l${LIBRARY_NAME}
```

表 2.8 给出了常用的 hipcc 参数，“--offload-arch”参数可以指定 HIP 程序中设备端程序编译的目标设备。“-x hip”可以将不是以.cpp 或.hip 结尾的源码文件作为 HIP 程序，按照 HIP 程序的编译和链接规则进行处理。如果想了解更详细的选项内容，可以通过 hipcc --help 来获取。

表 2.8 编译器常见参数

参数	作用
--offload-arch=<arg>	Offloading Device Architecture
-x hip	Treat subsequent input files as hip source file(.hip)
-Xarch_device	Pass <arg> to the HIP device compilation
-Xarch_host	Pass <arg> to the HIP host compilation
--rocm-path	ROCm installation path, used for finding and automatically linking required bitcode libraries.
--hip-link	Link clang-offload-bundler bundles for HIP
-ffp-contract	Form fused FP ops (e.g. FMAs)
-fno-gpu-rdc / -fgpu-rdc	Generate relocatable device code, also known as separate compilation mode

与传统程序设计语言不同的是，HIP 是一种异构计算编程模型，程序不仅有 CPU 上执行的部分，同时也有在 DCU 上执行的部分。因此，hipcc 需要同时编译主机端可执行程序和设备端可执行程序。将主机端代码编译为主机端可执行的程序，同时也需要将设备端代码编译为设备可执行的程序，并且将两者合并为一个可执行程序。hipcc 在编译 HIP 程序时的过程可以分为 4 个阶段：

- 对源码文件进行解析，分离出设备端源程序 Device Code 和主机端源程序 Host Code。
- 编译器对主机端源程序编译和链接，生成主机端可执行程序。
- 编译器对设备端源程序编译和链接，以针对不同的目标设备生成多种设

备端可执行程序。

- 将主机端可执行程序和多个设备端可执行程序合并为一个 Fat Binary。

为了满足复杂编译过程的需求，hipcc 提供了“-fno-gpu-rdc“和“-fgpu-rdc”参数，gpu-rdc 是指是否为设备端源程序生成可重定向的代码，可以对应图 2.14 中的 Kernel ISA Object 部分，此时设备端源程序已经被编译，但没有进行链接。如果使用其他链接器，则不能识别设备端可重定向代码，无法完成这个步骤，因此需要使用“-fno-gpu-rdc“参数，添加这个参数时，设备端源程序总是会在编译时就完成链接，直接生成设备端可执行程序。

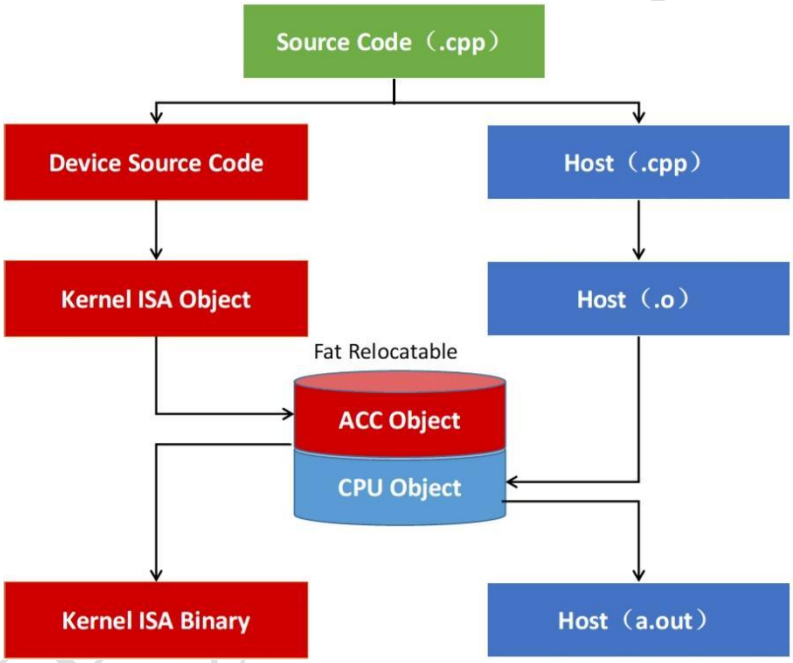


图 2.14 编译链接流程图

2.6 总结

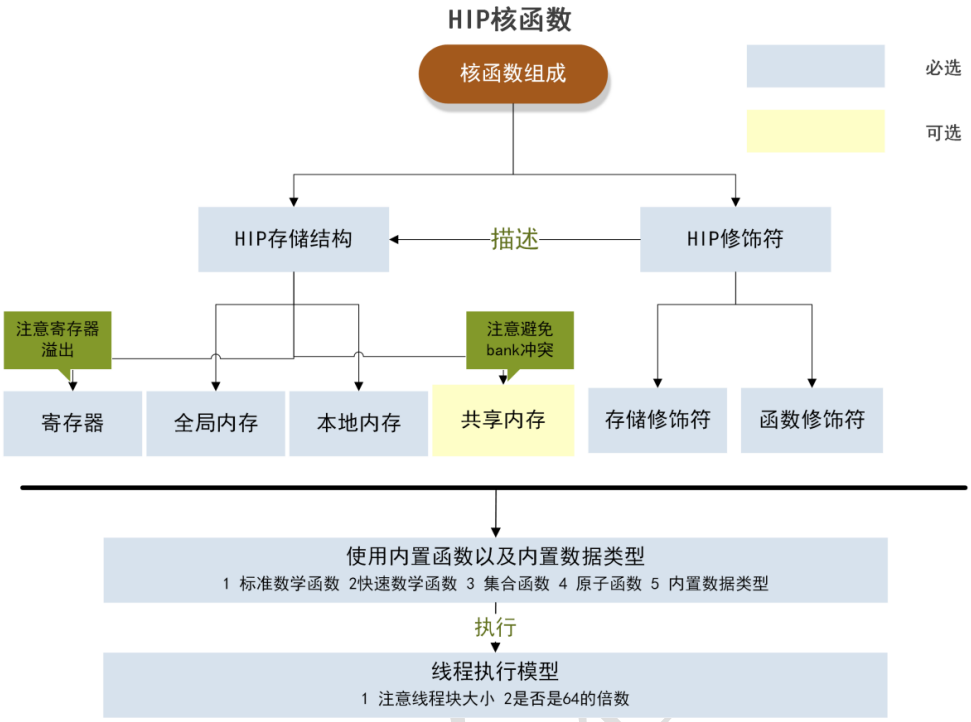


图 2.15 核函数小结

开发者

3 全局内存管理与数据传输

在上面的章节中我们已经讲解了 DCU 上的存储结构如寄存器、共享内存等以及如何通过修饰符来使用这些存储资源，这些内容都是基于数据已经传输到了 DCU 中，那么 DCU 中的数据是如何与 CPU 数据进行交换的呢？在本章中，我们将会着重了解 DCU 全局内存与主机内存的通信方式以及如何管理全局内存。

3.1 全局内存

全局内存（Global Memory）是整个 DCU 中容量最大、最常使用但延迟最高的内存。“全局”指的是它贯穿于程序的整个生命周期，并且可以被任何设备访问。我们的 DCU 编程体系是一个 CPU-DCU 异构计算体系，每个异构系统都有独立的内存空间。前面所讲的核函数就是在设备内存空间中运行的，我们的 HIP API 可以进行分配和释放设备内存。

接下来，将讲解如何对全局内存进行查询、分配和释放等操作，以及 Symbol 的概念。

- 查询接口

如果想查询全局内存信息，则可以使用下面接口：

```
hipError_t hipMemGetInfo(size_t *free,size_t *total)
```

其中，参数 free 返回的是当前可用内存的快照，参数 total 返回的是设备上可分配的总内存。

- 分配与释放接口

使用下面接口你可以进行全局内存分配：

```
hipError_t hipMalloc(void** ptr, size_t size);
```

其中，指针 ptr 返回的是该内存的地址，参数 size 表示要分配的内存大小。如果将参数 size 设置为 0，则表示不分配内存，指针 ptr 返回的是空地址。

使用以下接口可以进行全局内存的释放：

```
hipError_t hipFree(void *ptr);
```

其中，指针 ptr 指向的是要释放的内存地址。

- Symbol 定义

Symbol 是定义在全局的显存变量，可以用__device__限定符修饰：

```
__device__ float devdata;
```

在主函数中，全局变量的值通过下面的函数进行初始化：

```
hipMemcpyToSymbol(devdata, &value, sizeof(float));
```

可以使用下面的函数将 Symbol 的值从设备端复制回主机端：

```
hipMemcpyFromSymbol(&value, devdata, sizeof(float));
```

在核函数中，参数 devdata 被当作全局内存中的一个变量，它只是一个在 DCU 上表示物理位置的标识符，并不是设备全局内存的变量地址，但是可以显式地使用下面的 API 调用来获取一个全局变量的地址：

```
hipError_t cudaGetSymbolAddress(void** devptr, const void* symbol);
```

这个函数用来获取与提供设备符号相关的全局内存的物理地址。获得变量 devdata 的地址后，你可以使用 hipMemcpy 函数，代码如下：

```
float *dptr = NULL;hipGetSymbolAddress((void**)&dptr, devdata) ;
hipMemcpy(dptr, &value, sizeof(float), hipMemcpyHostToDevice);
```

开发者社区

3.2 页锁定内存

在上述例子中首先要在主机端申请内存，主机内存即系统中被 CPU 访问的内存，分为两种类型

- 可分页内存（pageable memory）

所谓可分页，就是可能会被页换出（Page out），它会按照系统的要求将主机虚拟内存上的数据移动到其它的位置。在上一小节中我们默认分配的就是可分页内存。当我们从可分页内存传输数据到设备内存时，主机会首先分配一个临时的固定主机内存（不可被页换出），然后将数据复制到固定内存中，再从固定内存传输数据给设备内存。

- 页锁定内存（page-locked memory 或 pinned memory）

页锁定内存就是固定内存，相当于是固定了地址，DCU 在知道了这个地址后，就可以使用 DMA 传输单元直接访问，如图 3.1 所示。如果你需要频繁在主机端和设备端传输数据，使用页锁定内存对性能提升较大。

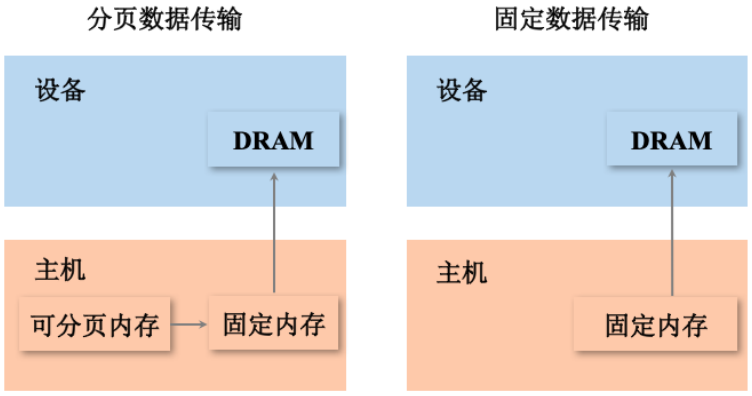


图 3.1 分页数据传输（左）和固定数据传输（右）

- 页锁定内存分配

设备端在主机端使用如下 API 申请页锁定内存：

```
hipError_t hipHostMalloc(void **ptr, size_t size, unsigned int flags);
```

其中，指针*ptr 指向分配的内存地址，大小为 size 字节，标志 flags 表示主机内存分配的类型，设置不同的标志位可以分配不同的内存类型，具体有以下几种：

- 当标志位设置为 hipHostAllocPortable 时，分配的是可共享锁页内存

（portable memory）。一般情况下，锁页内存只能由分配它的 CPU 线程访问，而可共享锁页内存允许被多个 CPU 线程访问，从而减少 CPU 线程间的数据传输和通信。

- 当标志位设置为hipHostAllocWriteCombined时，分配的是写结合锁页内存（write-combined memory）。一般情况下，CPU 对一块内存中的数据进行操作时，会先将这块内存上的数据缓存到 Cache 中，并监视这块内存的数据是否更改，以保证缓存数据的一致性。但由于在 CPU-DCU 异构计算中，要计算的数据大多由 DCU 操作，所以可选择不让 CPU 对数据进行缓存并监视。当你分配了写结合锁页内存后，CPU将不会对这部分内存进行缓存，也避免了CPU通过PCI-E总线对内存的监视，从而提高了主机端-设备端传输带宽。
- 当标志位设置为 hipHostAllocMapped 时，分配的是映射锁页内存（mapped memory）。映射锁页内存又称零拷贝内存，简单说就是不需要将数据从一个存储区域复制到另一个存储区域，因为映射锁定内存在主机和设备之间是共享的，你可以在核函数中直接访问映射锁页内存中的数据，不需要再进行数据拷贝。

➢ 页锁定内存释放

释放页锁定内存，则必须使用如下 API：

```
hipError_t hipHostFree(void* ptr);
```

➢ 注册页锁定内存

除了上述使用特定的 API 直接申请页锁定内存外，你还可以通过使用注册页锁定内存 API，来对一个已分配的可分页内存进行页锁定。注册页锁定内存会设置一些内部标志以确保内存不被换出，并告知该设备这块内存为锁页内存，可以直接使用它而不需要使用临时缓冲区。

我们可以使用如下 API 进行注册页锁定内存：

```
hipError_t hipHostRegister(void *hostPtr, size_t sizeBytes, unsigned int flags)
```

其中，指针 hostPtr 指向要注册的主机内存地址，内存大小为 sizeBytes，参数标志 flags 有以下几个可选项：

- hipHostRegisterDefault ：内存将被映射且可移植。

- hipHostRegisterPortable： 内存会被所有上下文视为固定地址，而不只是执行分配的那个。
- hipHostRegisterMapped：将分配映射到当前设备的地址空间。

如果你想取消注册的页锁定内存，使用如下 API，指针 hostPtr 指向的是想要取消的内存地址：

```
hipError_t hipHostUnregister(void *hostPtr)
```

开发者社区

3.3 内存亲和性

- NUMA

为了更高效的使用全局内存，我们使用 NUMA（Non-Uniform Memory Access，非统一内存访问）架构。随着现代计算机 CPU 核心数的不断增加，传统的统一内存访问（UMA）架构出现了带宽拥挤、访问冲突等问题。如图 3.2 所示，采用 NUMA 架构，给各个 CPU 核心都分配一块与自身距离最近的内存块，这就是一个 NUMA node，而每个 NUMA node 中都会存在一个或多个 DCU，在 node 内部进行全局内存的使用，对性能会有较高的提升。

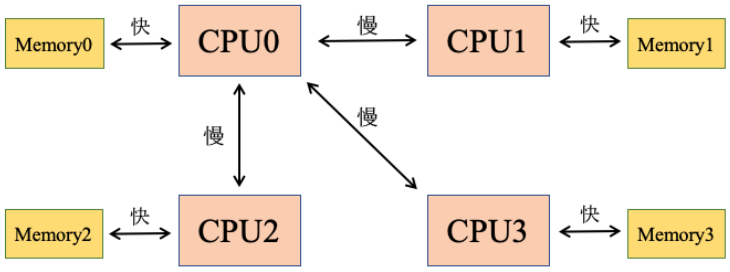


图 3.2 NUMA 架构图

- CPU Affinity

CPU Affinity 指的是 CPU 的亲和性。如图 3.3 所示，CPU1 和 CPU2 之间通过核间总线相连，CPU1 和 CPU2 通过 PCI-E 总线各自连接 4 个 DCU，CPU1 与相对应的 DCU1-4 可以进行直接快速的数据交换，但是如果与 DCU5-8 进行数据交换就只能通过 CPU2 进行中转。由此，我们可以说 DCU1-4 与 CPU1 有较高的亲和性，DCU5-8 与 CPU2 有较高的亲和性。

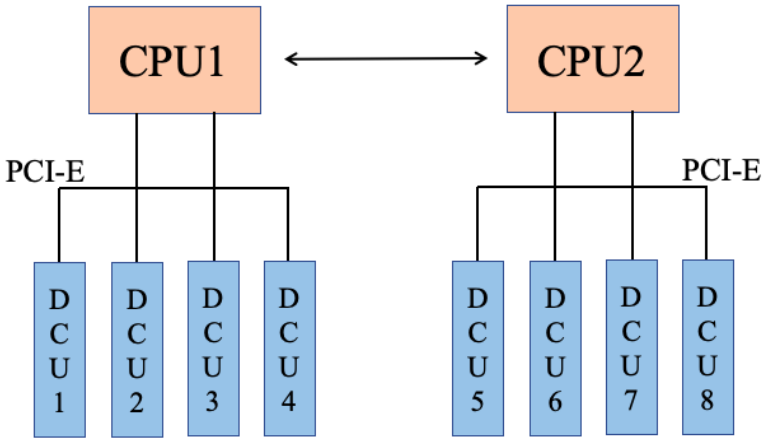


图 3.3 CPU Affinity

3.4 数据传输

从 HIP 编程存储机构中可以看出，主机端和设备端的存储是独立的，所以数据传输是必不可少的。当你需要计算的数据位于主机端时，必须先将主机端内存的数据传输给设备端内存，当设备端计算结束后，再将数据传输回主机端内存。

- 同步内存复制

同步内存复制含义为当使用这类内存拷贝操作时，CPU 端的操作会被阻塞，等待拷贝的完成才进行下一步指令，你可以使用下列函数进行同步内存复制：

```
hipError_t hipMemcpy (void *dst, const void *src, size_t sizeBytes, hipMemcpyKind kind);
```

其中变量 kind 指定了复制的方向，可以有下列取值：

- hipMemcpyHostToHost 从主机端复制到主机端
- hipMemcpyHostToDevice 从主机端复制到设备端
- hipMemcpyDeviceToHost 从设备端复制到主机端
- hipMemcpyDeviceToDevice 从设备端复制到设备端

这个函数根据 kind 指定的方向，把 src 地址中的数据复制到 dst 地址内存中，大小为 count 字节。需要注意的是，指针 dst 和 src 不能重叠。另外如果指针 dst 和 src 与 kind 指定的复制方向不一致，那么 hipMemcpy 的行为就是未定义行为。

- 异步内存复制

数据传输也可以异步进行，所谓异步执行指的是当使用这类拷贝操作时 CPU 端操作不受影响，拷贝开始时拷贝过程由 DMA 来管理，异步拷贝需要配合流操作来一起使用，可以使用以下函数：

```
hipError_t hipMemcpyAsync (void* dst, const void* src, size_t sizeBytes,hipMemcpyKind kind, hipStream_t stream __dparm(0));
```

注意附加的流标识符作为第五个参数。默认情况下，流标识符被设置为默认流。这个函数与主机是异步的，所以调用发布后，控制权将立即返回到主机。将复制操作和非空流进行关联是很容易的，但是首先需要使用如下代码创建一个非空流：

```
hipError_t hipStreamCreate (hipStream_t* stream);
```

其中，hipStreamCreate 创建了一个可以显式管理的非空流(关于流详细的介绍可以参看 HIP 并发执行章节)，之后返回到 stream 中的流就可以被当作流参数供 hipMemcpyAsync 和其他异步 hip 的 API 函数来使用。

- 数据拷贝带宽测试

```
#include
__global__ void test(int n, float a, float *x, float *y) {
   ...
}
int main(void) {
   ...
   hipEventRecord(start);
   hipMemcpy(d_x, x, N * sizeof(float), hipMemcpyHostToDevice);
   hipEventRecord(stop);
   ...
   hipLaunchKernelGGL(test, dim3((N + 511) / 512), dim3(512), 0, 0, N, 2.0f, d_x,
d_y);
   ...
   printf("Effective Bandwidth (GB/s): %f\n", N * 4 / milliseconds / 1e6);
}
```

- 结构体数组与数组结构体传输

首先介绍数组结构体(Array of Structure，AoS)和结构体数组(Structure of Array，SoA)，当存储结构化数据集时，它们代表了可以采用的两种强大的数据组织方式（结构体和数组）。

下面是存储成对的整型数据元素集的例子。首先，考虑这些成对数据元素集如何使用 AoS 方法进行存储。如下定义一个结构体，我们将它命名为 inStruct：

```
struct inStruct {
   int *x;
   int *y;
};
```

然后，按照下面的方法定义这些结构体数组：

```
struct inStruct myAoS [N] ;
```

这是利用 AoS 方式来组织数据的，它存储的是空间上相邻的数据（例如 x 和 y），这在 CPU 上会有良好的缓存局部性。

接下来，考虑使用 SoA 方法来存储数据：

```
struct inArray {
   int *x[N];
   int *y[N];
};
```

这里，在原结构体中每个字段的所有值都被分到各自的数组中。这不仅能将相邻数据点紧密存储起来，也能将跨数组的独立数据点存储起来。然后，可以使用如下结构体定义一个变量：

```
strcut inArray mySoa ;
```

图 3.4 展示了 AoS 和 SoA 的内存布局。在使用 AoS 内存布局时，假设 DCU 存储 x 字段数据，并执行一个只有 x 字段的应用程序，将会导致带宽损失，因为 y 值也会被隐式地加载，同时也浪费了缓存空间。如果使用 SoA 内存布局来存储数据，则可以充分利用 DCU 的内存带宽。由于没有相同字段元素的交叉存取，DCU 上的 SoA 内存布局提供了合并内存访问，并且可以对全局内存实现更高效的利用。

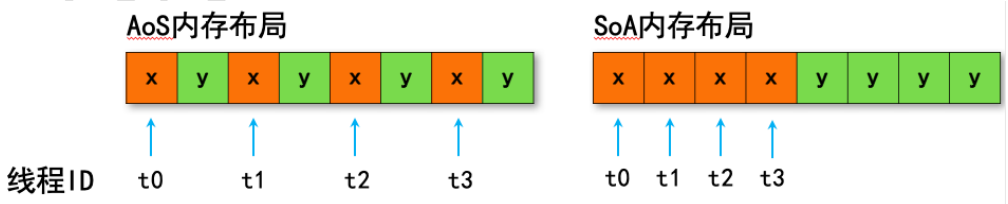


图 3.4 AoS 和 SoA 内存布局

3.5 总结

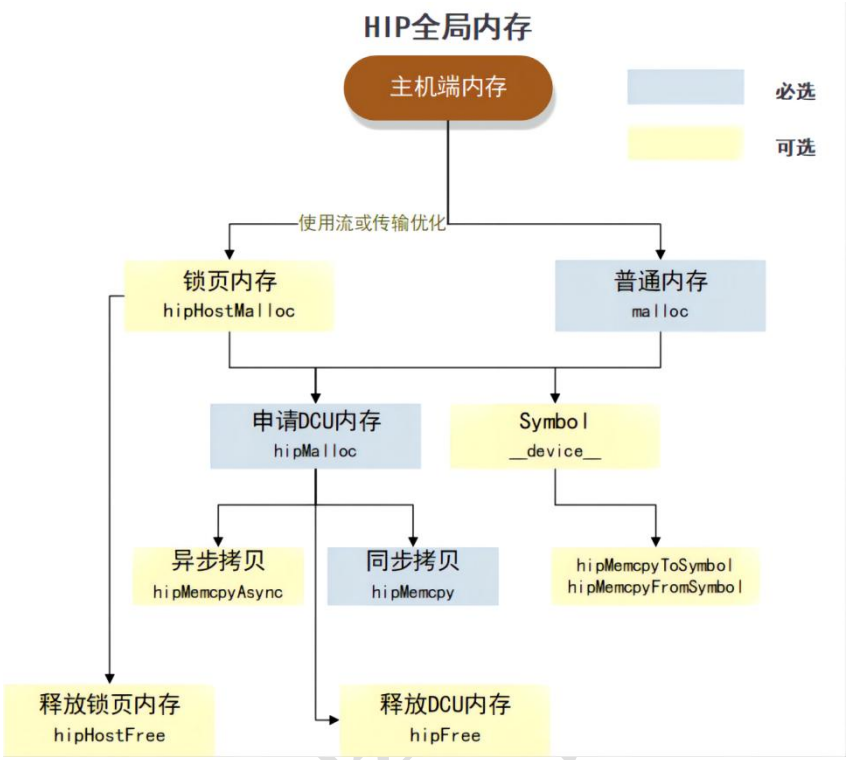


图 3.5 全局内存总结

开发者

4 HIP 并发执行

在 HIP 编程中，常会遇到三个经典操作，即：

- 将所需数据内存从主机端拷贝到设备端
- 在设备上执行内核运算
- 将结果内存从设备端拷贝到主机端

多数情况下，尤其是在进行较复杂的运算时，核函数的执行和数据拷贝可以以工作队列的形式进行编队，每一个队列就是一个流，善用流来进行规划计算过程可以提高 DCU 的利用率。用户可以通过流的调度，重叠以上三个操作，将主机与设备之间的数据拷贝隐藏在内核执行之下，从而使得程序总运行时间缩短。多流调度不仅仅可以应用在单个 DCU 上，在多个 DCU 上甚至也会有更好表现。

4.1 HIP 流

4.1.1 什么是流

HIP 流是包含着一串异步 HIP 调用，如异步拷贝、将核函数发送到指定工作队列、再通过异步拷贝将数据拷回主机端。流的作用不仅仅是封装这些操作，还需保证操作的顺序，同时支持查询排队操作的状态。可以将每个流视为 DCU 上的一个任务，并且这些任务可以并行执行。通俗一点来讲，流就是一个“班车”，每一个操作作为一个乘客坐上“班车”，按照代码确定的顺序一个一个执行，每一个流都是一趟独立的“班车”。

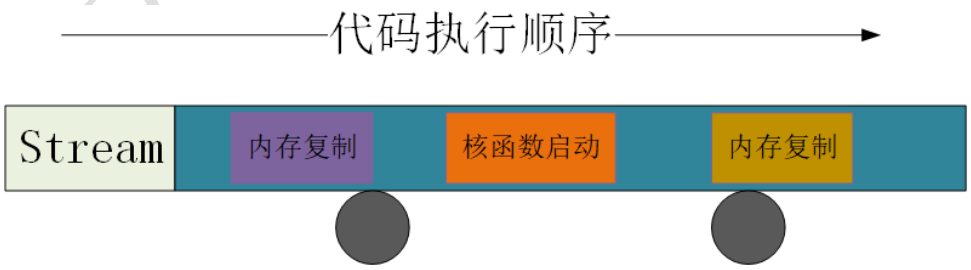


图 4.1 流定义

从主机端的角度看，流中一系列操作的执行总是异步的。runtime 决定操作在设备上执行的时间。而我们需要通过合理运用 HIP 的 API 来保证一个异步操作运行结果在被下一个操作使用之前完成。在同一个 HIP 流中的操作有严格的

执行顺序，而在不同 HIP 流中的操作在执行顺序上不受限制。使用多个流同时执行多个核函数，可以实现流间并发。

所有的 HIP 操作（包括内核和数据拷贝）都在一个流中显式或隐式地运行。流分为两种类型：隐式声明的空流（默认流），显式声明的流（非空流）。

若没有显式指定一个非空流时，那么内核执行和数据拷贝将使用默认流，需要注意的是默认流会造成流间同步，也就是说一旦设备端在执行默认流的代码，那么其余流就陷入阻塞状态。

另一方面，使用非空流可以很好的优化程序，解决例如数据拷贝和核函数执行时间重叠的问题。因为在 HIP 中非空流可以被显式地创建和管理。如果想要不同的 HIP 操作并行执行，就可以使用非空流。基于流的异步的内核启动和数据拷贝可以带来的好处：

- 并行主机和设备的计算
- 并行主机的计算和主机与设备之间的数据拷贝

4.1.2 流的执行模型

图 4.2 是常见的默认流操作执行，在默认流中串行完成：主机端到设备端数据拷贝、内核执行、设备端到主机端内存拷贝。



图 4.2 串行执行

图 4.3 所示，展示了使用 3 个流的 HIP 操作，数据拷贝和内核计算都是均匀分布在 3 个并发流中的，与串行执行比较，有了一定程度上的性能优化。虽然数据拷贝操作分布在不同的流中，但却没有并发执行。这是由于 PCIe 总线共享资源冲突导致的。虽然从编程模型的层面来看这些操作互不干扰，但是因为它们共享同一个硬件资源，所以它们的执行必须是串行的。只有具备双工 PCIe 总线的设备可以同时进行两个数据拷贝，且两个数据拷贝必须遵循一个规则，即：在不同的流中以及不同的拷贝方向上。因此我们在图中可以观察到，在一个流中从主机到设备的数据拷贝与另一个流中从设备到主机的数据拷贝是重叠进行的。

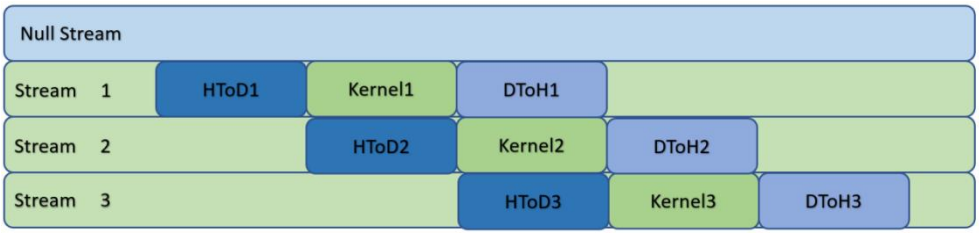


图 4.3 并行执行

在拥有足够硬件的情况下，理论上所有流可以同时运行，而事实上不一定是这样。本节将说明如何通过硬件调度多个 HIP 流内的并发内核操作。

目前 HIP 中所有的流运行时检查任务的依赖关系，如果仍有任务在执行，那么将会等待该任务依赖的任务执行完。而只有当所有依赖关系的任务都执行结束时，新任务才会被调度到可用的计算单元中。目前在软件层可以创建多个流，但是在硬件层面只有有限个硬件工作队列，所以超出硬件工作队列的流将会被阻塞等待相应的硬件工作队列。

- 不同硬件队列的不同依赖关系

比如 B 要等待 A 的结果，E 要等待 D 的结果，然而当我们把如下三个流塞到硬件队列中，如果硬件足够的情况，这个硬件队列中的任务可以并行执行，但是要考虑流之中的依赖关系，所以，我们按照各自流中操作顺序执行。但是当硬件队列不充足的情况：

1.执行 A，同时检查 B 是否有依赖关系，当然此时 B 依赖于 A 而 A 没执行完，所以整个队列阻塞。

2.A 执行完成后执行 B，同时检查 C，发现依赖，等待。

3.B 执行完后，执行 C 同时检查发现 D 没有依赖，如果此时硬件上有空闲的资源，D 开始执行。

4.D 执行时检查 E，发现 E 依赖 D，所以等待，接下来的情况也是如此。

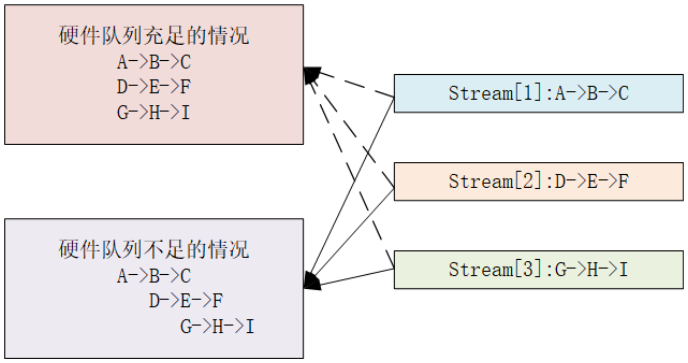


图 4.4 不同硬件队列下的流调度

- 流的优先级

某些特殊条件下需要将流设定优先级，使用下面的函数可以创建一个具有特定优先级的流：

```
hipStreamCreateWithPriority(&stream, hipStreamDefault, priority_normal);
```

该函数创建一个新的流，并赋予其 priority 的优先级，拥有高优先级的队列可以占用低优先级流资源执行。流优先级不会影响数据拷贝操作，只对计算内核有影响。此外，如果指定的优先级超出了设备可定义的范围，它会被自动定义成范围内的最低值或最高值。对于一个给定的设备，可以使用如下函数查询优先级的允许范围：

```
hipDeviceGetStreamPriorityRange(&priority_low, &priority_high)
```

这个函数的返回值存放在 priority_low 和 priority_high 中，分别对应于当前设备的最低和最高优先级。按照惯例，一个较低的整数值表示更高的优先级。如果设备不支持流优先级，hipDeviceGetStreamPriorityRange 将 0 返回给这两个参数。

而如果想要查询流的优先级可以使用下面函数:

```
hipStreamGetPriority(stream, &priority);
```

4.1.3 使用流

如下是一般使用默认流的场景，是否使用默认流需要配置三尖括号中的第四个参数，第四个参数指的是将该核函数发送到哪个流上运行，如果不写或者为 0 指的就是默认流：

```
hipMemcpy(....., hipMemcpyHostToDevice);
hipLaunchKernelGGL(Kernel, dimGrid, dimBlock);
hipMemcpy(....., hipMemcpyDeviceToHost);
```

下面列举了使用非默认流，stream[i]表示了该核函数将在目标流中执行。

```
Kernel_name <<< streamSize/WIDTH , WIDTH, 0,stream[i] >>>(A_d, B_d, C_d);
```

我们可以举一个在多流中使用 HIP 操作的常见例子，下面示例代码从声明流到创建流到使用异步数据拷贝最后到流的销毁，详细的介绍了多流的执行。

```
hipStream_t mystream[STREAM_NUM];//非默认流声明
for (int i = 0; i < STREAM_NUM; i++) {
hipStreamCreate(&mystream[i]);//非默认流进行创建
hipMemcpyAsync(&A_d[streamSize * i], &A_h[streamSize * i], streamBytes,
hipMemcpyHostToDevice, mystream[i]);
hipMemcpyAsync(&B_d[streamSize * i], &B_h[streamSize * i], streamBytes,
hipMemcpyHostToDevice, mystream[i]);
MatrixAddition_05 <<< streamSize / WIDTH, WIDTH, 0, mystream[i] >>>
(&A_d[streamSize * i], &B_d[streamSize
* i], &C_d[streamSize * i]);
hipStreamSynchronize(mystream[i]);
hipMemcpyAsync(&C_h[streamSize * i], &C_d[streamSize * i], streamBytes,
hipMemcpyDeviceToHost, mystream[i]);
}
...
for (int i = 0; i < STREAM_NUM; i++) {
   hipStreamDestroy(mystream[i]);//释放流中的资源
}
```

表 4.1 常用流初始化 API

函数名称	功能
hipError_t hipStreamCreate(hipStream_t * stream)	流的创建
hipError_t hipStreamDestroy(hipStream_t stream)	流的销毁
hipError_t hipStreamSynchronize(hipStream_t stream)	流同步
hipError_t hipStreamQuery(hipStream_t stream)	检查流中所有操作是否都完成

当在一个流中调用 hipStreamDestroy 函数时,如果该流中仍有未完成的工作,hipStreamDestroy 函数将等待流中的任务执行完毕后才会销毁流，与流相关的资源将被自动释放。

除了 hipStreamDestroy，HIP 还提供了两个函数来检查流中所有操作是否都已经完成：

```
hipError_t hipStreamSynchronize(hipStream_t stream);
hipError_t hipStreamQuery(hipStream_t stream);
```

hipStreamSynchronize 强制阻塞主机，直到在给定流中所有的操作都完成了。hipStreamQuery 会检查流中所有操作是否都已经完成，但在它们完成前不会阻塞主机。当所有操作都完成时 hipStreamQuery 函数会返回 hipSuccess，当一个或多个操作仍在执行或等待执行时返回 hipErrorNotReady。

4.1.4 事件

在使用流的相关 API 之前，我们需要先了解 HIP 中的事件。HIP 中事件指的就是在流中插入标记，这就好像在一条纸带上插入一个图钉，事件最常见用途为测量某个流中的核函数耗时，可以在核函数执行前后插入事件，测量两个事件之间的时间就是核函数运行时间。HIP 支持用户创建所需要的事件，事件是一个类型为 hipEvent_t 的结构体。

事件的主要用途为：

- 同步流的执行
- 在流中计时

HIP 的 API 接口提供在流中任意位置插入事件以及查询事件是否完成的函数。事件函数与流函数类似，但只作用于事件本身。事件标记了流执行过程中的一个点，用户可以检查正在执行的流操作是否到达该检查点。

首先通过表 4.2 熟悉事件函数。

表 4.2 常用事件函数列表

函数名称	功能
hipError_t hipEventCreate(hipEvent_t* event);	事件声明，并对该事件进行创建
hipError_t hipEventCreateWithFlags(hipEvent_t* event, unsigned flags);	事件声明，并对该事件进行创建且特殊标记
hipError_t hipEventRecord(hipEvent_t event, hipStream_t stream);	在指定流中记录事件
hipError_t hipEventQuery(hipEvent_t event);	检查事件完成状态
hipError_t hipEventSynchronize(hipEvent_t event);	同步事件，阻塞主机端线程的调用直到事件完成
hipError_t hipEventElapsedTime(float* ms, hipEvent_t start, hipEvent_t stop);	计算被两个事件标记的 HIP 操作运行时间返回事件开始和事件结束之间的运行时间，以 ms 为单位。

hipError_t hipEventDestroy(hipEvent_t event);	销毁特定事件，当hipEventDestroy函数被调用时，如果事件尚未起作用，则调用立即返回，当事件被标记完成时自动释放与该事件相关的资源。

用户可以通过以下例子来了解事件从创建到销毁的一生：

```
//首先声明一个事件
hipEvent_t event;
//事件声明后,对该事件进行创建：
hipEventCreate(&event);
//将事件插入到 stream 中
hipEventRecord(event, stream);
...
//
hipEventSynchronize(event);
//事件执行完毕后，需要将事件销毁
hipEventDestroy(event);
```

合理使用事件函数可以实现流同步，计时等操作：

```
//声明事件
hipEvent_t start, stop;
//对已声明事件进行创建
hipEventCreate(&start);
hipEventCreate(&stop);
float eventMs = 1.0f;
// 记录开始事件
hipEventRecord(start, NULL);
// 内核执行
kernel<<<grid,block>>>(arguments);
// 记录结束事件
```

```
hipEventRecord(stop, NULL);
// 同步结束事件
hipEventSynchronize(stop);
//计算返回事件开始和事件结束之间的运行时间
hipEventElapsedTime(&eventMs, start, stop);
//销毁事件
hipEventDestroy(start);
hipEventDestroy(stop);
```

4.1.5 流的同步

非默认流中所有的操作都不会阻塞主机线程运行，因此会有在同一个流中运行主机和运算操作同步的场景。

从主机端来看，HIP 操作可以分为两大类：

- 内存相关操作，例如内存拷贝；
- 内核执行；
- 流有两种类型：
- 异步流（非空流）
- 同步流（空流/默认流）

在使用多流之前，必须先了解多流相关的规则：

- 流内的拷贝操作和核函数执行按照调用顺序执行。
- 默认流和非默认流混合使用的时候，默认流上的任何指令都会造成非默认流的同步，即只有当默认流中的指令执行完毕才能执行非默认流。

为了清楚展示默认流的阻塞行为，我们首先使用四个非默认流循环装载四个核函数，使四个核函数在每个流中都被调度：

```
for (int i; i < nstreams; i++) {
   kernel_1<<<grid, block, 0, stream[i] >>>(...);
   kernel_2<<<grid, block, 0, stream[i] >>>(...);
   kernel_3<<<grid, block, 0, stream[i] >>>(...);
   kernel_4<<<grid, block, 0, stream[i] >>>(...);
}
```

如图 4.5 是在四个创建的非默认流异步执行的场景，非默认流之间是可以并行执行的。

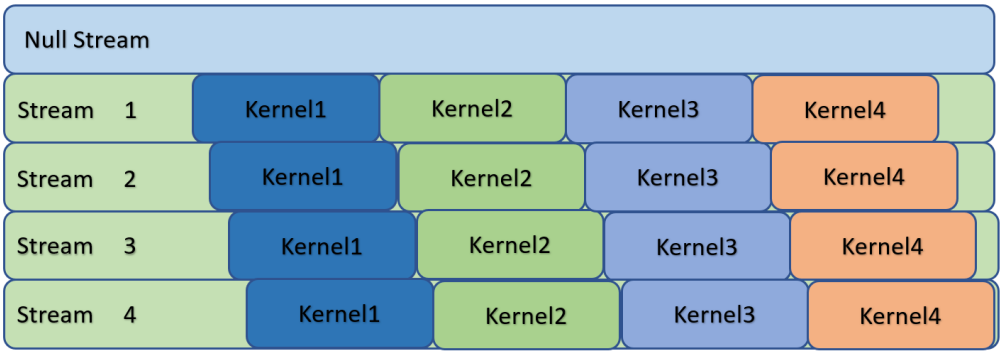


图 4.5 非默认流

而如果我们将其中一个核函数改为默认流执行，内核运行情况就会发生改变：

```
for (int i; i < nstreams; i++) {
   kernel_1<<<grid, block, 0, stream[i] >>>(...);
   kernel_2<<<grid, block, 0, stream[i] >>>(...);
   kernel_3<<<grid, block >>>(...);//调用默认流执行
   kernel_4<<<grid, block, 0, stream[i] >>>(...);
}
```

上述代码将 kernel_3 修改为在默认流中执行，如图 4.6 当出现默认流的时候，默认流会等待在它之前的非默认流队列执行完毕再执行，同时只有当默认流执行完毕才会执行其他的流。

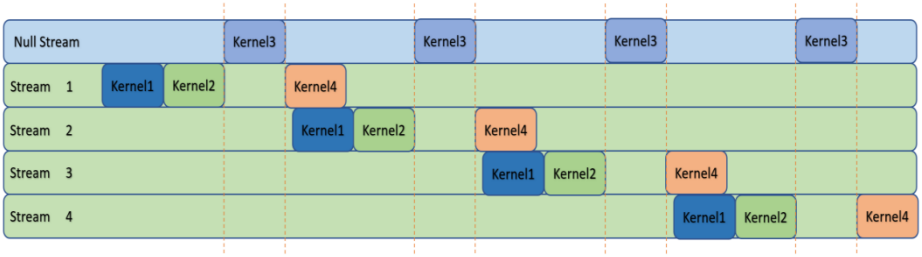


图 4.6 默认流与非默认流

- 隐式同步

调用 hipMemcpy 函数，可以隐式同步设备和主机，这是由于主机的应用程序在数据传输完成之前会被阻塞。注意使用隐式同步非常重要它会导致意想不

到的性能下降。

- 显式同步

支持显式同步 HIP 程序的几种方法：

- 同步设备，使用 hipDeviceSynchronize()这个函数使主机端等待所有设备相关的计算和通信完成。
- 同步流，使用 hipStreamSynchronize()函数只对指定的流和主机端进行同步操作，可用于同步主机和特定流，同时允许其它流继续执行。
- 同步流中的事件，使用 hipEventSynchronize()函数和 hipEventQuery()函数，事件也可以用于细粒度阻塞和同步。
- 使用事件同步，调用 hipStreamWaitEven()后加入到指定流的所有命令暂缓执行直到事件完成。

4.1.6 流回调

如果想要在设备端所有操作执行完的时候执行一个特定的主机端函数，可以使用流回调。流回调是一种特别的技术，有点像是事件的函数，这个回调函数被放入流中，当其前面的任务都完成了，就会调用这个函数，但是比较特殊的是，在回调函数中，需要遵守下面的规则。

- 回调函数中不可以调用 HIP 的 API
- 不可以执行同步
- 流函数有特殊的参数规格，必须写成下面形式参数的函数

```
hipError_t hipStreamAddCallback(hipStream_t stream, hipStreamCallback_t callback, void *userData, unsigned int flags)
```

下面的代码示例表明只有当 mystream 流中的所有工作都完成后，回调函数才开始在主机上运行,这个时候我们创建的主机端函数将会打印 Callback success! 并且返回一个 cbDone = true 。

```
void HIPRT_CB Callback(hipStream_t stream, hipError_t status, void* userData) {
   printf("Callback success!\n");
   cbDone = true;
}
 void main(){
```

```
    ......
    hipStream_t mystream;
    hipStreamCreateWithFlags(&mystream,
hipStreamNonBlocking);//hipStreamNonBlocking
    hipMemcpyAsync(A_d, A_h, Nbytes, hipMemcpyHostToDevice, mystream);
    const unsigned blocks = 512;
    const unsigned threadsPerBlock = 256;
    hipLaunchKernelGGL(vector_square,  dim3(blocks),  dim3(threadsPerBlock),  0,
mystream, C_d, A_d,N);
    hipMemcpyAsync(C_h, C_d, Nbytes, hipMemcpyDeviceToHost, mystream);


    hipStreamAddCallback(mystream, Callback, NULL, 0);
}
```

流与回调函数结合使用：

```
hipLaunchKernelGGL(vector_square,  dim3(blocks),  dim3(threadsPerBlock),  0,
mystream, C_d, A_d,N);
hipMemcpyAsync(C_h, C_d, Nbytes, hipMemcpyDeviceToHost, mystream);
hipStreamAddCallback(mystream, Callback, NULL, 0);
```

4.2 流的异步并发执行实例

在了解流、事件和同步的概念以及相关函数接口后，我们可以通过接下来的示例演示来掌握在多流中并发运行多个核函数。

首先写一个将要被执行的核函数，这是一个求平均值的核函数。

```
__global__ void MatrixAddition_03(int* a, int *b, int*c) {
    int threadID = blockIdx.x * blockDim.x + threadIdx.x;
    if (threadID < N) {
        c[threadID] = (a[threadID] + b[threadID]) / 2;
    }
}
```

如果想要多流并发，首先必须要创建一组非空流。这组非空流中，发布每

个流中的内核启动应该在 DCU 上同时运行，但是将 STREAM_NUM 的大小设定在合适的范围，应该就不存在由于硬件资源限制而导致的虚假依赖关系。

```
hipStream_t mystream[STREAM_NUM];
for (int i = 0; i < STREAM_NUM; i++) {
   hipStreamCreate(&mystream[i]);
}
```

使用一个 for 循环遍历所有的流，这样核函数在每个流中都会被调度多次：

```
for (int i = 0; i < STREAM_NUM; i++) {
   hipMemcpyAsync(A_d, &A_h[0], streamBytes, hipMemcpyHostToDevice, mystream[i]);
   hipMemcpyAsync(B_d, &B_h[0], streamBytes, hipMemcpyHostToDevice, mystream[i]);
   MatrixAddition_03 <<< streamSize / WIDTH, WIDTH, 0, mystream[i] >>> (A_d, B_d, C_d);
   hipMemcpyAsync(&C_h[0], C_d, streamBytes, hipMemcpyDeviceToHost, mystream[i]);
}
```

这些内核启动的执行配置被指定为单一线程块中的单一线程，以保证有足够的 DCU 资源能并发运行所有的内核。因为每个内核启动相对于主机来说都是异步的，所以可以通过使用单一主机线程同时调度多个内核到不同的流中。

在理想情况下，流之间不应该有非计划之内的依赖关系（即虚假的依赖关系）。然而，在复杂的应用程序中，流与流之间的关联在很多情况下是很紧密的，因此引入流间依赖关系是很有必要的，比如它可以在一个流中阻塞操作来等待另一个流中的操作完成再进行下一步的处理执行。另外通过事件可以用来添加流间依赖关系。

假如我们想让一个流中的工作在其他所有流中的工作都完成后才开始执行，那么就可以使用事件来创建流之间的依赖关系。首先，将标志设置为 hipEventDisableTiming，创建同步事件，代码如下：

```
for (int i = 0; i < NSTREAM; i++){
```

```
hipEventCreateWithFlags(&(kernelEvent[i]),hipEventDisableTiming);
}
```

接下来，使用 hipEventRecord 函数，在每个流完成时记录不同的事件。然后，使用 hipStreamWaitEvent 使最后一个流（即 streams[n_streams-1]）等待其他所有流,最后效果就是只有当其他的流中的操作队列执行完毕才会执行最后一个流中的操作队列：

```
for (int i = 0; i < NSTREAM; ++i){
    int ioffset = i * iElem;
    hipMemcpyAsync(&d_A[ioffset], &h_A[ioffset],
iBytes,hipMemcpyHostToDevice, stream[i]);
    hipMemcpyAsync(&d_B[ioffset], &h_B[ioffset],
iBytes,hipMemcpyHostToDevice, stream[i]);
    sumArrays1<<<grid, block, 0, stream[i]>>>(&d_A[ioffset],
&d_B[ioffset],&d_C[ioffset], iElem);
    hipMemcpyAsync(&dcuRef[ioffset], &d_C[ioffset],
iBytes,hipMemcpyDeviceToHost, stream[i]);
    hipEventRecord(kernelEvent[i], stream[i]);
    hipStreamWaitEvent(stream[NSTREAM - 1], kernelEvent[i], 0);
}
```

4.3 总结

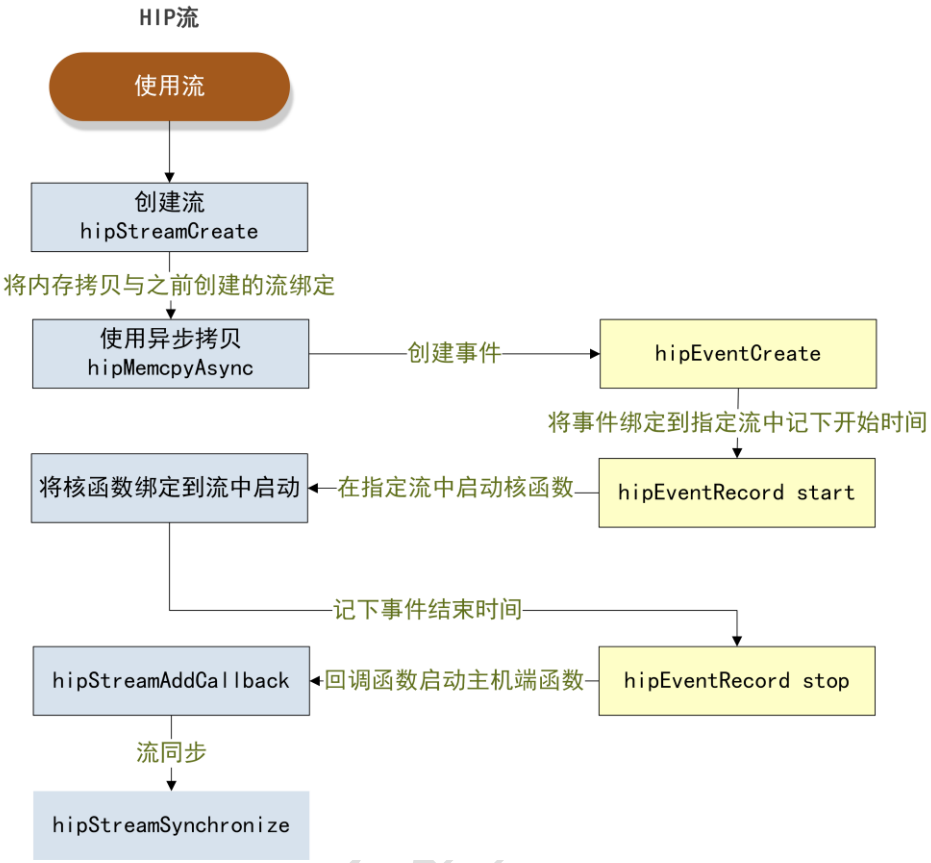


图 4.7 流与事件

4.4 hipGraph

4.4.1 hipGraph 简介

hipGraph 是一系列操作（包括内核启动），由依赖关系连接，独立于其执行定义。这允许一个 graph 被定义一次，然后重复启动。

使用 graph 的工作提交分为三个不同的阶段：定义、实例化和执行。

- 在定义阶段，程序创建 graph 中操作的描述以及 node 之间的依赖关系。
- 在实例化阶段，获取 graph 模块的快照，对其进行验证，并执行大部分工作的设置和初始化，目的是最大限度的减少启动时需要完成的工作，生成的实例称为可执行图。
- 在执行阶段，可执行图启动到流中，类似其他异步操作，它可以在重复实例化的情况下启动任意次数。

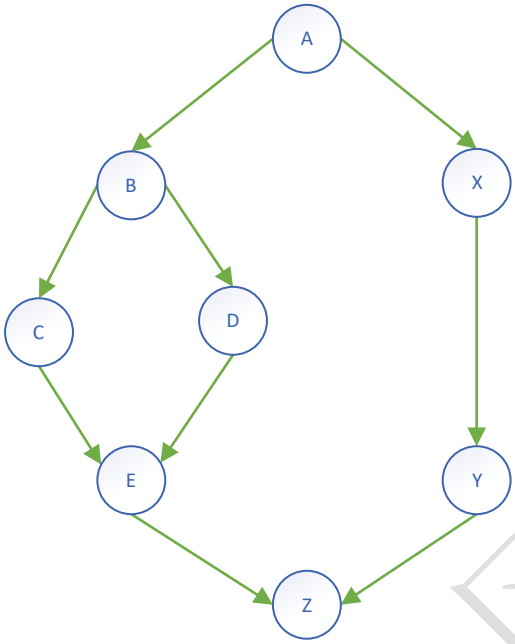


图 4.8 graph 架构

一个操作在 graph 中形成一个节点；操作之间的依赖关系是边；这些依赖关系限制了操作的执行顺序。

一个操作可以在它所依赖的节点完成后随时调度。

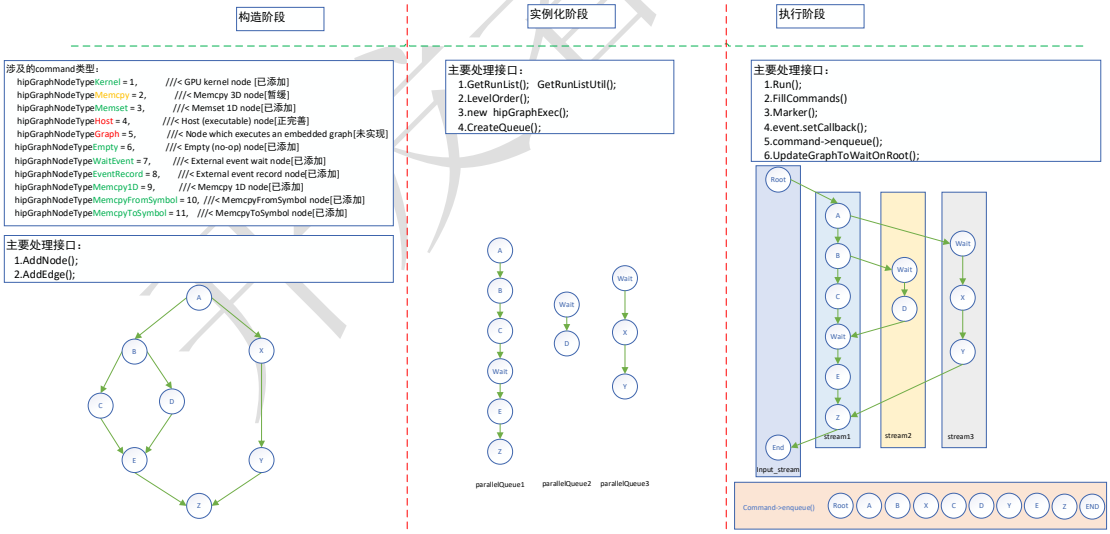


图 4.9 hipGraph 处理框图

4.4.2 函数接口说明

➢ hipGraph 创建

```
hipError_t hipGraphCreate(hipGraph_t* pGraph, unsigned int flags);
```

参数说明：

pGraph ：返回新创建的 graph, 输出参；

flags ： graph 创建的 flags，【必须是 0】;

返回值：

hipSuccess、hipErrorInvalidValue、hipErrorMemoryAllocation

作用：创建一个空的 graph，赋值给 pGraph

使用：

```
hipGraph_t graph = nullptr;
hipError_t status = hipGraphCreate(&graph, 0);
```

- hipGraph 实例化

```
hipError_t hipGraphInstantiate(hipGraphExec_t *pGraphExec,
                               hipGraph_t graph,
                               hipGraphNode_t *pErrorNode,
                               char *pLogBuffer,
                               size_t bufferSize);
```

参数说明：

pGraphExec ：返回实例化后的 graph 输出参；

graph ：将要实例化的 graph

pErrorNode ：在实例化错误的情况下，可以修改它以指示导致错误的节点

pLogBuffer ：用于存储诊断消息的字符缓冲区

bufferSize ： log buffer size

返回值：

hipSuccess or hipErrorInvalidValue

使用：

```
hipGraphExec_t graphExec = nullptr;
hipError_t status = hipGraphInstantiate(&graphExec,
                                        graph, NULL, NULL, 0);
```

- hipGraph 执行

```
hipError_t hipGraphLaunch(hipGraphExec_t graphExec, hipStram_t stream);
```

参数说明：

graphExec ：将要启动的实例化后的 graph

stream ： 在指定的 stream 上启动 graph,需要提前申请创建 stream；

返回值：

hipSuccess or hipErrorInvalidValue

使用：

```
hipError_t status = hipGraphLaunch(graphExec, streamForGraph);
```

- 释放资源

1. Graph exec 的释放

```
hipError_t status = hipGraphExecDestory(graphExec);
```

2. graph 的释放

```
hipError_t status = hipGraphDestroy(graph);
```

- kernel

```
hipError_t hipGraphAddKernelNode(hipGraphNode_t *pGraphNode,
                                 hipGraph_t graph,
                                 const hipGraphNode_t *pDependencies,
                                 size_t numDependencies,
                                 const hipKernelNodeParams *pNodeParams);
```

参数说明：

pGraphNode ：返回参，创建 kernel 类型的 graph node

graph ：将 memcpy node 添加到的 graph

pDependencies ：依赖 node 集合 [可为 NULL]

numDependencies : 依赖项个数

[如果 pDependencies 为 NULL，依赖个数须为 0]

pNodeParams ： GPU 执行函数参数，核函数相关参数

参数结构体定义如下：

```
typedef struct hipKernelNodeParams {
    dim3 blockDim;
    void** extra;
    void* func;
```

```
    dim3 gridDim;
    void** kernelParams;
    unsigned int sharedMemBytes;//对于每个线程 block 可共享的内存
} hipKernelNodeParams;
```

返回值：

hipSuccess、hipErrorInvalidValue、hipErrorInvalidDevice

使用：

```
std::vector<hipGraphNode_t> nodeDependencies;
hipGraphNode_t kernelNode = nullptr;
hipKernelNodeParams kernelNodeParams = {0};
void* kernelArgs[4] = {(void*)&inputVec_d, (void*)&outputVec_d,
                       &inputSize, &numOfBlocks};
kernelNodeParams.func = (void*)add; //kernel function name
kernelNodeParams.gridDim = dim3(gridSize, 1, 1);
kernelNodeParams.blockDim = dim3(blockSize, 1, 1);
kernelNodeParams.extra = NULL;
kernelNodeParams.sharedMemBytes = 0;
kernelNodeParams.kernelParams = (void**)kernelArgs;
hipError_t status = hipGraphAddKernelNode(&kernelNode, graph,
                                          NULL, 0,
                                          &kernelNodeParams);
```

如有依赖项，则：

```
nodeDependencies.push_back(memcpyNode);
nodeDependencies.push_back(memsetNode);
hipError_t status = hipGraphAddKernelNode(&kernelNode, graph,
                                          nodeDependencies.data(),
                                          nodeDependencies.size(),
                                          &kernelNodeParams);
```

- 拷贝

```
hipError_t hipGraphAddMemcpyNode1D(hipGraphNode_t *pGraphNode,
                                   hipGraph_t graph,
                                   const hipGraphNode_t *pDependencies,
                                   size_t numDependencies,
                                   void *dst, const void *src, size_t count,
                                   hipMemcpyKind kind);
```

参数说明：

pGraphNode ：返回参，创建 memcpy 类型的 graph node

graph ：将 memcpy node 添加到的 graph

pDependencies ：依赖 node 集合 [可为 NULL]

numDependencies ：依赖项个数

[如果 pDependencies 为 NULL，依赖个数为 0]

dst ：目的内存地址

src ：源内存地址

count ：拷贝字节数

kind ：拷贝方向类型如下

hipMemcpyHostToHost\ hipMemcpyHostToDevice\

hipMemcpyDeviceToDevice\ hipMemcpyDeviceToDevice\

hipMemcpyDefault

返回值：

hipSuccess、hipErrorInvalidValue

使用：

```
hipGraphNode_t memcpyNode = nullptr;
hipError_t status = hipGraphAddMemcpyNode1D(&memcpyNode, graph
                                            NULL, 0,
                                            inputVec_d, inputVec_h,
                                            sizeof(float) * inputSize,
                                            hipMemcpyHostToDevice);
```

- Memset

```
hipError_t hipGraphAddMemsetNode(hipGraphNode_t *pGraphNode,
                                 hipGraph_t graph,
                                 const hipGraphNode_t *pDependencies,
                                 size_t numDependencies,
                                 const hipMemsetParams *pMemsetParams);
```

参数说明：

pGraphNode : 返回参，创建 memset 的 graph node

graph : 将 memcpy node 添加到的 graph

pDependencies : 依赖 node 集合 [可为 NULL]

numDependencies : 依赖项个数

[如果 pDependencies 为 NULL，依赖个数为 0]

pMemsetParams : 内存清理参数 结构体

返回值：

hipSuccess、hipErrorInvalidValue、hipErrorInvalidDevice

使用：

```
hipMemSetParams memsetParams = {0};
memsetParams.dst          = (void*)outputVec_d; //目的内存地址
memsetParams.value        = 0;                  //待设置值
memsetParams.pitch        = 0;    //如果 height 为 1，则 unused
memsetParams.width        = 1;
memsetParams.height       = 1;
memsetParams.elementSize  = sizeof(float);
                                    //每个 element 字节，必须是 1、2 或 4
hipError_t status = hipGraphAddMemsetNode(&memsetNode, graph,
                                          NULL, 0,
                                          &memsetParams);
```

开发者社区

- 接口支持

DTK 版本	支持的创建方式
DTK-22.10.1	利用 API 显示方式创建
DTK-23.04.1/DTK-23.10	利用 API 显示创建 利用 stream capture 方式创建
DTK-25.04/DTK-25.04.1	新增部分 graph API 新增部分可以捕获的 API

表 4.2 DTK-23.04.1/23.10 支持的 hipGraph 接口

DTK-23.04.1/23.10	接口说明
hipGraphAddDependencies	添加依赖项
hipGraphAddEmptyNode	添加 empty node
hipGraphAddEventRecordNode	添加 event record node
hipGraphAddEventWaitNode	添加 event wait node
hipGraphAddHostNode	添加 host execution node
hipGraphAddKernelNode	添加 kernel exection node
hipGraphAddMemcpyNode1D	添加 1D memcpy node
hipGraphAddMemcpyNodeFromSymbol	添加 memcpy node 通过设备的 symbol 拷贝
hipGraphAddMemcpyNodeToSymbol	添加 memcpy node 拷贝到设备的 symbol
hipGraphAddMemsetNode	添加 memset node
hipGraphCreate	创建 graph
hipGraphDestroy	释放 graph
hipGraphEventRecordNodeGetEvent	获取关联 node 的 event
hipGraphEventRecordNodeSetEvent	设置记录 node 的 event
hipGraphEventWaitNodeGetEvent	获取关联等待 node 的 event
hipGraphEventWaitNodeSetEvent	设置等待 node 的 event
hipGraphExecDestroy	释放可执行 graph
hipGraphExecKernelNodeSetParams	为可执行 graph 指定的内核节点设置参数
hipGraphMemsetNodeSetParams	为可执行 graph 指定的 memset 节点设置参数
hipGraphGetEdges	获取 graph 中依赖关系
hipGraphGetNodes	获取 graph 的节点信息

hipGraphGetRootNodes	获取 graph 中根节点信息
hipGraphHostNodeGetParams	获取 graph 中 host node 的参数
hipGraphHostNodeSetParams	为 graph 中指定的 host node 设置参数
hipGraphInstantiate	实例化
hipGraphKernelNodeSetParams	为 graph 中指定的 kernel node 设置参数
hipGraphLaunch	启动 graph
hipGraphMemcpyNodeGetParams	获取 graph 中 memcpy node 的参数
hipGraphMemcpyNodeSetParams	设置 graph 中 memcpy node 的参数
hipGraphMemsetNodeGetParams	获取 graph 中 memset node 的参数
hipGraphMemsetNodeSetParams	设置 graph 中 memset node 的参数
hipGraphAddChildGraphNode	创建子 graph 节点并添加到 graph 中
hipGraphChildGraphNodeGetGraph	获取子 graph 节点嵌入的 graph 句柄
hipGraphClone	克隆 graph
hipGraphDestroyNode	从 graph 中删除某节点
hipGraphExecChildGraphNodeSetParams	更新给定的 graphExec 中子 graph 节点参数
hipGraphExecEventRecordNodeSetEvent	为给定 graphExec 中事件记录节点设置事件
hipGraphExecHostNodeSetParams	设置给定 graphExec 中 host 节点的参数
hipGraphExecMemcpyNodeSetParams	设置给定 graphExec 中 memcpy 节点的参数
hipGraphExecMemcpyNodeSetParams1D	设置给定 graphExec 中一维 memcpy 节点的参数
hipGraphExecMemcpyNodeSetParamsFromSymbol	设置给定 graphExec 中要从设备上的符号复制的 memcpy 节点的参数
hipGraphExecMemcpyNodeSetParamsToSymbol	设置给定 graphExec 要复制到设备上的符号的 memcpy 节点的参数
hipGraphExecUpdate	插件是否可以使用 graph 更新可执行 graph，并在可能的情况下执行更新
hipGraphInstantiateWithFlags	通过 flag 控制将 graph 实例化为可执行 graph
hipGraphKernelNodeSetAttribute	设置 kernel 节点属性
hipGraphMemcpyNodeSetParams1D	设置一维 memcpy 节点参数
hipGraphMemcpyNodeSetParamsFromSymbol	设置要从设备上的符号复制的 memcpy 节点参数

hipGraphMemcpyNodeSetParamsToSymbol	设置要复制到设备上的符号的 memcpy 节点参数
hipGraphNodeFindInClone	查找节点的克隆版本
hipGraphNodeGetDependencies	返回节点的依赖项
hipGraphNodeGetDependentNodes	返回节点的依赖节点
hipGraphNodeGetType	获取节点的类型
hipGraphReleaseUserObject	从 graph 释放用户对象引用
hipGraphRemoveDependencies	从 graph 中删除依赖关系边缘
hipGraphRetainUserObject	保留对 graph 中用户对象的引用
hipUserObjectCreate	创建用户对象
hipUserObjectRelease	释放对用户对象的引用
hipUserObjectRetain	保留对用户对象的引用
hipStreamBeginCapture	开始对 stream 进行 graph 捕获
hipStreamEndCapture	结束对 stream 的捕获，返回捕获的 graph

表 4.3 DTK-25.04/25.04.1 新增的 hipGraph 接口

DTK-25.04/25.04.1	接口说明
hipGraphAddExternalSemaphoresSignalNode	创建外部信号机节点并将其添加到图中
hipGraphAddExternalSemaphoresWaitNode	创建外部信号机等待节点并将其添加到图中
hipGraphExecExternalSemaphoresSignalNodeSetParams	设置给定 graphExec 中外部信号量信号节点中的节点参数
hipGraphExecExternalSemaphoresWaitNodeSetParams	设置给定 graphExec 中外部信号量等待节点中的节点参数
hipGraphExternalSemaphoresSignalNodeGetParams	返回外部信号量信号节点参数
hipGraphExternalSemaphoresSignalNodeSetParams	设置外部信号量信号节点中的节点参数
hipGraphExternalSemaphoresWaitNodeGetParams	返回外部信号量等待节点参数
hipGraphExternalSemaphoresWaitNodeSetParams	更新外部信号量等待节点中的节点参数
hipGraphAddNode	创建内核执行节点并将其添加到 graph 中
hipGraphExecGetFlags	返回可执行图上的标志
hipGraphExecNodeSetParams	设置可执行图上已创建节点的参数

hipGraphInstantiateWithParams	从图创建可执行图
hipGraphNodeSetParams	更新已创建节点的参数
hipGraphConditionalHandleCreate	创建条件句柄
hipStreamBeginCaptureToGraph	开始在 stream 上捕获现有 graph

表 4.4 DTK-23.04.1/23.10 支持的 hipGraph stream capture 接口

DTK-23.04.1/23.10	接口说明
hipMemcpy3DAsync	异步 3 维拷贝
hipMemcpy2DAsync	异步 2 维拷贝
hipMemcpy2DFromArrayAsync	在主机和设备之间复制数据
hipMemcpyFromArrayAsync	从设备的给定符号复制数据
hipMemcpy2DToArrayAsync	在主机和设备之间复制数据
hipMemcpyToArrayAsync	在主机和设备之间复制数据
hipMemcpy	在主机和设备之间复制数据
hipMemcpyAsync	在主机和设备之间异步复制数据
hipMemcpyFromSymbolAsync	从设备上的给定符号复制数据异步
hipMemcpyToSymbolAsync	将数据复制到设备上的给定符号异步
hipMemsetAsync	初始化设备内存或者将设备内存设置为一个值异步操作
hipMemset2DAsync	初始化设备内存或者将设备内存设置为一个值 2 维异步
hipMemset3DAsync	初始化设备内存或者将设备内存设置为一个值 3 维异步
hipLaunchHostFunc	启动 host 端 function

表 4.5 DTK-25.04/25.04.1 新增的 hipGraph stream capture 接口

DTK-25.04/25.04.1	接口说明
hipEventRecord	记录事件
hipStreamWaitEvent	Stream 流等待事件

4.4.3 示例

- 重复计算场景

使用场景：适用于重复执行完整程序，即多次计算获取结果

编程流程概括：

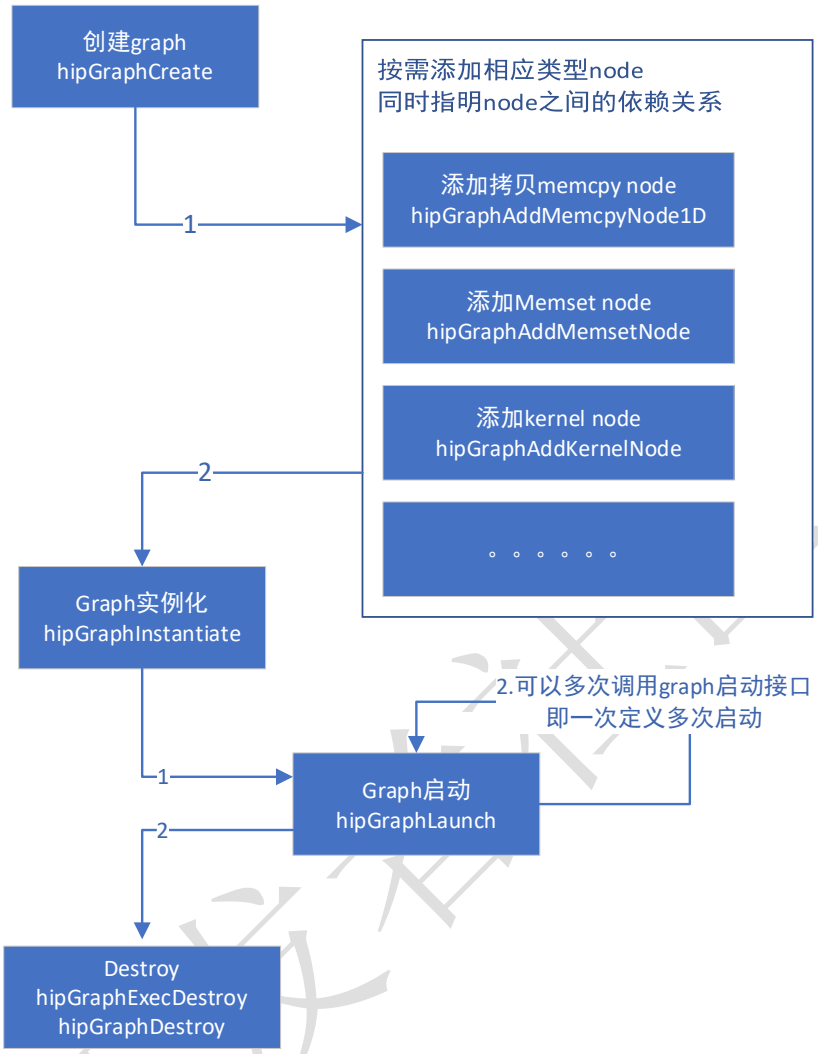


图 4.10 hipGraph 针对重复启动编程流程

参考编程如下：

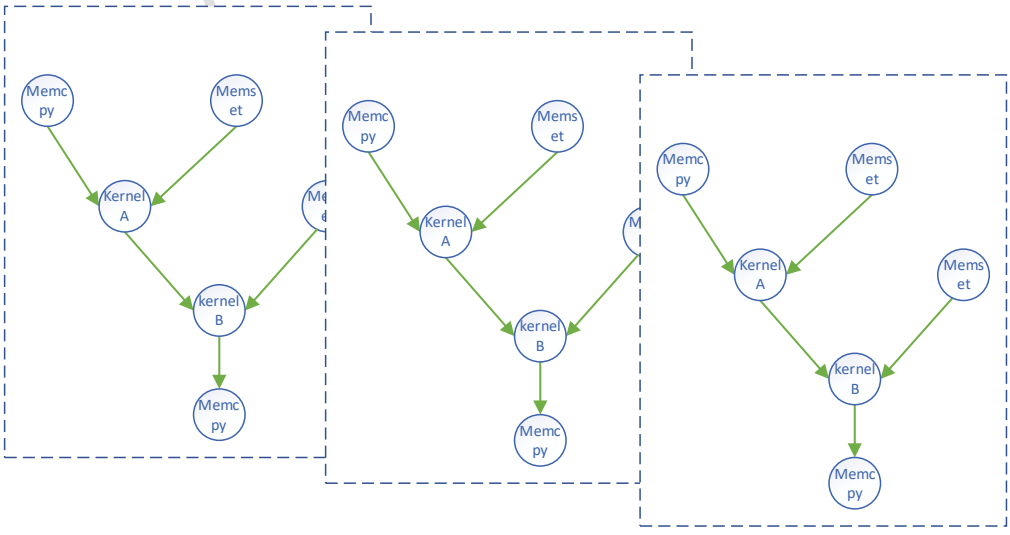


图 4.11 重复计算场景下 graph 结构

```
#define HIP_CHECK(stat)                                    \
   ({                                                  \
      hipError_t restat = stat;                        \
      if(restat != hipSuccess)                         \
      {                                                \
         printf("Error: hip error in line %d,stat:%d\n",__LINE__,restat) ;      \
      }                                                \
   })

hipGraph_t graph;
hipStream_t streamForGraph;
std::vector<hipGraphNode_t> nodeDependencies;
hipGraphNode_t memcpyNode, kernelNode, memsetNode;
double *result_h = NULL;
//申请资源
HIP_CHECK(hipHostMalloc(&inputVec_h,          sizeof(float)          *          size,
hipHostMallocDefault));
HIP_CHECK(hipMalloc(&inputVec_d, sizeof(float) * size));
HIP_CHECK(hipMalloc(&outputVec_d, sizeof(double) *maxBlocks));
HIP_CHECK(hipMalloc(&result_d, sizeof(double)));
HIP_CHECK(hipHostMalloc(&result_h, sizeof(double), hipHostMallocDefault));
//初始化主机端
for (int i = 0; i < size; i++) {
      inputVec_h[i] = 1.0f;
      inputVec_2_h[i] = 2.0f;
      result_h[i] = 0.0f;
}
//创建 stream
```

```
HIP_CHECK(hipStreamCreate(&streamForGraph));
//创建 hipGraph
HIP_CHECK(hipGraphCreate(&graph, 0));
//拷贝 H2D
HIP_CHECK(hipGraphAddMemcpyNode1D(&memcpyNode, graph,
                                  NULL, 0,
                                  inputVec_d, inputVec_h,
sizeof(float)*inputSize,
                                  hipMemcpyHostToDevice));
//Memset 接口
hipMemsetParams memsetParams = {0};
memsetParams.dst = (void*)outputVec_d;
memsetParams.value = 0;
memsetParams.pitch = 0;
memsetParams.elementSize = sizeof(float);
memsetParams.width = numOfBlocks * 2;
memsetParams.height = 1;
HIP_CHECK(hipGraphAddMemsetNode(&memsetNode, graph, NULL, 0,
&memsetParams));
//添加依赖
nodeDependencies.push_back(memcpyNode);
nodeDependencies.push_back(memsetNode);
//核函数添加，有两个依赖项
hipKernelNodeParams kernelNodeParams = {0};
void* kernelArgs[4] = {(void*)&inputVec_d, (void*)&outputVec_d, &inputSize,
&numOfBlocks};
kernelNodeParams.func = (void*)reduce;
kernelNodeParams.gridDim = dim3(inputSize / THREADS_PER_BLOCK, 1, 1);
kernelNodeParams.blockDim = dim3(THREADS_PER_BLOCK, 1, 1);
```

```
kernelNodeParams.sharedMemBytes = 0;
kernelNodeParams.kernelParams = (void**)kernelArgs;
kernelNodeParams.extra = NULL;
HIP_CHECK(hipGraphAddKernelNode(&kernelNode, graph,
                                nodeDependencies.data(),
nodeDependencies.size(),
                                &kernelNodeParams));
//添加依赖
nodeDependencies.clear();
nodeDependencies.push_back(kernelNode);
//Memset 接口
memset(&memsetParams, 0, sizeof(memsetParams));
memsetParams.dst = result_d;
memsetParams.value = 1;
memsetParams.elementSize = sizeof(double);
memsetParams.width = 1;
memsetParams.height = 1;
HIP_CHECK(hipGraphAddMemsetNode(&memsetNode, graph, NULL, 0,
&memsetParams));
nodeDependencies.push_back(memsetNode);
//核函数添加，两个依赖项
memset(&kernelNodeParams, 0, sizeof(kernelNodeParams));
kernelNodeParams.func = (void*)reduceFinal;
kernelNodeParams.gridDim = dim3(1, 1, 1);
kernelNodeParams.blockDim = dim3(THREADS_PER_BLOCK, 1, 1);
kernelNodeParams.extra = NULL;
kernelNodeParams.sharedMemBytes = 0;
void* kernelArgs2[3] = {(void*)&outputVec_d, (void*)&result_d, &numOfBlocks};
kernelNodeParams.kernelParams = kernelArgs2;
```

```
HIP_CHECK(hipGraphAddKernelNode(&kernelNode, graph,
                                nodeDependencies.data(),
nodeDependencies.size(),
                                &kernelNodeParams));
//拷贝，D2H
nodeDependencies.clear();
nodeDependencies.push_back(kernelNode);
HIP_CHECK(hipGraphAddMemcpyNode1D(&memcpyNode, graph,
                                  nodeDependencies.data(),
nodeDependencies.size(),
                                  result_h, result_d, sizeof(double),
                                  hipMemcpyDeviceToHost));
//获取 node 信息
hipGraphNode_t hostNode;
hipGraphNode_t nodes[5] = {};
size_t numNodes = 5;
HIP_CHECK(hipGraphGetNodes(graph, nodes, &numNodes));
HIP_CHECK(hipGraphGetRootNodes(graph, nodes, &numNodes));
//hipGraph 实例化接口
hipGraphExec_t graphExec;
HIP_CHECK(hipGraphInstantiate(&graphExec, graph, NULL, NULL, 0));
//启动 hipGraph，可多次调用 [GRAPH_LAUNCH_ITERATIONS 启动次数]
for(int i = 0; i < GRAPH_LAUNCH_ITERATIONS; i++){
    HIP_CHECK(hipGraphLaunch(graphExec, streamForGraph));
}
//stream 同步
HIP_CHECK(hipStreamSynchronize(streamForGraph));
//destroy (graph & graphExec & streamForGraph)
HIP_CHECK(hipGraphExecDestroy(graphExec));
```

```
HIP_CHECK(hipGraphDestroy(graph));
HIP_CHECK(hipStreamDestroy(streamForGraph));
//释放资源
HIP_CHECK(hipHostFree(inputVec_h));
HIP_CHECK(hipFree(inputVec_d));
HIP_CHECK(hipFree(outputVec_d));
HIP_CHECK(hipFree(result_d));
```

- 大量轻型 kernel 场景

使用场景：对大量轻型 kernel 处理

编程流程概括：

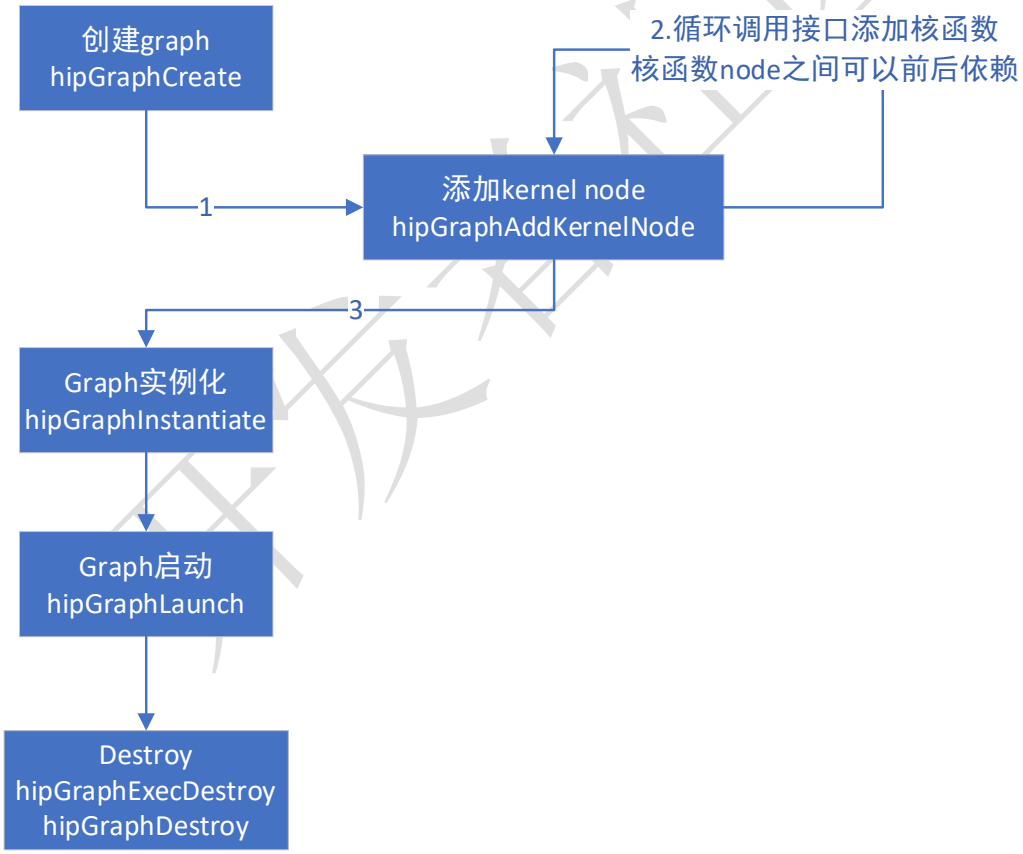


图 4.12 hipGraph 对大量轻型 kernel 函数编程流程

参考编程如下：

```
#define failed(...)                                    \
    printf("%serror: ", KRED);                         \
```

```
    printf(__VA_ARGS__);                                    \
    printf("\n");                                           \
    printf("error: TEST FAILED\n%s", KNRM);                 \
    abort();
#define HIPCHECK(error)                                     \
    {                                                       \
        hipError_t localError = error;
        if ((localError != hipSuccess) && (localError != hipErrorPeerAccessAlreadyEnabled)) {     \
            printf("%serror: '%s'(%d) from %s at %s:%d%s\n", KRED, hipGetErrorString(localError),  \
                    localError, #error, __FILE__, __LINE__, KNRM);                  \
            failed("API returned error code.");                                     \
        }                                                   \
    }
__global__ void add(float *d_x, float *d_y, float *d_z, size_t N) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        d_z[i] = d_x[i] + d_y[i];
    }
}
//申请资源，并对主机端进行初始化
HIPCHECK(hipHostMalloc(&inputVec_h, sizeof(float) * size, hipHostMallocDefault));
HIPCHECK(hipHostMalloc(&inputVec_2_h, sizeof(float) * size, hipHostMallocDefault));
HIPCHECK(hipHostMalloc(&result_h, sizeof(float) * size, hipHostMallocDefault));
for (int i = 0; i < size; i++) {
    inputVec_h[i] = 1.0f;
```

```
    inputVec_2_h[i] = 2.0f;
    result_h[i] = 0.0f;
}
HIPCHECK(hipMalloc(&inputVec_d, sizeof(float) * size));
HIPCHECK(hipMalloc(&inputVec_2_d, sizeof(float) * size));
HIPCHECK(hipMalloc(&outputVec_d, sizeof(float) * size));
//创建 stream
hipStream_t streamForGraph;
HIPCHECK(hipStreamCreate(&streamForGraph));
//拷贝接口，H2D
HIPCHECK(hipMemcpyAsync(inputVec_d, inputVec_h, sizeof(float) * inputSize,
hipMemcpyHostToDevice, streamForGraph));
HIPCHECK(hipMemcpyAsync(inputVec_2_d, inputVec_2_h, sizeof(float) * inputSize,
hipMemcpyHostToDevice, streamForGraph));
//创建 graph
hipGraph_t graph;
HIPCHECK(hipGraphCreate(&graph, 0));
//核函数添加
hipGraphNode_t kernelNode;
hipKernelNodeParams kernelNodeParams ;
void *kernelArgs[4] = {(void *)&inputVec_d, (void *)&inputVec_2_d, (void
*)&outputVec_d, &size};
memset(&kernelNodeParams, 0, sizeof(kernelNodeParams));
for (int i = 0; i < kernelNum; i++) {
    kernelNodeParams.func = (void *) add;
    kernelNodeParams.gridDim = dim3(gridSize, 1, 1);
    kernelNodeParams.blockDim = dim3(blockSize, 1, 1);
    kernelNodeParams.sharedMemBytes = 0;
    kernelNodeParams.kernelParams = kernelArgs;
```

```
    kernelNodeParams.extra = NULL;
    HIPCHECK(hipGraphAddKernelNode(&kernelNode, graph, NULL, 0,
&kernelNodeParams));
}
//hipGraph 实例化
hipGraphExec_t graphExec;
HIPCHECK(hipGraphInstantiate(&graphExec, graph, NULL, NULL, 0));
//hipGraph 启动
HIPCHECK(hipGraphLaunch(graphExec, streamForGraph));  //execute graph
//stream 同步
HIPCHECK(hipStreamSynchronize(streamForGraph));
//拷贝接口，D2H
HIPCHECK(hipMemcpyAsync(result_h, outputVec_d, sizeof(float) * inputSize,
hipMemcpyDeviceToHost, streamForGraph));
//destroy (graph & graphExec & streamForGraph)
HIPCHECK(hipGraphExecDestroy(graphExec));
HIPCHECK(hipGraphDestroy(graph));
HIPCHECK(hipStreamDestroy(streamForGraph));
//释放资源
HIPCHECK(hipFree(inputVec_d));
HIPCHECK(hipFree(inputVec_2_d));
HIPCHECK(hipFree(outputVec_d));
HIPCHECK(hipHostFree(inputVec_h));
HIPCHECK(hipHostFree(inputVec_2_h));
HIPCHECK(hipHostFree(result_h));
```

耗时对比：

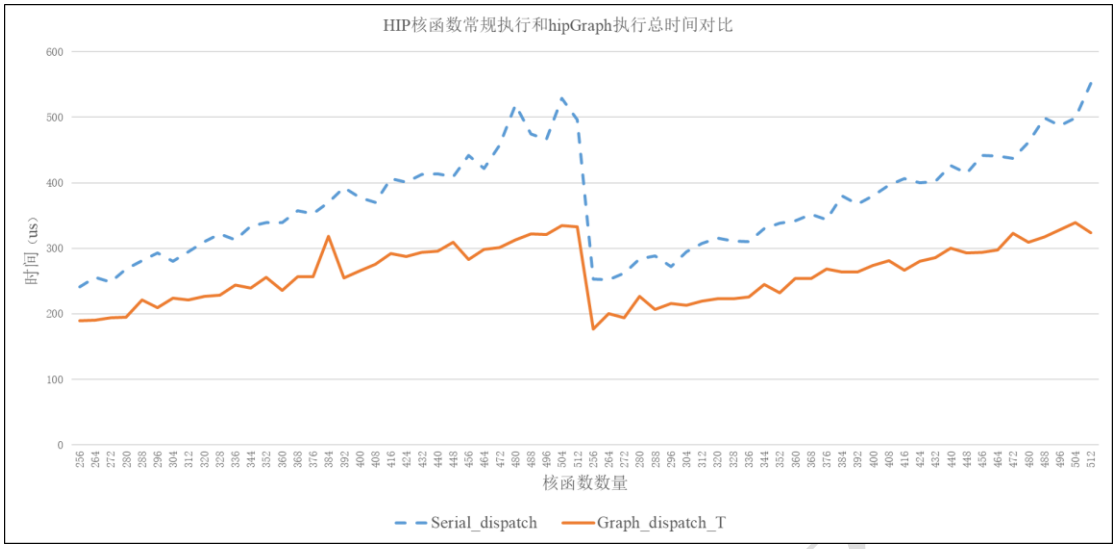


图 4.13 hipGraph 处理和常规处理耗时对比

注：添加的核函数数量取值在 256 至 512；循环执行两次。

通过对比两条曲线我们可以发现，相较于 HIP 原有核函数执行方式，随着核函数数量的增加，hipGraph 加速效果越明显，两者耗时不断加大。而且异构程序热启动对 hipGraph 也有效果。

开发者社区

5 HIP 设备管理

5.1 HIP 设备查询与选择

当我们开始使用 HIP 编写异构并行程序时，首先需要知道我们当前使用的机器是否拥有可以执行异构并行程序的设备，这些设备除了程序运行基础的 CPU、内存外，还有 HIP 所定义的 Device 设备，这些设备对应了物理上的 DCU 加速器。在物理上，这些 DCU 通过 PCI-E 总线与 CPU 进行互连通信。我们可以使用 HIP 定义的接口，帮我们方便地查询支持的设备和属性，也可以在各个设备之间自由切换使用。设备查询在一些工程设计比较规范的项目中比较常见，比较常见以下两种用途：

- 查询 DCU 数目，判断是否启用多 DCU 部分的代码
- 查询多个设备属性，选取某个计算能力最强的设备来运行代码

所以在编写异构并行程序之初，可以增加以下几个步骤来让你的代码更通用：

- 获取设备个数
- 依次查询设备属性
- 根据设备属性筛选使用的计算卡或者在代码中决定代码运行的分支

以上的过程通常被称为设备初始化，设备初始化对程序运行可能并不是必须的一个流程，如果我们针对某个特定的计算卡定制了一套程序，而且打算只让该程序运行于该型号的计算卡上，可以不进行设备初始化。当然这么做可能会出现一种后果，那便是某些不了解程序细节的人下载了我们的程序，并且使用了和我们不同的计算卡，在他运行该程序的时候可能会出现一些莫名的错误。这是因为 HIP 的使用是和硬件息息相关的，HIP 提供了一套 C 的编程接口，我们所编写的 HIP 异构并行程序通过编译器被编译为机器码交给计算卡运行。如果某个计算卡不支持编译器所编译出的二进制指令那么就会出现不可预料的错误。请看下面一段代码

```
...
__global__ void kernelAdd(float *a, float *b, float *c) {
  __shared__ float cache[threadsPerBlock];
```

```
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int cacheIndex = threadIdx.x;
    float temp = 0;
    while (tid < N) {
        temp += a[tid] + b[tid];
        tid += blockDim.x * gridDim.x;
    }
    cache[cacheIndex] = temp;
    //对线程块中的线程进行同步
    __syncthreads();
    int i = blockDim.x / 2;
    while (i != 0) {
        if (cacheIndex < i) {
            cache[cacheIndex] += cache[cacheIndex + i];
        }
        __syncthreads();
        i /= 2;
    }
    if (cacheIndex == 0) {
        c[blockIdx.x] = cache[0];
    }
}
...
```

上面的代码为了提高运行效率，使用了共享内存，但不是所有的 DCU 都支持共享内存，如果你想保留共享内存又想让程序具有通用性，使得其可以在不支持共享内存的计算卡上运行，可以使用分支策略，例如先通过设备属性查询，是否支持共享内存，如果支持走分支 1，不支持走分支 2，这样做既保留了对共享内存的支持，又可以保证在不支持共享的卡上正常运行。请看下面代码样例，首先进行设备筛选，选取合适的设备后，再进行设备属性判断，如果支持共享

内存，调用 kernel1 否则调用 kernel2。

```
#include "hip/hip_runtime.h"
...
int count;
int deviceId = 0;
hipGetDeviceCount(&count);
for (int i = 0; i < count; i++) {
   hipGetDeviceProperties(prop, i);
   if ... {
      deviceId = i;
      break;
   }
}
...
hipSetDevice(deviceId)
if (sharedMemPerBlock > 0) kernel1<<<grid, block >>> ();
else kernel2<<<grid, block >>> ();
...
```

在本节中介绍了设备初始化的重要性，根据设备初始化返回的结果灵活设计程序逻辑，可以让你的程序更加鲁棒。

表 5.1 常用设备属性解释

pciDeviceID	设备 ID
clockRate	核心频率
memoryClockRate	显存频率
totalGlobalMem	显存总大小
warpSize	线程束大小
sharedMemPerBlock	每个计算单元上共享内存的大小

上述设备属性查询的完整代码可以查阅 hip 自带的 samples，查询 samples 的路径使用如下命令(请确保您已经安装 hip 计算平台并正确配置了 hip 的路径)。

```
echo $(dirname $(dirname which hipcc))/hip/samples/1_Utils/hipInfo/hipInfo.cpp
```

上行命令中，which hipcc 用来寻找编译器的路径，将寻找到的编译器路径与 dirname 结合，可以找到程序的位置。其中，dirname 命令的作用是去除文件名中的非目录部分，仅显示与目录有关的内容，在这里是将存放 hipInfo.cpp 文件的绝对路径输出。

5.2 总结

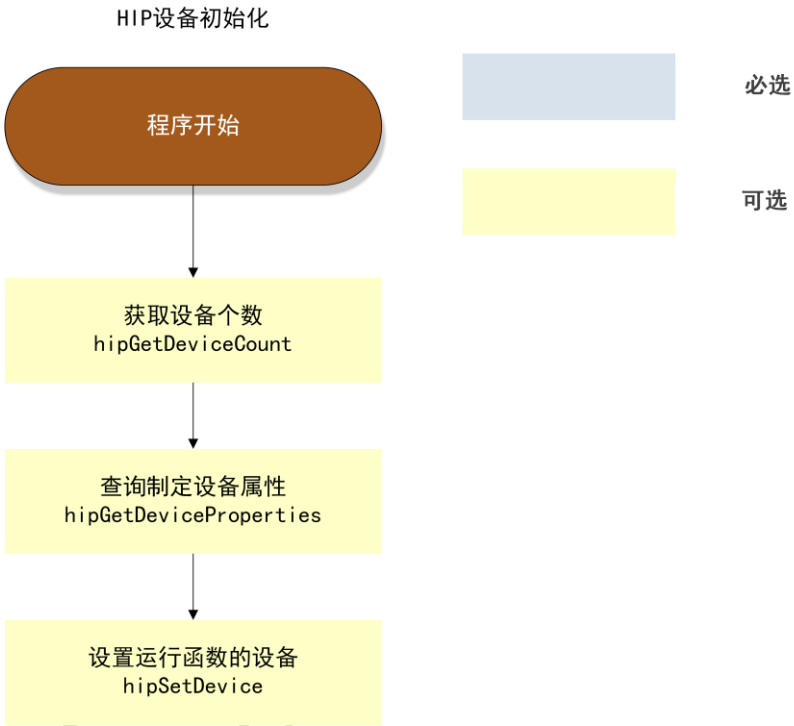


图 5.1 常用设备属性解释

6 单进程多 DCU 编程

在本节中，将会依次介绍多 DCU 编程的概念、如何在多个 DCU 间进行异步执行、学习 DCU 间的数据传输与通信，以及多 DCU 间的数据如何进行同步。这里需要说明的是，接下来讲解的多 DCU 编程的使用场景均是在单进程下进行。

6.1 多 DCU 操作

使用多 DCU 进行分布式计算的常见原因主要包括以下几个方面：

- 现有的数据和模型规模太大，单 DCU 内存大小无法容纳。
- 单 DCU 串行地处理任务时效率太低，为了提升应用程序的吞吐量和效率，可以使用多 DCU 并发地处理多任务。

当使用多 DCU 运行应用程序时，需要正确设计 DCU 间的通信。DCU 间数据传输的效率取决于 DCU 是如何连接在一个节点上并跨集群的。在多 DCU 系统里有两种连接方式：

- 多 DCU 通过单个节点连接到 PCIe 总线上
- 多 DCU 连接到集群中的网络交换机上

如图 6.1 展示了一个集群的简化拓扑结构，它包含两个计算节点（node0 和 node1），通过 InfiniBand 交换机相连。每个节点通过 PCIe 总线连接两个 DCU，如 DCU0 和 DCU1 通过 PCIe 总线连接到 node0 上，DCU2 和 DCU3 通过 PCIe 总线连接到 node1 上。

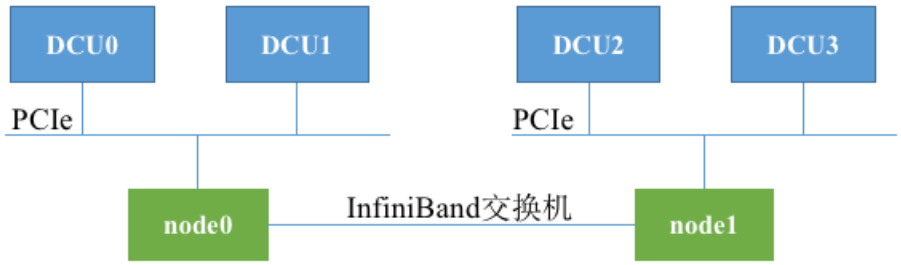


图 6.1 集群拓扑结构

为了实现在多 DCU 上执行应用程序，HIP 提供了一系列 API 支持，用于在多 DCU 系统中管理设备以及提供了执行内核的多种方式。

首先，在单一节点内，可以使用如下函数查看当前设备中能够使用的 DCU 设备数量：

```
hipError_t hipGetDeviceCount(int* count);
```

获取这些设备的性能可以使用下述函数：

```
hipError_t hipGetDeviceProperties(hipDeviceProp_t *prop, int deviceId);
```

下面的代码说明了如何查看当前设备中能够使用的 DCU 设备数量，并对其进行遍历，查询性能。

```
int nDCUs;
hipGetDeviceCount(&ngups);
for(int i = 0; i < nDCUs; i++){
        hipDeviceProp devProp;
        hipGetDeviceProperties(&devProp, i);
        printf("Device %d has compute capability %d.%d.\n", i, devProp.major, devProp.minor);
}
```

在使用多 DCU 之前，必须指定在哪个设备上进行操作。通过如下函数设置当前设备：

```
hipError_t hipSetDevice(int deviceId);
```

其中，指定变量 deviceId 的设备为当前设备。该函数为异步函数，因此是一个低开销的调用。使用此函数，可以随时从所有主机线程中指定设备。如果不指定设备，默认设备 id 为 0。否则，指定的设备 id 应为 0 到 ndcus-1。一旦选定了当前设备，所有的 HIP 运算将被应用到该设备中：

- 任何从主线程中分配来的设备内存将完全地常驻于该设备上
- 任何由主机线程创建的流或事件都会与该设备相关
- 任何由主机线程启动的内核都会在该设备上执行

例如，下面的代码说明了如何执行核函数和在单主机线程中进行内存拷贝：

```
for(int i = 0; i < ndcus; i++){
        //set the current device
        hipSetDevice(i);

        //execute kernel on current device
        hipLaunchKernel(kernelName, dimGrid, dimBlock,...);
```

```
        //asynchronously transfer data between the host and current device
        hipMemcpyAsync(...);
}
```

因为循环中的内核启动和数据传输是异步的，所以在每次调用操作后控制将很快返回到主机线程。即使内核或由当前线程发出的传输仍然在当前设备上执行时，也可以安全地转变设备，因为 hipSetDevice 函数不会导致主机同步。

6.2 多 DCU 异步执行

在本节中，将会讲解如何使用多 DCU 进行向量加法，学习如何通过多个设备分离输入和输出向量。

1.在多设备上分配内存

首先查看当前设备中可用的设备数量：

```
int ndcus;
hipGetDeviceCount(&ndcus);
printf("HIP-capable devices: %i\n", ndcus);
```

确定好可以使用的设备数量后，接下来需要声明主机内存、设备内存、流和事件。代码如下：

```
float *d_A[NDCUS], *d_B[NDCUS], *d_C[NDCUS];
float *h_A[NDCUS], *h_B[NDCUS], *hostRef[NDCUS], *dcuRef[NDCUS];
hipStream_t stream[NDCUS];
```

在向量加法的例子中，元素的总输入大小为 16M，平均分配到各个设备上，每个设备分到 iSize 个元素：

```
int size = 1 << 24;
int iSize = size / ndcus;
```

设备上一个浮点向量的字节大小按如下方式进行计算：

```
size_t iBytes = iSize * sizeof(float);
```

接下来分配主机和设备内存，为每个设备创建流：

```
for (int i = 0; i < ndcus; i++){
   // set current device
```

```
    hipSetDevice(i);
    // allocate device memory
    hipMalloc((void **) &d_A[i], iBytes);
    hipMalloc((void **) &d_B[i], iBytes);
    hipMalloc((void **) &d_C[i], iBytes);
    // allocate page locked host memory for asynchronous data transfer
    hipHostMalloc((void **) &h_A[i], iBytes);
    hipHostMalloc((void **) &h_B[i], iBytes);
    hipHostMalloc((void **) &hostRef[i], iBytes);
    hipHostMalloc((void **) &dcuRef[i], iBytes);
    // create streams for timing and synchronizing
    hipStreamCreate(&stream[i]);
}
```

需要注意的是，分配锁页（固定）主机内存是为了在设备和主机之间进行异步数据传输。同时，在分配任何内存或创建任何流之前，使用上面提到的 hipsetDevice 函数在每次循环迭代的开始，设置当前设备。

2.单主机线程分配工作

首先为每个设备初始化主机数组的状态：

```
for (int i = 0; i < ndcus; i++) {
    hipSetDevice(i);
    initialData(h_A[i], iSize);
    initialData(h_B[i], iSize);
}
```

接下来使用一个循环在多个设备之间分配数据和计算：

```
for (int i = 0; i < ndcus; i++) {
    hipSetDevice(i));
    hipMemcpyAsync(d_A[i], h_A[i], iBytes, hipMemcpyHostToDevice, stream[i]));
    hipMemcpyAsync(d_B[i], h_B[i], iBytes, hipMemcpyHostToDevice, stream[i]));
    hipLaunchKernelGGL(iKernel, dim3(grid), dim3(block), 0, stream[i], d_A[i],
```

```
d_B[i], d_C[i], iSize);
    hipMemcpyAsync(dcuRef[i], d_C[i], iBytes, hipMemcpyDeviceToHost, stream[i]));
}
```

上述代码对设备进行遍历，将输入数组从主机端异步传输到设备端。在相同的流中操作 iSize 个数据元素以便启动内核。最后将计算结果进行异步拷贝，从内核返回到主机。由于所有的函数都是异步的，所以控制会立即返回到主机线程。因此，当任务仍在当前设备上运行时，切换到下一个设备是安全的。

3.编译和执行

用以下命令进行编译：

```
hipcc -O3 simpleMultiDCU.cpp -o simpleMultiDCU
```

输出结果为：

```
> starting ./simpleMultiDCU HIP-capable devices: 4
> total array size 16 M, using 4 devices with each device handling 4 M
1 DCU timer elapsed: 151.89ms
```

将命令行选项设为 1，即只用一个设备上运行：

```
./simpleMultiDCU 1
```

输出结果如下：

```
> starting ./simpleMultiDCU HIP-capable devices: 4
> total array size 16 M, using 1 devices with each device handling 16 M
1 DCU timer elapsed: 373.08ms
```

对比输出结果可以看到，使用 4 个设备时的性能相对更好。

6.3 DCU 间数据传输与通信

在同一个 PCIe 根节点内连接了 DCU0 和 DCU1，如果 DCU0 的计算结果或者数据想传到 DCU1 中，两个 DCU 之间的通信是完全依赖 CPU，即 DCU0 首先将数据传送到 CPU，CPU 再把数据传送到 DCU1 中。此时可以看到数据传输带宽受限于 CPU 带宽，且由于拷贝了两次，数据延迟也较大。

为了解决以上问题，提出了 DCU 的点对点通信技术，即连接到同一个 PCIe 根节点上的 DCU 之间能够直接传送数据。该技术可以直接提高 DCU 的带宽利用率以及降低 DCU 之间的数据传输延迟。

- 点对点访问：在 HIP 内核和 DCU 间直接加载和存储地址
- 点对点传输：在 DCU 之间直接拷贝数据

在一个系统内，如果两个 DCU 连接到不同的 PCIe 根节点上，则会被点对点 HIP API 提醒不能直接地进行点对点访问。此时驱动器将通过主机内存透明地传输数据，而不是通过 PCIe 总线直接传输数据。

实现点对点访问

点对点访问使得多个DCU连接到同一个PCIe总线上时可以进行资源共享，每个 DCU 可以直接访问和使用存储在其他 DCU 设备内存上的数据。对于透明的内核，引用的数据将通过 PCIe 总线传输到请求的线程上。

并非所有的 DCU 都支持点对点访问，因此需要使用下述函数检查设备是否支持点对点访问：

```
hipError_t hipDeviceCanAccessPeer (int* canAccessPeer, int device, int peerDevice);
```

如果设备 device 能够直接访问到对等设备 peeDevice 的全局内存，那么变量 canAccessPeer 的返回值为 1，否则返回 0。

使用如下函数启用设备间的点对点内存访问：

```
hipError_t hipDeviceEnablePeerAccess (int peerDevice, unsigned int flag);
```

该函数授权的访问是单向的，即允许从当前设备到对等设备 peerDevice 进行点对点访问，但不允许从 peerDevice 到当前设备的访问。如果需要从对等设备访问当前设备的内存，则需要从对等设备方向进行调用。flag 参数将被保留以备将来使用，目前必须将其设置为 0。一旦成功，该对等设备的内存将立即由当前设备进行访问。

使用如下函数禁用点对点访问：

```
hipError_t hipDeviceDisablePeerAccess (int peerDeviceId);
```

32 位应用程序不支持点对点访问。

- 点对点的内存复制

启用点对点内存访问后，可以使用如下函数使得两个设备间异步地进行数据拷贝。

```
hipError_t hipMemcpyPeerAsync(void* dst, int dstDev, void* src, int srcDev, size_t sizeBytes, hipStream_t stream)
```

该函数实现从源设备 srcDev 的设备内存传输到目的设备 dstDev 的设备内存中。函数 hipMemcpyPeerAsync 对于主机和所有其他设备来说是异步的。如果 srcDev 和 dstDev 共享相同的 PCIe 根节点，那么数据传输是沿着 PCIe 最短路径执行的，不需要通过主机内存中转。

- 样例

在本节中，将介绍两个 DCU 之间的数据传输。分别测试以下两种情况：两个 DCU 之间的单向内存复制，以及两个 DCU 之间的双向内存复制。

首先，对所有设备启用双向点对点访问，如以下代码所示：

```
inline void enableP2P(int ndcus) {
   for (int i = 0; i < ndcus; i++) {
      hipSetDevice(i);
      for (int j = 0; j < ndcus; j++) {
         if (i == j) continue;

         int peer_access_available = 0;
         hipDeviceCanAccessPeer(&peer_access_available, i, j);

         if (peer_access_available) {
       hipDeviceEnablePeerAccess(j, 0);
       printf("> DCU%d enabled direct access to DCU%d\n", i, j);
         } else {
            printf("%d, %d", i, j);
         }
      }
   }
}
```

函数 enableP2P 遍历所有设备对(i，j)，如果支持点对点访问，则使用 hipDeviceEnablePeerAccess 函数启用双向点对点访问。

启用点对点访问后，可以在两个设备之间直接复制数据。如果不支持点对

点访问，该例子输出不能启用点对点访问的设备 ID（不能启用的最有可能的原因是因为它们没有连接到同一个 PCIe 根节点上），并且没有错误继续运行了。然而，上文提到，如果在两个 DCU 之间不支持点对点访问，那么这两个设备之间的点对点内存复制将通过主机内存中转，从而会降低其性能。性能降低对应用程序的影响程度取决于内核进行时间计算和执行对等传输需要的时间。如果有足够的时间来进行计算，那么可以隐藏点对点复制的延时，该延时主要是通过主机内存使用设备计算进行重叠的。

启用点对点访问后，下面的代码在两个设备间执行 ping-pong 同步内存复制，次数为一百次。如果点对点访问在所有设备上都被成功启用了，那么将直接通过 PCIe 总线进行数据传输而不用与主机交互。

```
hipEventRecord(start, 0);
for (int i = 0; i < 100; i++) {
   if (i % 2 == 0) {
      hipMemcpy(d_src[1], d_src[0], iBytes, hipMemcpyDeviceToDevice);
   } else {
      hipMemcpy(d_src[0], d_src[1], iBytes, hipMemcpyDeviceToDevice));
   }
}
```

因为跨设备的内存复制不需要显式地设置当前设备，所以在内存复制之前没有进行设备转换。如果在内存复制前指定了设备，也不会影响它的行为。

如需衡量设备之间数据传输的性能，需要把启动和停止事件记录在同一设备上，并将 ping-pong 内存复制包含在内。然后，用 hipEventElapsedTime 计算两个事件之间消耗的时间。

```
hipSetDevice(0);
hipEventRecord(start, 0);
for (int i = 0; i < 100; i++) {
   ......
}
hipSetDevice(0);
```

```
hipEventRecord(stop, 0);
hipEventSynchronize(stop);
float elapsed_time_ms;
hipEventElapsedTime(&elapsed_time_ms, start, stop);
```

然后，通过 ping-pong 测试所获得的带宽按照下面所示的代码进行估计：

```
elapsed_time_ms /= 100.0f;
printf("Ping-pong unidirectional hipMemcpy: %8.2f ms ", elapsed_time_ms);
printf("performance: %8.2f GB/s\n", (float)iBytes / (elapsed_time_ms * 1e6f));
```

编译和运行样例 simpleP2P_PingPong.cpp，如下所示：

```
hipcc -O3 simpleP2P_PingPong.cpp -o simpleP2P_PingPong
./simpleP2P_PingPong
```

simpleP2P_PingPong 的输出如下所示：

```
Allocating buffers (64MB on each DCU and CPU Host)...
Ping-pong unidirectional hipMemcpy: 4.98 ms performance: 13.49 GB/s
```

因为 PCIe 总线支持任何两个端点之间的全双工通信，所以也可以使用异步拷贝 API 来进行双向的且点对点的内存复制：

```
hipEventRecord(start, 0));
for (int i = 0; i < 100; i++) {
  hipMemcpyAsync(d_src[1], d_src[0], iBytes, hipMemcpyDeviceToDevice,
stream[0]));
  hipMemcpyAsync(d_rcv[0], d_rcv[1], iBytes, hipMemcpyDeviceToDevice,
stream[1]));
}
```

注意，因为 PCIe 总线是一次在两个方向上使用的，所以获得的带宽增加了一倍。在 simpleP2P_PingPong.cpp 中移除对 enableP2P 的调用可以禁用点对点访问，但由于通过主机内存中转传输，所以测得的带宽将会下降。

统一虚拟寻址（UVA）是将 CPU 系统内存和设备的全局内存映射到一个单一的虚拟地址空间中，如图 6.2 所示。所有由 hipHostAlloc 分配的主机内存和由 hipMalloc 分配的设备内存驻留在这个统一的地址空间内。内存地址所驻留的设

备可以根据地址本身确定。

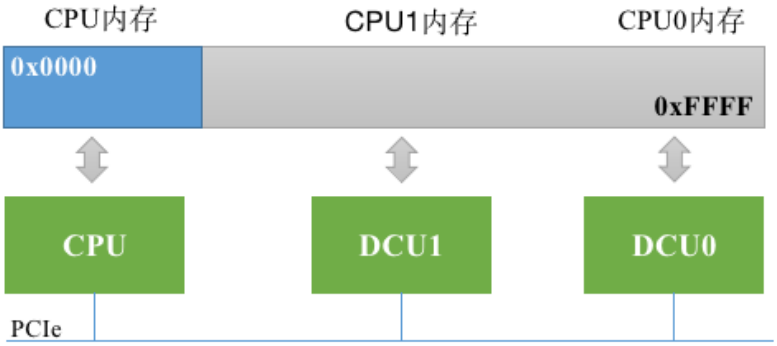


图 6.2 UVA 的 P2P 内存访问

使用统一虚拟寻址的点对点，对任何设备的内存的访问都是透明的。不需要再手动管理单独的内存缓冲区，也不用从主机内存中进行显式的复制。底层系统能使我们避免显式地执行这些操作，从而简化了代码。不过，过多的使用 UVA 进行点对点访问也会带来一些问题，如跨 PCIe 总线的许多小规模的传输会明显地有过大的消耗。

使用下述代码检查设备是否支持统一寻址：

```
int deviceId = 0;
hipDeviceProp prop;
hipGetDeviceProperties(&devProp, deviceId);
printf("DCU%d: %s unified addressing\n", deviceId, prop.unifiedAddressing ? "supports" : "does not support");
```

使用 UVA 时，应用程序必须在 64 位操作系统上进行编译。当 UVA 与点对点访问相结合时，在一个设备上执行的核函数，可以解除另一个设备上存储的指针。

下述代码实现了将输入数组扩展了 2 倍，并将结果存储在输出数组中。可以用来测试 DCU 的直接点对点内存访问：

```
__global__ void iKernel(float *src, float *dst) {
    const int idx = blockIdx.x * blockDim.x + threadIdx.x;
    dst[idx] = src[idx] * 2.0f;
}
```

以下代码将设备 0 设置为当前设备，并有一个核函数使用指针 d_src[1]从设

备 1 中读取全局内存，同时通过全局内存的指针 d_rcv[0]将结果写入当前设备中。

```
hipSetDevice(0);
hipLaunchKernel(iKernel, dimGrid, dimBlock, ..., d_rcv[0], d_src[1]);
```

以下代码将设备 1 设置为当前设备，并有一个核函数使用指针 d_src[0]从设备 0 中读取全局内存，同时通过全局内存的指针 d_rcv[1]将结果写入当前设备中。

```
hipSetDevice(1);
hipLaunchKernel(iKernel, dimGrid, dimBlock, ..., d_rcv[1], d_src[0]);
```

以下输出表明这些核函数运行成功：

```
2. Running kernel on DCU1, taking source data from DCU0 and writing to DCU1...
3. Running kernel on DCU0, taking source data from DCU1 and writing to DCU0...
```

如果 DCU 没有连接到相同的 PCIe 根节点上，或点对点访问被禁止，那么将会出现以下的错误信息：

```
> DCU0 disabled direct access to DCU1
> DCU0 disabled direct access to DCU2
> DCU0 disabled direct access to DCU3
```

6.4 多 DCU 同步方法

用于流和事件的 HIP API，也适用于多 DCU 应用程序。我们需要为每一个流和事件指定一个运行设备。在单 DCU 应用程序上使用的同步函数同样适用于多个设备情况，但是必须指定适合的当前设备。在多个设备上使用流和事件的典型工作流程如下所示：

首先，查看当前可用设备，使用 hipSetDevice(id)指定程序将在哪个设备上运行，并为每个设备创建流和事件。

```
hipSetDevice(0);
int nDCUs;
hipStream_t stream;
hipEvent_t start, stop;
hipGetDeviceCount(&numDevices);
hipStreamCreate(&stream);
hipEventCreate(&start);
```

```
hipEventCreate(&stop);
```

接着需要给每个设备分配设备资源，如设备内存。如下例所示，使用hipMalloc()函数为设备 A 和 B 分配内存。

```
#define N 1024
size_t Nbytes = N * sizeof(int);
int *A_d, *B_d;
hipMalloc(&A_d, Nbytes);
hipMalloc(&B_d, Nbytes);
```

通过流在每个 DCU 上启动任务，如数据传输或内核执行。 下述代码描述了从 0 号设备 A_d 的设备内存通过流传输 Nbytes 数据到 1 号设备的设备内存。

```
.....
hipSetDevice(1);
hipMalloc(&X_d, Nbytes);
hipMemcpyPeerAsync(X_d,1,A_d,0,Nbytes,stream);
```

使用流和事件来查询和等待任务完成。

```
float eventMs = 1.0f;
hipEventRecord(start, stream);
.....
hipEventRecord(stop, stream);
hipEventSynchronize(stop);
hipEventElapsedTime(&eventMs, start, stop);
```

最后，程序运行完成需要释放所有设备的资源。

```
hipStreamDestroy(stream);
hipHostFree(A_h);
hipHostFree(B_h);
.....
hipFree(A_d);
hipFree(B_d);
```

6.5 总结

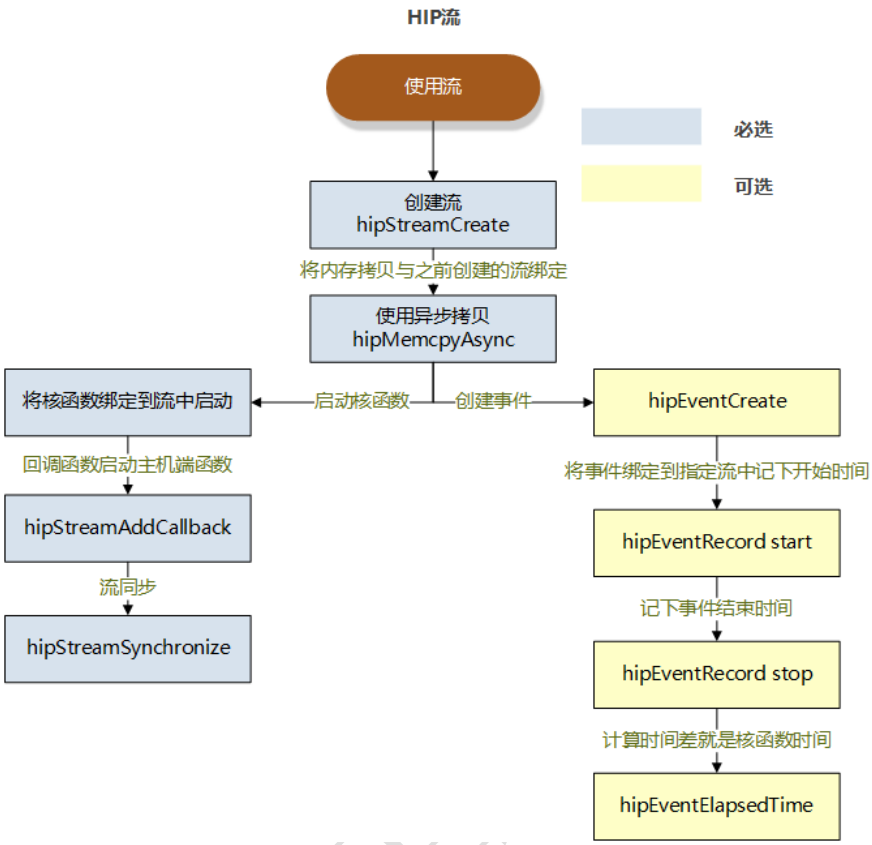


图 6.3 多 DCU 编程流程

7 性能分析

7.1 使用 hipprof 分析程序

hipprof 是 DCU Toolkit 的性能分析工具，主要功能是根据用户的指令选项，启动待进行性能分析的应用程序，抓取性能分析数据，最终对性能分析数据进行存储、导出和展示。hipprof 的基本用法是 hipprof 加指令选项加程序启动命令，在正确加载 DTK 环境后，可以使用 hipprof -h 查看 hipprof 的帮助信息。

```
Usage:
  hipprof [options] <app command line>
Options:
  -h                    - this help
  -o <output file>          - output CSV file [<result_pid>.csv]
  -d <data directory>       - to change the data directory [.]
tracing options:
  --hiptx-trace             - to enable hipTX application code annotation trace, "Markers and Ranges" JSON trace section.
  --hip-trace               - to trace HIP, generates API execution stats and JSON file chrome-tracing compatible.
  --mpi-trace               - to trace mpi application.
  --db <db file name>       - re export json file by exist db file.
  --db-merge <db file path>   - merge mutipe db files and export new combined json.
  --index-range <start:end>   - to export partial data by index range, start and end are separated by ':'.
  --group-stream            - to export json time line group by stream.
```

参数	主要参数释义
-d	性能分析工具产生的数据文件的存储目录，默认为当前目录
-o	输出文件名称，输出文件为 CSV 文件，默认名称为.csv
--hiptx-	开启 hiptx 性能分析，结合 roctx 接口，做应用性能分析

trace	
--hip-trace	hip trace 性能分析类型，hipprof 默认分析 hip trace，这个指令可以不加
--mpi-trace	开启 mpi 程序的 trace 性能分析，支持 mpi 多节点多进程
--db	命令后加 db 文件，可以重新生成 json 可视化文件
--db-merge	命令后加目录路径，可以合并目录下的 db 文件，并生成合并后的数据库文件和前端脚本
--index-range	后面加 start 和 end 的 hip api 顺序号，用冒号分开，可以做某一范围性能分析
--group-stream	默认情况下生成的前端脚本 memcpy 和 kernel 是分开显示的，加上此命令，可以使 copy 和 kernel 统一按照 stream 分组显示

使用一个代码文件名为“simpleStream.cpp”示例对 hip trace 操作流程进行讲解，演示程序的关键部分代码如下：

```
__global__ void sumArrays(float *A, float *B, float *C, const int N) {
  int idx = blockIdx.x * blockDim.x + threadIdx.x;
  if (idx < N) {
    for(int j=0; j<60; j++) {
        C[idx] = A[idx] + B[idx];
    }
  }
}
...
  //warmup
  hipMemcpy(d_A, h_A, nBytes, hipMemcpyHostToDevice);
  hipMemcpy(d_B, h_B, nBytes, hipMemcpyHostToDevice);
  hipLaunchKernelGGL(sumArrays, dim3(grid), dim3(block), 0, 0, d_A, d_B, d_C,
nElem);
```

```
    hipMemcpy(h_C, d_C, nBytes, hipMemcpyDeviceToHost);
...
    hipStream_t stream[NSTREAM];
    for (int i = 0; i < NSTREAM; ++i) {
        hipStreamCreate(&stream[i]);
    }
    // initiate all asynchronous transfers to the device
    for (int i = 0; i < NSTREAM; ++i) {
        int ioffset = i * iElem;
        hipMemcpyAsync(&d_A[ioffset], &h_A[ioffset], iBytes,
hipMemcpyHostToDevice, stream[i]);
        hipMemcpyAsync(&d_B[ioffset], &h_B[ioffset], iBytes,
hipMemcpyHostToDevice, stream[i]);
    }
    // launch a kernel in each stream
    for (int i = 0; i < NSTREAM; ++i) {
        int ioffset = i * iElem;
        hipLaunchKernelGGL(sumArrays, dim3(grid), dim3(block), 0, stream[i],
&d_A[ioffset], &d_B[ioffset], &d_C[ioffset], iElem);
    }
    // enqueue asynchronous transfers from the device
    for (int i = 0; i < NSTREAM; ++i) {
        int ioffset = i * iElem;
        hipMemcpyAsync(&h_C[ioffset], &d_C[ioffset], iBytes,
hipMemcpyDeviceToHost, stream[i]);
    }
```

1.编译程序

使用 hipcc 编译可执行程序 simpleStream，编译完成后可以使用./simpleStream 验证程序是否可以正常运行。

```
$> hipcc simpleStream.cpp -o simpleStream
```

2.运行程序

使用 hipprof 运行程序

```
$>  hipprof --hip-trace ./simpleStream
HIP_PROF:on Sat Jan 29 09:46:51 2022
HIP_PROF:path '/public/home/lizhigong/hiphsa/build/bin/'
HIP_PROF:process id '2079'
HIP_PROF:db file './hip-prof-2079.db'
HIP_PROF:out put file 'result_2079'
HIP_PROF:profiling './simpleStream'
> ./simpleStream Starting...
> Using Device 0: Device 66a1
> vector size = 67108864
> size nBytes = 268435456
> grid (131072,1) block (512,1)
Measured timings (throughput):
 Memcpy host to device  : 37.980602 ms (14.135398 GB/s)
 Memcpy device to host  : 18.986862 ms (14.137958 GB/s)
 Kernel time: 40.116814 ms
 Total time: 97.084274 ms
> iElem = 33554432
> iBytes = 134217728
> grid (65536,1) block (512,1)
Actual results from overlapped data transfers:
overlap with 2 streams : 83.272194 ms
semaphore_p end
HIP_PROF:process end
------------------------------------------------------------------------------------------------------------------------------
------------------------------
```

Name	Calls	TotalDurationNs	AverageNs	Percentage
hipMemcpy	6	158241955	26373659	36.7554288234653
hipHostMalloc	3	124157069	41385689	28.8384095896665
hipEventSynchronize	4	123400399	30850099	28.6626551234894
hipHostFree	3	19747349	6582449	4.58678787570357
hipMalloc	3	2117511	705837	0.491841906550035
hipDeviceReset	1	867251	867251	0.201439513323626
hipStreamCreate	2	711281	355640	0.165211799670847
hipFree	3	610970	203656	0.141912202413529
hipStreamDestroy	2	341010	170505	0.0792076209061618
hipLaunchKernel	4	126510	31627	0.0293849333475222
hipMemcpyAsync	6	60560	10093	0.0140664893172551
__hipPushCallConfiguration	4	50580	12645	0.0117483987725687
hipEventRecord	8	32780	4097	0.0076139286628075
hipEventElapsedTime	4	26860	6715	0.00623886894090938
hipEventCreate	2	22900	11450	0.00531906547828834
__hipPopCallConfiguration	4	5180	1295	0.00120317725666086
hipEventDestroy	2	3000	1500	0.000696820805015939
hipGetDeviceProperties	1	1870	1870	0.000434351635126602

```
hipSetDevice          1              1720             1720          0.000399510594875805
--------------------------------------------------------------------------------------------------------------
------------------------------
HIP PROF:dump json (79/79)
HIP_PROF:finish
```

除程序原有的输出外，我们还可以看到 hipprof 已经为程序输出了统计信息，按照总消耗时间从大到小的顺序，依次展示 HIP Runtime API 的调用名称、次数、总消耗时间和平均消耗时间。运行完成后当前目录生成默认结果文件，包括：

- hip-prof-25624.db
- resault_25624.csv
- resault_25624_copy.json

如果使用指令 hipprof --hip-trace --group-stream ./simpleStream 会生成 **stream.json 文件：

resault_25624_stream.json

其中 25624 是本次执行 hipprof 的进程号，每次执行 hipprof 的时进程号不一定相同，db 结尾的文件是 sqlite3 数据库文件，里面存放了全部的性能分析数据，csv 文件可以用 excel 打开，是 hip api 的统计数据，包括调用次数和耗时统计情况，json 文件是可以下载下来进行可视化的文件。

3.下载 json 文件可视化

正确生成 json 文件后，通过下载工具将 json 文件下载到本地，打开 chrome 浏览器，地址栏输入: chrome://tracing，点击 load 按钮，选择下载好的 json 文件，就可以看到可视化的性能数据，用键盘的 A、D、W、S 按键可以左右移动以及放大缩小显示区域，可视化后页面的最左侧是目录结构，中间是对应的性能时间线。可视化结果如图 7.1 所示：

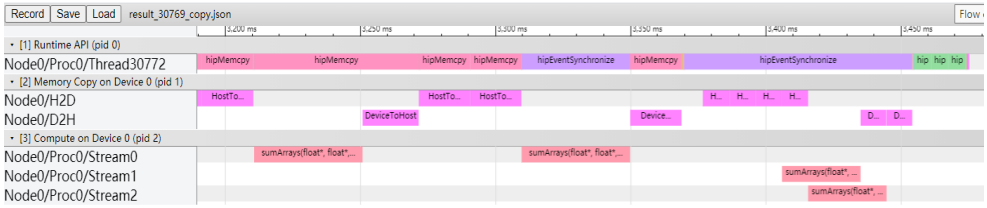


图 7.1 性能分析工具

Runtime API：显示 hip API 的性能情况，时间线反映了 API 的调用顺序以及占用线程时长。

Memory Copy on Device 0：指的是在 device Id 为 0 的设备上发生 memcpy 的情况，下边的 H2D 和 D2H 是 memcpy 的类型方向，比如 hipDeviceToHost、hipHostToDevice。Compute on Device 0 指的是在 device Id 是 0 的设备上发生的核函数的运行情况，下边的数字分别代表不同的 stream，0 代表的是默认 stream。

由于核函数或者 memcpy 和 hip 应用之间存在异步执行的情况，在点击选择核函数后，页面下部会显示一些详细信息及事件信息，点击 Preceding events 后边的链接，可以显示出对应 hip 接口的时间情况。增加--group-stream 选项的可视化结果如图 7.2 所示：

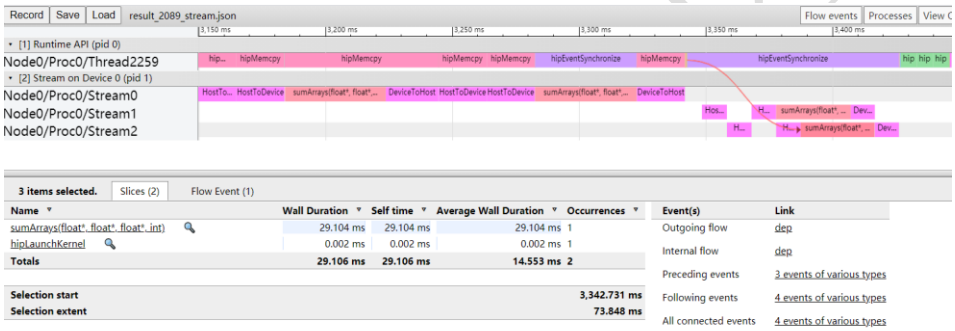


图 7.2 异步性能分析

7.2 hiptx-trace 功能

hiptx-trace 功能是 profile 将性能分析能力通过 API 提供给用户，用户可以通过 roctx.h 里面提供的接口实现对用户应用程序自身标注的性能监测点进行可视化分析。

hiptx-trace 的使用指令是 hipprof --hiptx-trace + 用户应用的运行指令。hiptx-trace 提供的接口介绍：

```
void roctxMark(const char * message);
```

创建一个标记消息，标记消息在显示 timeline 的时候时长为 1ms。参数 message 是消息内容。

```
Int roctxRangePush(const char * message);
```

创建一个时间范围的标记消息，标记消息显示 timeline 的起点是调用 roctxRangePush 的时间点，终点是调用 roctxRangePop 的时间点。消息的管理方

式是使用栈的方式。参数 message 是消息内容，返回值是压栈前消息的数量。

```
int roctxRangePop();
```

记录标记消息的时间终点，返回是 pop 后的栈消息数量。

```
roctx_range_id_t roctxRangeStart(const char *message);
```

创建一个时间范围的标记消息，标记消息显示 timeline 的起点是调用 roctxRangeStart 的时间点，终点是调用 roctxRangeStop 的时间点。消息通过返回的 roctx_range_id 来对应。参数 message 是消息内容，返回值 roctx_range_id 是一个 ID 值，调用 roctxRangeStop 的时候传入，用来对应 start。

```
void roctxRangeStop(roctx_range_id_t id);
```

记录标记消息的时间终点，参数 roctx_range_id 是调用 roctxRangeStart 时返回的 ID 值，用来做对应。

hiptx 的操作步骤和流程与 hiptrace 相同，这里给出关键部分的测试代码和相映的 timeline 效果：

```
  roctxMark("before hipLaunchKernel");
  int rangeId = roctxRangeStart("hipLaunchKernel range");
  roctxRangePush("hipLaunchKernel");
  // Lauching kernel from host
  hipLaunchKernelGGL(
    matrixTranspose, dim3(WIDTH / THREADS_PER_BLOCK_X, WIDTH / THREADS_PER_BLOCK_Y),
    dim3(THREADS_PER_BLOCK_X, THREADS_PER_BLOCK_Y), 0, 0, gpuTransposeMatrix, gpuMatrix, WIDTH);
  roctxMark("after hipLaunchKernel");
   // Memory transfer from device to host
  roctxRangePush("hipMemcpy");
  hipMemcpy(TransposeMatrix, gpuTransposeMatrix, NUM * sizeof(float), hipMemcpyDeviceToHost);
  roctxRangePop();  // for "hipMemcpy"
  roctxRangePop();  // for "hipLaunchKernel"
```

```
roctxRangeStop(rangeId);
```

对应的 timeline 效果如图 7.3:

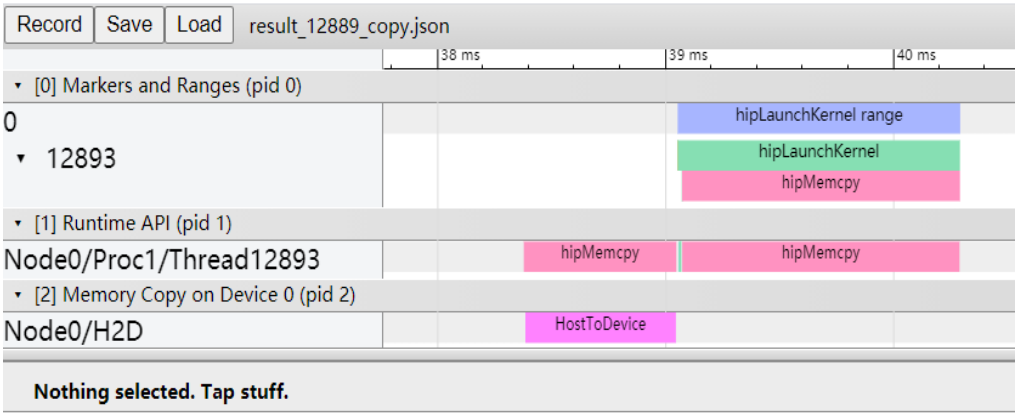


图 7.3 运行效果

7.3 多节点与多进程性能优化

hipprof 支持对 MPI 多节点多进程程序的性能分析，体现在两个方面：

- hipprofile 支持对 MPI 程序多节点多进程的性能分析；
- hipprofile 集成了 mpitracelog，可以可视化分析展示 mpi 的 trace。

具体使用方法分以下几种情况：

1.对于单节点多进程的程序

hipprof 可以直接对多进程的程序进行性能分析，包括但不限于 MPI 程序，单节点多进程 MPI 示例如下：

```
hipprof mpirun -np 4 ./testmpi
```

其中 hipprof 是性能分析命令，mpirun -np 4 是在单一节点上启动 4 个进程来运行 MPI 程序，./testmpi 是 MPI 程序的启动指令。本实例中使用了 4 个进程的 mpi 程序，其中一个 mpi 进程用于接收消息，另外三个 MPI 进程计算后发送消息给第一个 MPI 进程，因此性能分析可视化后，展示出了三个进程。左边目录可以区分节点、进程、线程、Copykind、Stream。如果需要 MPI 的 Trace 可视化信息，可以增加 mpitracelog 的指令。对应的 timeline 效果如图 7.4：

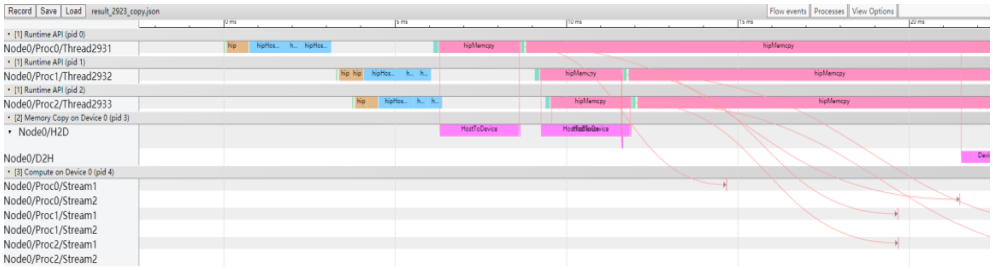


图 7.4 MPI 时间线

MPI 包含 MPItrace 的示例如下：

```
Hipprof --mpi-trace mpirun -x LD_PRELOAD=mpitracer.so:/opt/hpc/software/mpi/hpcx/v2.7.4/gcc-7.3.1/lib/libmpi.so.40 -np 4 ./testmpi
```

与之前指令对比，hipprof 增加--mpi-trace 的命令选项，hipprof 会根据此命令去解析 MPI 程序生成的 MPI trace log。mpirun 增加-X LD_PRELOAD 的设置，用来设置 mpitrace 库文件使 MPI 程序生成 MPI trace log。和前文的可视化结果对比，可视化后左边的目录结构中多了 MPI API 的目录，显示了 MPI API trace 性能数据，可视化效果如图 7.5：

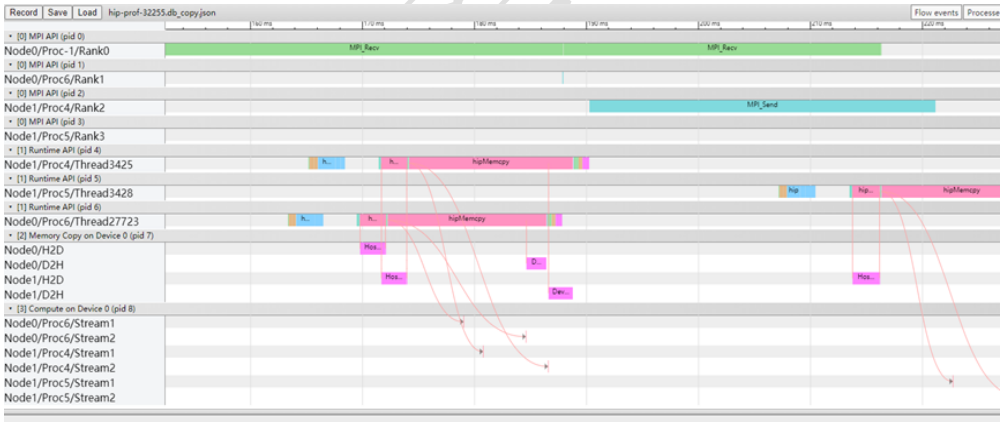


图 7.5 MPI API trace 可视化

2.对于多节点多进程的程序

多节点多进程的 MPI 程序性能分析分为两个步骤：

- hipprof 对每一个进程分别进行性能分析并生成性能分析数据文件。
- hipprof 对性能分析数据文件进行合并，生成最终的性能分析文件。

多节点多进程性能分析示例如下：

```
mpirun -x LD_PRELOAD=mpitracer.so:/opt/hpc/software/mpi/hpcx/v2.7.4/gcc-
```

```
7.3.1/lib/libmpi.so.40 -np 4 -H node1:2,node2:2 hipprof --mpi-trace ./testmpi
```

在指令中，-H 表示指定了两个节点，每个节点启动两个进程，和单节点的指令对比，hipprof 放在了 mpirun 的后面，这时候 MPI 启动的多进程，其实是启动了多个 hipprof 程序，再由 hipprof 来启动每一个 testmpi，多个 hipprof 会产生多个性能分析数据文件。此时可以执行如下指令将多个文件进行合并：

```
hipprof --db-merge ./
```

--db-merge 表示要合并多个 db 文件， ./指的是 db 文件所在的目录为当前目录，指令效果如图 7.6：

```
HIP_PROF:create new db file './hip-prof-32255.db'
HIP_PROF:merge file ./hip-prof-27716.db.
HIP_PROF:merge file ./hip-prof-27717.db.
HIP_PROF:merge file ./hip-prof-3421.db.
HIP_PROF:merge file ./hip-prof-3422.db.
-------------------------------------------------------------------------------------------------------------------------
Name                          Calls                TotalDurationNs      AverageNs            Percentage
-------------------------------------------------------------------------------------------------------------------------
hipMemcpy                     27                   150559824            5576289              233.900484672944
hipHostMalloc                 27                   22076763             817657               34.29710150106
hipStreamCreate               18                   6069690              337205               9.42949715997625
hipDeviceReset                9                    5461893              606877               8.48526111409218
hipHostFree                   27                   3250560              120391               5.04987013971593
hipStreamDestroy              18                   2906280              161460               4.51507790142425
hipMalloc                     27                   1838430              68090                2.85607180330711
hipLaunchKernel               18                   597660               33203                0.928487826006172
hipFree                       27                   156540               5797                 0.243190918386719
__hipPushCallConfiguration    18                   88710                4928                 0.137814401239848
hipGetDeviceProperties        9                    53130                5903                 0.0825395010469296
__hipPopCallConfiguration     18                   29580                1643                 0.045953669131718
hipSetDevice                  9                    18480                2053                 0.0287093916684973
-------------------------------------------------------------------------------------------------------------------------
HIP_PROF:dump json (99/99))
HIP_PROF:finish
```

图 7.6 运行效果

根据提示，文件合并后产生了新的 db 文件，下载对应的 json 文件使用 chrome://tracing/查看可视化结果。在可视化的页面中使用 Node 表示和区别不同的节点，可视化效果如图 7.7。

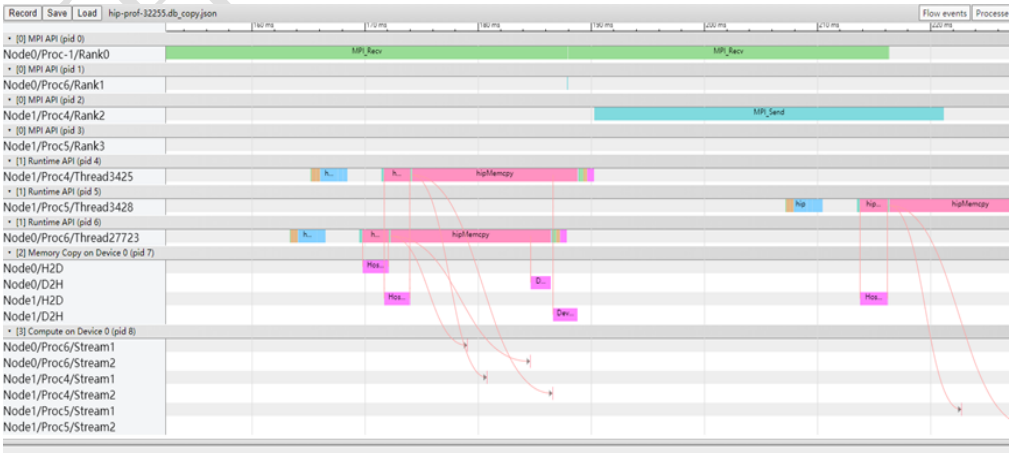


图 7.7 多节点时间线

7.4 总结

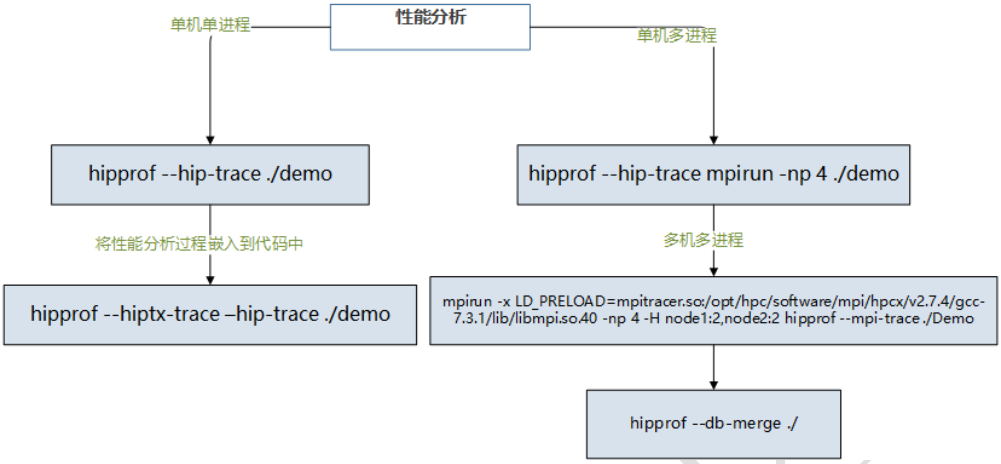


图 7.8 profile 功能汇总

开发者社区

保密声明

在此声明，该文档展示的全部技术信息及其相关内容，版权皆属于开发者社区（https://developer.sourcefind.cn/）所有。未经允许，严禁截屏、大规模传播及转发。另外，对使用该技术文档而导致任何侵犯第三方专利或其他权利的行为，开发者社区不承担任何责任。

感谢您的理解与支持。

开发者社区